Visvesvaraya Technological University
Jnana Sangama, Belagavi – 590018, Karnataka



BANGALORE INSTITUTE OF TECHNOLOGY
K. R. Road, V. V. Pura, Bengaluru – 560004
DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

PROJECT PHASE-II REPORT

on

"SETU: Multilingual Preference Distillation for Offline Indian Language SLM Translators"

Submitted in partial fulfilment of the requirements for the award of the degree of

BACHELOR OF ENGINEERING

in

ARTIFICIAL INTELLIGENCE AND MACHINE LEARNING

Submitted by

JANNAT ZAFRIN	**1BI23AI017**
PRABHAV SHARMA	**1BI23AI040**
RISHABH SHARMA	**1BI23AI047**
SINDHUJA TIWARI	**1BI23AI054**

Under the Guidance of

Prof. Shobha Y.
Associate Professor
Dept. of AI & ML, BIT, Bengaluru

Academic Year 2026–27

BANGALORE INSTITUTE OF TECHNOLOGY

K. R. Road, V. V. Pura, Bengaluru – 560004

DEPARTMENT OF ARTIFICIAL INTELLIGENCE & MACHINE LEARNING

CERTIFICATE

This is to certify that the Project Phase-II work entitled **"SETU: Multilingual Preference Distillation for Offline Indian Language SLM Translators"** is a bonafide work carried out by **Jannat Zafrin (1BI23AI017)**, **Prabhav Sharma (1BI23AI040)**, **Rishabh Sharma (1BI23AI047)** and **Sindhuja Tiwari (1BI23AI054)**, bonafide students of Bangalore Institute of Technology, in partial fulfilment for the award of the degree of **Bachelor of Engineering in Artificial Intelligence and Machine Learning** of **Visvesvaraya Technological University, Belagavi**, during the academic year **2026–27**.

It is certified that all corrections and suggestions indicated during the internal assessment have been incorporated in the report. The project report has been approved as it satisfies the academic requirements in respect of the Project Phase-II work prescribed for the said degree.

Prof. Shobha Y.	**Dr. Kempanna M.**	**Dr. Vijaya Prakash A M**
Associate Professor	Professor & Head	Principal
Guide	Dept. of AI & ML, BIT	BIT, Bengaluru
Dept. of AI & ML, BIT		

Project Coordinators: Dr. Jyothi D. G. | Dr. Shruthiba A.

External Viva

Name of the Examiners	**Signature with Date**
1. _____	_____
2. _____	_____

DECLARATION

We, the undersigned students of the Department of Artificial Intelligence and Machine Learning, Bangalore Institute of Technology, Bengaluru, hereby declare that the Project Phase-II work entitled **"SETU: Multilingual Preference Distillation for Offline Indian Language SLM Translators"** has been carried out by us under the guidance of **Prof. Shobha Y.**, Associate Professor, Department of Artificial Intelligence and Machine Learning, Bangalore Institute of Technology, Bengaluru, and submitted in partial fulfilment of the requirements for the award of the degree of Bachelor of Engineering in Artificial Intelligence and Machine Learning of Visvesvaraya Technological University, Belagavi, during the academic year 2026–27.

We further declare that the work reported in this report is original, has been carried out by us, and that the results embodied in this report have not been submitted to any other university or institute for the award of any other degree or diploma. Every external source of information, software, dataset or pre-trained model used in this work has been duly acknowledged in the text, in the references and in the acknowledgements.

Name	**USN & Signature**
JANNAT ZAFRIN	1BI23AI017
PRABHAV SHARMA	1BI23AI040
RISHABH SHARMA	1BI23AI047
SINDHUJA TIWARI	1BI23AI054

Place: Bengaluru **Date:** _____

ACKNOWLEDGEMENT

The knowledge and satisfaction that accompany the successful completion of any task would be incomplete without acknowledging the people who made it possible. Their guidance and encouragement have crowned our efforts with success.

We would like to profoundly thank the Management of **Rajya Vokkaligara Sangha** and **Bangalore Institute of Technology** for providing such a healthy environment for the successful completion of our project work.

We would like to express our sincere thanks to the Principal, **Dr. Vijaya Prakash A M**, for his encouragement and support that motivated us toward the successful completion of this project.

We express our sincere gratitude to **Dr. Kempanna M.**, Professor and Head, Department of Artificial Intelligence and Machine Learning, Bangalore Institute of Technology, for providing the infrastructure, the departmental support and the freedom to pursue an ambitious systems project.

We owe our deepest thanks to our guide, **Prof. Shobha Y.**, Associate Professor, Department of Artificial Intelligence and Machine Learning, for her patient supervision throughout this work. Her insistence that we report what we measured rather than what we hoped for is the reason this report contains a refuted hypothesis, two documented training collapses and an honest count of directions that fall short of target, alongside the results that succeeded.

We thank our Project Coordinators, **Dr. Jyothi D. G.**, Professor and Dean R & D, and **Dr. Shruthiba A.**, Assistant Professor, for the review milestones that kept the project on schedule and for their constructive criticism at every evaluation.

We are grateful to the entire teaching and non-teaching staff of the Department of Artificial Intelligence and Machine Learning for their cooperation and assistance.

This project stands on freely published research artifacts, and we acknowledge them explicitly. We thank **AI4Bharat** for the IndicTrans2 model family and the Samanantar parallel corpus, and **Dr. Raj Dabre** for releasing the ungated rotary IndicTrans2 distilled checkpoints that served as our teacher when the original checkpoints proved inaccessible. We thank the maintainers of PyTorch, Hugging Face Transformers, ONNX Runtime, Optimum, SentencePiece and sacreBLEU, without which a project of this scope could not have been completed within an academic year.

Finally, we thank our families and our classmates for their patience during the long training runs, the failed experiments and the debugging sessions that this report compresses into a few pages.

Jannat Zafrin	1BI23AI017
Prabhav Sharma	1BI23AI040
Rishabh Sharma	1BI23AI047
Sindhuja Tiwari	1BI23AI054

ABSTRACT

India recognises twenty-two scheduled languages, yet nearly every machine translation service that serves them well is a cloud service. That design excludes precisely the users who need translation most: a health worker in a village with no signal, a soldier on a border patrol, a citizen at a gram panchayat counter. Cloud translation also means every sentence a user types leaves their device. The obvious alternative, running a large translation model locally, fails on arithmetic: state-of-the-art Indic systems such as IndicTrans2 need GPU-class memory and hold billions of parameters, which no commodity phone or ARM board can host.

SETU (Sanskrit सेतु, "bridge") closes that gap by distilling a strong teacher into compact students that run entirely on the device. The project began from the hypothesis that *preference distillation*, ranking teacher hypotheses by chrF and training the student with Direct Preference Optimization (DPO), would beat standard sequence-level knowledge distillation for tiny translation models. A controlled head-to-head at matched student size refuted that hypothesis and produced the project's central empirical finding: at 100,000 training sentences, sequence-level knowledge distillation (SeqKD) on the teacher's own one-best output reached **17.09 BLEU** against **11.10 BLEU** for reference-trained DPO, an improvement of 6.0 BLEU (roughly 54 %). The bottleneck was never the optimisation objective; it was noise in the mined human references. That result redirected the entire system towards SeqKD, and the DPO machinery was retained as a measured, reported negative result rather than quietly discarded.

The delivered system trains one 52.6-million-parameter Marian student per translation direction on approximately 497,000 teacher-distilled sentence pairs, then quantises it to INT4 ONNX at **103.9 MB**, a fourfold compression from 411 MB FP32. Forty-four directions covering all twenty-two scheduled languages in both directions were trained on an NVIDIA A40 vGPU. Measured against the teacher on a held-out Samanantar development slice, **14 of 22 directions meet the ≥ 0.80 student/teacher BLEU ratio target**, with a mean

ratio of **0.838** and a median of **0.855**; the strongest direction, Marathi→English, reaches **0.955**. All three non-quality targets pass without exception: every artifact is 103.9 MB (target $\leq$ 200 MB), mean latency is 105–202 ms per sentence on commodity CPU (target $<$ 500 ms), and inference completes with all network sockets disabled. Because no Indic-to-Indic student exists, an English-pivot router chains two students, so all 22 languages translate in every direction: 44 direct models serve 506 usable directions. Two training collapses (Telugu→English and Malayalam→English, both degenerating to a fixed output string) were diagnosed as learning-rate instability at 5×10^{-4} and fully recovered at 3×10^{-4}, lifting ratios from 0.036 to 0.817 and from 0.013 to 0.898 respectively. The system ships as a FastAPI service, an offline progressive web application, a Next.js web client and a command-line tool, all sharing a single inference engine, and is validated by an automated suite of 85 tests.

Keywords: Neural Machine Translation; Sequence-Level Knowledge Distillation; Direct Preference Optimization; Small Language Models; Indic Languages; On-Device Inference; Model Quantisation; ONNX; English Pivot.

TABLE OF CONTENTS

Abstract **i**

List of Tables **v**

List of Figures **vii**

1 Introduction **1**

1.1 Overview 1

1.2 Objectives 3

1.3 Purpose, Scope and Applicability 4

1.4 Organisation of the Report 5

2 Literature Survey **7**

2.1 Introduction to the Survey 7

2.2 Summary of Papers Surveyed 8

2.3 Comparative Study of Available Technologies 13

2.4 Drawbacks of the Existing System 15

2.5 Problem Statement 16

2.6 Proposed Solution 17

3 Requirement Engineering **20**

3.1 Software and Hardware Tools Used 20

3.2 Conceptual / Analysis Modelling 23

3.3 Software Requirements Specification 28

4 Project Planning **34**

4.1 Project Planning and Scheduling 34

5 System Design **43**

5.1 System Architecture 43

5.2 Component Design / Module Decomposition 45

5.3 Interface Design 50

5.4 Data Structure Design 51
5.5 Algorithm Design 53

6 Implementation **57**
6.1 Implementation Approaches 57
6.2 Coding Details and Code Efficiency 58

7 Testing **68**
7.1 Testing Approach 68

8 Results, Discussion and Performance Analysis **78**
8.1 The Deciding Experiment: SeqKD versus Preference Distillation 78
8.2 Data Scaling of Sequence-Level Distillation 81
8.3 Training Run History and Failure Analysis 81
8.4 Training Collapse and Recovery 83
8.5 Test Reports 84
8.6 Qualitative Analysis of Translation Quality 88
8.7 User Documentation 91

9 Conclusion, Applications and Future Work **94**
9.1 Conclusion 94
9.2 Applications 96
9.3 Future Scope 97
9.4 Closing Remarks 98

References **100**

Appendix **103**
A. Paper and Publication Details 103
B. Reproducing the Results 105

Glossary **108**

LIST OF TABLES

4.1 Work breakdown structure with completion evidence 34
4.2 Schedule of activities with durations 38
4.3 PERT duration estimates for Phase 1 activities (weeks) 39
4.4 Risk register with outcomes 40
4.5 Division of responsibility 42

5.1 External interface specification 50
5.2 Core data structures 52
5.3 Algorithm 1: SeqKD training of one direction 53
5.4 Algorithm 2: routing a translation request 54
5.5 Algorithm 3: export and progressive quantisation 55
5.6 Algorithm 4: chrF-ranked preference pair construction 56

6.1 Optimisations applied and their measured effect 65

7.1 Test suite organisation 68
7.2 Unit test UT-1 to UT-5: translation engine, Hindi→English INT4 ONNX student 70
7.3 Unit tests UT-6 to UT-10: corpus cleaning filters 71
7.4 Unit tests UT-11 to UT-15: training mechanics 72
7.5 Integration tests IT-1 to IT-4: English-pivot chaining for Indic-to-Indic pairs 73
7.6 Integration tests IT-5 to IT-9: REST service against the routing layer 73
7.7 Integration tests IT-10 to IT-13: end-to-end pipeline 74
7.8 System tests ST-1 to ST-6: non-functional targets 75
7.9 Defects found during testing 76
7.10 Final test execution summary 77

8.1 The four systems compared 79
8.2 Training run history: failures, diagnoses and fixes 82
8.3 The two training collapses and their recovery 83
8.4 Aggregate statistics across all 22 directions 84

8.5 Final scorecard against the four project targets 88
8.6 Representative simple-sentence translations across directions 89
8.7 Complex-sentence probe: *"Although the weather was terrible, the farmers kept working in the fields until sunset."* 90
8.8 Counterfactual conditional probe: *"If you had told me earlier, I would have booked the tickets for the evening train."* 90
8.9 Recurring error patterns, observed in near-miss directions 91
8.10 REST API reference 93

9.1 Outcome against each stated objective 94

LIST OF FIGURES

2.1 Head-to-head at 100,000 training sentences, Hindi→English, identical 52.6 M student, evaluated on held-out human references. SeqKD (S1) wins on both measures; DPO-distill (S2) is indistinguishable from the reference-trained baseline (S0) 18

3.2 Sequence diagram: direct translation and the two-hop English pivot 24

3.4 State chart diagram: model lifecycle from raw corpus to deployed artifact, including the observed collapse and recovery transitions 25

3.5 Level-1 data flow diagram: corpus to deployed translation 26

3.6 Entity-relationship model of SETU's persistent artifacts 27

3.7 Class diagram: training classes (left) are used offline by the ML Engineer; inference classes (right) ship to the device 28

3.1 Use case diagram: End User, ML Engineer and Edge Device actors 32

3.3 Activity diagram: offline training pipeline (above) and on-device inference (below) 33

4.1 Gantt chart: both phases complete, all 22 scheduled languages delivered . . 37

4.2 PERT network for Phase 1 with the critical path highlighted 40

5.1 System architecture: SeqKD distillation and offline deployment pipeline . . 44

6.1 Effect of quantisation on size, latency and quality (Hindi→English student). INT4 is the shipped variant: it is the only precision that clears both the 200 MB and the 500 ms budget, at no measurable cost in BLEU 66

8.1 Head-to-head at 100,000 sentences, Hindi→English, matched student size, evaluated on human references (teacher BLEU 28.16). SeqKD (S1) gains $+6.14$ BLEU over the reference-trained baseline S0; DPO-distill (S2) gains $+0.15$ 79

8.2 SeqKD data-scaling curve, Hindi→English. The ratio rises steeply to 250,000 sentences and then flattens as the student approaches its teacher's ceiling, crossing the 0.80 target at 500,000 81

8.3 Student/teacher BLEU ratio for all 22 evaluated directions on the 500-sentence Samanantar development slice. Fourteen clear the 0.80 target; the dashed line marks the threshold 85

8.4 Per-language ratio in both directions for all eleven languages, ordered by their two-direction mean. The two directions track closely, so the method is not biased towards translating into English 86

8.5 Median and p90 end-to-end latency for the 18 deployed directions (INT4 ONNX on the VM host CPU), ordered by p90. Every median sits well inside the 500 ms budget; only English→Tamil crosses it at p90 87

CHAPTER 1

INTRODUCTION

This chapter establishes the setting in which SETU was built. It describes the linguistic and infrastructural problem that motivates an offline Indic translator, summarises the state of the art the project builds upon, states the objectives that the system was measured against, and closes with a guide to the remaining chapters of this report.

1.1 Overview

The Eighth Schedule of the Constitution of India recognises twenty-two languages. Together with several hundred dialects, these languages define the everyday linguistic reality of more than a billion people. Digital services, however, remain overwhelmingly English-first. The gap between the language a citizen speaks and the language a service responds in is not a matter of convenience; it decides whether a person can read a government form, understand a prescription, or follow an advisory during a disaster.

Machine translation (MT) is the standard technical answer to that gap, and for Indian languages the answer has become a very good one. IndicTrans2 [12] covers all twenty-two scheduled languages at a quality that was unthinkable five years ago, trained on the Bharat Parallel Corpus Collection (BPCC) and evaluated on the IN22 and FLORES-200 benchmarks. Earlier multilingual efforts (mT5-based Indic systems [13], Indic-to-Indic pivoting systems [17], and one-to-many low-resource systems for Assamese and Bengali [16]) established that shared multilingual representations transfer usefully across related Indian languages.

Every one of those systems, however, shares an architectural assumption that this project rejects: that the model runs in a data centre and the user's text travels to it. Two consequences follow. The first is availability. A cloud translator is useless exactly where translation matters most: the rural primary health centre with intermittent connectivity, the border post beyond cellular coverage, the relief camp after a cyclone has taken the towers down. The second is privacy. A cloud translator sees every sentence a user submits. For a citizen dictating a medical complaint or a legal grievance, that is a meaningful disclosure, and it is a disclosure with no technical necessity.

The natural alternative is to run the model on the device. Here the arithmetic is unforgiving. The flagship IndicTrans2 checkpoints hold roughly a billion parameters and

occupy several gigabytes in 32-bit floating point; even the distilled 200-million-parameter variants need close to a gigabyte of resident memory. A mid-range Android handset or an ARM Cortex-A55 single-board computer cannot host either. Closing the gap therefore requires a model one order of magnitude smaller that nevertheless retains most of the teacher's translation quality.

Model compression research offers several routes to such a model. Knowledge distillation trains a compact student to imitate a larger teacher; DistilBERT [1] demonstrated the approach for encoders, and for translation specifically, sequence-level knowledge distillation replaces human reference targets with the teacher's own decoded output. SMaLL-100 [3] showed that a deliberately shallow multilingual translator can remain competitive for low-resource languages. Work on neural translation for mobile devices [2] and on shallow-to-deep training [5] addressed the latency and stability problems that arise when translation models are shrunk. In parallel, Direct Preference Optimization (DPO) [9] made preference-based training practical without a reward model or reinforcement learning, and subsequent work applied it to neural machine translation with minimum Bayes risk decoding [15].

SETU (from the Sanskrit सेतु, meaning *bridge*) sits at the intersection of these lines of work. It takes IndicTrans2 as a teacher and distils it into compact students of 52.6 million parameters, one per translation direction, quantised to INT4 ONNX and executed by ONNX Runtime with a local SentencePiece tokenizer. Nothing leaves the device at inference time.

The project's original thesis was that preference distillation would be the mechanism: generate several teacher hypotheses per source sentence, rank them by chrF, pair the best against the rest, and train the student with DPO. That thesis was tested directly against the standard sequence-level knowledge distillation baseline at matched student size, and it lost decisively: 17.09 BLEU for sequence-level distillation against 11.10 BLEU for the preference-trained student. The diagnosis, developed in Chapter 8, is that the limiting factor for a compact Indic translator is not the training objective but the noise in web-mined human references. Training on the teacher's clean one-best output removes that noise, and the student improves accordingly. This report presents that refutation as a result in its own right, because a negative result obtained under controlled conditions is more useful to the field than an unexamined assumption.

The delivered system trains each direction on approximately 497,000 teacher-distilled sentence pairs, quantises the result to 103.9 MB, and serves it through a REST API, an offline progressive web application, a browser client and a command-line tool. All twenty-two scheduled languages are covered in both directions. Because no ungated Indic-to-Indic teacher was available, pairs such as Hindi–Bengali are served by chaining two students through English, which turns 44 trained models into 506 usable translation directions.

1.2 Objectives

The project set out to design and deliver a multilingual distillation pipeline that converts a large Indic translation teacher into compact, quantised students capable of fully offline inference on edge hardware; to determine empirically whether preference-based distillation or sequence-level knowledge distillation produces the better compact student at matched size; to cover a substantial subset of the twenty-two scheduled languages in both directions and extend coverage to every Indic-to-Indic pair without training a quadratic number of models; and to expose the resulting engine through interfaces that a non-specialist can actually use.

Stated as measurable targets, the objectives are:

O1. Quality. Each student shall achieve a BLEU score at least 80 % of its teacher's BLEU on the same held-out evaluation set, i.e. $\mathrm{BLEU}_{\text{student}}/\mathrm{BLEU}_{\text{teacher}} \geq 0.80$.

O2. Latency. Translation of a single sentence shall complete in under 500 ms on ARM-class CPU hardware, with no GPU and no accelerator.

O3. Size. The deployable quantised artifact for one direction shall not exceed 200 MB.

O4. Offline operation. Inference shall succeed with all network sockets disabled; no user text shall leave the device.

O5. Empirical finding. The preference-distillation hypothesis shall be tested against a sequence-level knowledge distillation baseline at matched student size, on identical data, with evaluation on real human references, and the outcome reported whichever way it falls.

O6. Coverage and extensibility. A substantial subset of the scheduled languages shall be supported bidirectionally, all Indic-to-Indic pairs shall be reachable, and adding a new language shall be a configuration and data change rather than a code change.

Objectives O2, O3 and O4 are met by every trained direction without exception. Objective O1 is met by 14 of 22 directions; the remaining eight fall between 0.755 and 0.787, and Chapter 8 analyses why. Objective O5 produced a refutation of the project's own founding hypothesis. Objective O6 is met in full: all twenty-two scheduled languages are supported bidirectionally.

1.3 Purpose, Scope and Applicability

1.3.1 Purpose

The purpose of SETU is to make high-quality Indic machine translation available where the network is not, and to do so without requiring the user to disclose what they are translating.

Three concerns motivate the work. The first is *access*. Connectivity in India is neither uniform nor reliable, and the populations least well served by it overlap heavily with the populations least well served in English. A translator that requires bandwidth systematically fails the users with the greatest need for it. The second is *privacy*. Translation input is frequently sensitive: symptoms, grievances, legal matters, personal correspondence. On-device inference makes the privacy guarantee structural rather than contractual: there is no server to trust because there is no server. The third is *sovereignty of infrastructure*. A public institution that depends on a commercial translation API is exposed to pricing changes, API deprecations and terms-of-service revisions. A 104 MB artifact that runs on the institution's own hardware is not.

Theoretically, the project addresses a question that the compact-model literature has not settled for translation: when a student is an order of magnitude smaller than its teacher and the available parallel data is mined rather than curated, which training signal should the student be given? The literature offers reference targets, teacher one-best targets and preference-ranked teacher hypotheses. SETU evaluates all three under matched conditions and reports the outcome.

1.3.2 Scope

Methodology. The system distils the ungated rotary IndicTrans2 distilled-200M checkpoints into 52.6-million-parameter Marian sequence-to-sequence students, one per direction, using sequence-level knowledge distillation on teacher one-best output. Students are quantised post-training to INT8 and INT4 ONNX and executed by ONNX Runtime. Evaluation uses sacreBLEU (BLEU and chrF) against real human references held out from the training corpus, with the student/teacher BLEU ratio as the headline metric.

Languages delivered. All twenty-two scheduled languages (Assamese, Bengali, Bodo, Dogri, Gujarati, Hindi, Kannada, Kashmiri, Konkani, Maithili, Malayalam, Manipuri, Marathi, Nepali, Odia, Punjabi, Sanskrit, Santali, Sindhi, Tamil, Telugu and Urdu), each paired bidirectionally with English, giving 44 trained directions. Every Indic-to-Indic pair among them is served by chaining two students through English.

Assumptions. The work assumes that the ungated distilled-200M teacher is an acceptable quality ceiling; that Samanantar, though web-mined and noisy, is representative

enough for distillation; that the student architecture and its 16,000-token vocabulary are sized correctly for the data volume available; and that x86 CPU latency measurements are a reasonable proxy for ARM behaviour. Each assumption is examined in Chapter 8.

Limitations, stated up front. The project is text-to-text only; speech input and output are out of scope. The quality ratio is bounded by the ungated 200M teacher, which is itself weaker than the gated one-billion-parameter flagship. Reported ratios are computed on an internal Samanantar development slice rather than on FLORES-200, so they are comparable within this report but not directly comparable to published IndicTrans2 numbers. ARM latency is projected from x86 measurement rather than measured on ARM silicon. Finally, INT4 quantisation yields the same on-disk size as INT8 in ONNX Runtime's CPU execution provider, which lacks a true 4-bit kernel.

1.3.3 Applicability

Direct applications. In rural healthcare, a doctor or ASHA worker can converse with a patient across a language boundary on a handset with no connectivity. In defence and border operations, patrol units operating beyond cellular coverage can translate without transmitting anything. In education, multilingual tutoring tools can serve students in their own language without a per-query cloud cost. In government service delivery, forms, notices and entitlement documents can be translated at a gram panchayat counter. In disaster relief, field workers can communicate with affected communities during the exact window in which the network is down.

Indirect contributions. The distillation pipeline is not Indic-specific; the same configuration-driven process applies to any language family with a strong teacher and a weak-reference parallel corpus. The controlled comparison between preference distillation and sequence-level distillation contributes a data point to the compact-NMT literature that practitioners can act on. The English-pivot router demonstrates that n languages can be served in n^2 directions with $2n$ models, a result of practical importance when each additional model costs 104 MB of device storage. And the engineering record in this report (particularly the two training collapses and their diagnosis) documents failure modes that are rarely published but routinely encountered.

1.4 Organisation of the Report

Chapter 2, Literature Survey, reviews twenty papers published from 2019 onward across mobile and compact NMT, knowledge distillation, preference optimisation and multilingual Indic translation, compares the available technologies, and derives the drawbacks of existing systems, the problem statement and the proposed solution.

Chapter 3, Requirement Engineering, specifies the hardware and software used for training and deployment, presents the conceptual models (use case, sequence, activity, state chart, data flow, entity relationship and class diagrams) and states the user, system, functional, non-functional and domain requirements.

Chapter 4, Project Planning, gives the work breakdown structure, the Gantt schedule across both project phases, a PERT network with the critical path, the risk register and the division of responsibility within the team.

Chapter 5, System Design, presents the high-level architecture, decomposes the system into eight modules, specifies the external interfaces and the core data structures, and gives the algorithms for distillation, training, quantisation and pivot routing.

Chapter 6, Implementation, describes the implementation approach and coding standards, presents the significant code with commentary, and discusses the optimisations that made training feasible on the available hardware.

Chapter 7, Testing, sets out the testing strategy and reports the unit, integration and system test cases with their data and outcomes.

Chapter 8, Results, Discussion and Performance Analysis, reports the full 22-direction scorecard, the preference-versus-sequence-distillation head-to-head, the data-scaling curve, the collapse and recovery analysis, qualitative translation samples and the user documentation.

Chapter 9, Conclusion, Applications and Future Work, summarises the outcome against each objective, lists applications and limitations, and defines the work that follows.

The report closes with **References**, an **Appendix** covering the publication plan and reproduction instructions, and a **Glossary**.

CHAPTER 2

LITERATURE SURVEY

This chapter surveys the research that SETU builds on and reacts against. The survey has three purposes. First, it establishes what is technically available for compressing a translation model to edge size, for training a compact student from a strong teacher, and for covering many Indian languages at once. Second, it compares those available technologies so that the choices made in Chapter 5 can be justified rather than asserted. Third, it isolates the specific capability that none of the surveyed systems provides: offline, on-device, multi-directional Indic translation within 200 MB, which becomes the problem statement of this project.

Twenty papers published from 2019 onward are reviewed, ordered by year of publication. They fall into five groups: compact and mobile neural machine translation, knowledge distillation, preference optimisation and alignment, multilingual and Indic-specific translation, and offline or edge deployment.

2.1 Introduction to the Survey

Neural machine translation converged on the Transformer architecture, and with it on a scaling regime in which quality is bought with parameters and data. That regime produced IndicTrans2, which translates all twenty-two scheduled Indian languages at high quality but requires GPU-class hardware. The research directions that matter for this project are the ones that ask what can be recovered after the parameters are taken away again.

Three lines of work answer that question. Compression research (pruning, quantisation, shallow architectures) reduces a trained model to a size a device can hold. Distillation research transfers behaviour from a large teacher to a small student, either by matching output distributions or by training the student on the teacher's decoded output. Alignment research, developed for large language models, replaces a single reference target with a preference signal over several candidate outputs. The central question that this project settles empirically is which of the latter two gives a very small translation student the better training signal when the available parallel data is web-mined and noisy.

A fourth line, multilingual translation, matters for coverage. Shared multilingual models exploit transfer between related languages, which is valuable for the Indian language family, but they also introduce interference between directions and pivoting error when no direct model exists.

2.2 Summary of Papers Surveyed

Each entry below gives the paper, its authors and where it appeared, followed by what it contributes and where it falls short for this project's purposes.

[1] DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter, V. Sanh *et al.*, arXiv, 2019

- Halves the layer count of BERT via distillation while retaining about 97 % of language-understanding performance, and establishes triple-loss distillation as a practical recipe.
- **Drawback:** demonstrated for encoders on classification, not for autoregressive sequence generation.
- A measurable accuracy loss remains on complex tasks.

[2] Neural Machine Translation on Mobile Devices, B. Li *et al.*, IEEE, 2020

- Compresses NMT models for mobile and edge deployment, cutting memory footprint and inference latency to make real-time on-device translation feasible.
- **Drawback:** the reduction techniques degrade translation quality.
- The degradation is most visible on long and syntactically complex sentences.

[3] SMaLL-100: Introducing Shallow Multilingual Machine Translation for Low-Resource Languages, A. Mohammadshahi *et al.*, Springer, 2020

- A compact multilingual translator covering many low-resource directions with substantially lower memory use and faster inference than M2M-100.
- **Drawback:** quality still trails the large model it compresses, particularly on high-resource pairs where capacity matters.

[4] The Tatoeba Translation Challenge – Realistic Data Sets for Low Resource and Multilingual MT, J. Tiedemann, WMT, 2021

- Releases a benchmark spanning 555 languages, enabling systematic evaluation of low-resource, zero-shot and few-shot translation on realistic data.
- **Drawback:** the data is noisy and inconsistent across languages.
- Exploiting it fully still requires large-capacity models.

[5] Shallow-to-Deep Training for Neural Machine Translation, B. Li *et al.*, ACL, 2021

- Introduces incremental shallow-to-deep training, stacking Transformer layers progressively to improve convergence speed and training stability at depth.
- **Drawback:** the final models remain computationally heavy and the staged schedule adds engineering complexity.

[6] Nearest Neighbor Knowledge Distillation for Neural Machine Translation, Z. Yang, R. Sun, X. Wan, IEEE, 2022

- Uses k-nearest-neighbour retrieval over the training set to supply additional supervision, improving quality in low-resource conditions.
- **Drawback:** the datastore is memory-intensive and retrieval slows both training and inference, which is disqualifying for edge deployment.

[7] Speech-to-Speech Translation Improvements with Self-Supervised Learning, C. Xu *et al.*, IEEE, 2022

- Combines self-supervised pre-training, multitask learning and synthetic data to obtain large BLEU gains in speech-to-speech translation, especially for low-resource pairs.
- **Drawback:** depends on large-scale pre-training and a multi-stage pipeline that is impractical to deploy on a device.

[8] Direct Speech Translation, Y. Jia *et al.*, Springer, 2023

- Surveys end-to-end speech translation and characterises its principal obstacles: data scarcity, latency and bias.
- **Drawback:** a survey rather than a system; no concrete model or experimental validation is offered.

[9] Direct Preference Optimization: Your Language Model is Secretly a Reward Model, R. Rafailov *et al.*, NeurIPS, 2023

- Reformulates preference alignment as a simple classification loss over preferred and dispreferred responses, removing the separate reward model and the reinforcement-learning loop of RLHF, with better stability and lower cost.
- **Drawback:** effectiveness depends heavily on the quality and diversity of the preference data; the paper studies alignment of large language models, not distillation into small ones.

[10] All-in-One Knowledge Distillation Framework for Neural Machine Translation, Z. Miao *et al.*, EMNLP, 2023

- Distils into several student models simultaneously within one framework, improving training efficiency and enabling scalable multilingual distillation.
- **Drawback:** joint optimisation across students creates gradient conflicts.
- Low-resource directions are the ones that lose.

[11] Continual Knowledge Distillation for Neural Machine Translation, Y. Zhang, P. Li, M. Sun, Y. Liu, ACL, 2023

- Adds new languages to an existing multilingual model incrementally without forgetting previously learned ones, supporting long-lived multilingual deployments.
- **Drawback:** training complexity grows with the number of tasks and the stability-plasticity balance requires careful tuning.

[12] IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for All 22 Scheduled Indian Languages, J. Gala *et al.*, TMLR, 2023

- Delivers open translation models covering all twenty-two scheduled Indian languages, together with the BPCC corpus and the IN22 benchmark; the current reference point for Indic MT.
- **Drawback:** the flagship models need GPU-class compute and multiple gigabytes of memory, which rules out offline edge deployment.

[13] Multilingual Indian Language Neural Machine Translation System Using mT5 Transformer, A. Jha *et al.*, IEEE, 2023

- Fine-tunes mT5 for Indian-language translation and reports strong BLEU on the directions covered, confirming that large pre-trained multilingual encoders transfer to Indic data.
- **Drawback:** evaluated on only a few languages and built on a large pre-trained backbone, so it scales neither to all 22 languages nor to on-device inference.

[14] IWSLT 2024 Low-Resource Speech Translation Submissions, M. Zafar *et al.*, IWSLT, 2024

- Shows that cascaded ASR-then-MT pipelines still outperform end-to-end speech translation in genuinely low-resource settings, and reports competitive BLEU.
- **Drawback:** cascading compounds recognition errors into translation errors; end-to-end alternatives remain data-starved.

[15] Direct Preference Optimization for Neural Machine Translation with Minimum Bayes Risk Decoding, G. Yang, J. Chen, W. Lin, B. Byrne, NAACL, 2024

- Applies DPO to NMT using MBR decoding to construct preference pairs, showing that preference learning can improve translation quality with relatively little data.
- **Drawback:** evaluation does not extend to extremely low-resource languages.
- The models studied are far larger than an edge-deployable student.

[16] Multilingual NMT System for English to Low-Resource Indic Languages (Assamese and Bengali), K. Kashyap, S. K. Sarma, IEEE, 2024

- A one-to-many multilingual system that improves low-resource quality through shared representations and back-translated synthetic data.
- **Drawback:** synthetic data introduces its own noise.
- Coverage is limited to two languages in one direction.

[17] Multilingual Neural Machine Translation System for Indic-to-Indic Languages, S. B. Das *et al.*, ScienceDirect, 2024

- Translates directly among several Indic languages using shared models and pivoting, addressing a direction pair largely ignored by English-centric systems.
- **Drawback:** error propagates through the pivot language and accuracy degrades relative to direct models.

[18] SURYA-TAC: LLM-Based Offline Portable Speech-to-Speech Translation System, P. S. Bohra, K. Berwal, IEEE, 2024

- Proposes an offline, edge-deployed tactical translation system for use in harsh environments without connectivity, the closest published motivation to this project.
- **Drawback:** largely conceptual, without real-world validation.
- Constrained by data scarcity for the target languages.

[19] Latency-Aware Simultaneous Machine Translation Using Large Language Models, B. Fu *et al.*, Springer, 2024

- Introduces latency-aware token generation so that large language models can perform simultaneous translation while maintaining accuracy.
- **Drawback:** computationally expensive and premised on models far too large for a handset.

[20] Offline English-to-Hindi Statistical Machine Translation System, S. Gangwar, I. Naim, ScienceDirect, 2024

- Implements genuinely offline English-to-Hindi translation using the Moses SMT toolkit, demonstrating that offline operation is achievable at low cost.
- **Drawback:** statistical MT is substantially weaker than neural approaches.
- The system covers a single direction of a single language pair.

2.2.1 Discussion of Key Papers

Four of the surveyed papers determined the shape of this project and merit closer discussion.

IndicTrans2 [12] is the teacher. Its contribution is not only the models but the surrounding artifacts: the BPCC corpus and the IN22 benchmark, which together made Indic MT reproducible. SETU adopts its script-qualified FLORES language codes (`hin_Deva`, `eng_Latn`) as internal identifiers so that student tokenizer tags remain stable as languages are added. The project's quality target (80 % of teacher BLEU) is defined relative to this system. A practical constraint arose here: the official `ai4bharat` checkpoints and the BPCC dataset are access-gated on Hugging Face, returning HTTP 401 without an authenticated account that has accepted the terms. The project therefore uses the ungated rotary distilled-200M checkpoints released by IndicTrans2 co-author Raj Dabre, and the ungated Samanantar corpus. This lowers the achievable ceiling, and Chapter 8 quantifies by how much.

Direct Preference Optimization [9] supplied the project's original hypothesis. Its key insight (that the optimal policy under a Bradley-Terry preference model has a closed form, so preference learning reduces to a classification loss on preferred and dispreferred pairs) makes preference training cheap enough to attempt on a student model. SETU implements DPO with a frozen supervised-fine-tuning reference model, chrF-ranked teacher hypotheses as preferred candidates, and k-nearest-neighbour retrieved translations as additional negatives. The implementation works as specified: the margin accuracy reaches 1.0, meaning the trained policy ranks every preferred candidate above its dispreferred counterpart. It nonetheless fails to improve translation quality over sequence-level distillation, for reasons developed in Section 2.6 and Chapter 8.

Sequence-level knowledge distillation, introduced by Kim and Rush [21] and instantiated in this project's baseline, replaces the human reference target with the teacher's own one-best decoded output. It is the approach that won. The mechanism is best understood as denoising: the teacher's output is internally consistent and drawn from a single distribution, whereas web-mined references are inconsistent in register, alignment quality and fidelity. A 52-million-parameter student has no spare capacity to average over that inconsistency.

DistilBERT [1] and **SMaLL-100** [3] between them establish the size-quality frontier that this project operates on. DistilBERT shows that roughly 97 % of a large model's performance survives halving its depth; SMaLL-100 shows the same for multilingual translation specifically. Both suggest that a compact student can retain most of a teacher's ability, which is what the 0.80 ratio target formalises.

2.3 Comparative Study of Available Technologies

This section compares the candidate technologies for each design decision the project faced, and records the choice made together with its justification.

Each design decision below lists the alternatives that were considered, the trade-off between them, and the choice the project made with its reason.

Teacher model

Alternatives considered: IndicTrans2 1B; IndicTrans2 distilled-200M; mT5-based Indic NMT; NLLB-200. *Trade-off:* The 1B flagship gives the highest ceiling but is gated and needs roughly 5 GB for weights alone; mT5 covers fewer Indic languages. **Ungated rotary IndicTrans2 distilled-200M.** Accessible without an authenticated account, fits available memory, and covers all 22 scheduled languages. The lower ceiling is reported rather than hidden

Training signal for the student

Alternatives considered: Human references (plain SFT); teacher one-best (SeqKD); chrF-ranked preference pairs with DPO; SeqKD followed by DPO. *Trade-off:* References are abundant but noisy; teacher output is clean but caps the student at teacher quality; preference pairs add ranking information but need a reliable ranker. **SeqKD on teacher one-best**, selected by the controlled experiment in Section 2.6: 17.09 BLEU against 11.10 for reference-trained DPO at matched size

Student architecture

Alternatives considered: Marian encoder–decoder; distilled Transformer; shallow multilingual (SMaLL-100 style); decoder-only LM. *Trade-off:* Encoder–decoder remains the strongest inductive bias for translation; decoder-only needs far more parameters for equivalent quality. **Marian-style Transformer, 6 encoder and 6 decoder layers,** $d = 512$**, 52.6 M parameters.** Quantises to 103.9 MB, comfortably inside the 200 MB budget

Vocabulary

Alternatives considered: Shared 32,000-token SentencePiece; 16,000-token SentencePiece; byte-level BPE. *Trade-off:* A larger vocabulary shortens sequences but leaves rare tokens undertrained on a corpus of this size. **16,000-token SentencePiece.** A 32,000-token vocabulary was measured to produce degenerate decoding on this data volume (Section 8.3)

Multilingual strategy

Alternatives considered: One model per direction; one shared multilingual model; shared model with All-in-One KD. *Trade-off:* Per-direction models avoid interference and permit independent quality gates; a shared model saves storage but suffers low-resource interference [10]. **One student per direction**, with the All-in-One KD mechanism implemented and tested for future consolidation

Indic-to-Indic coverage

Alternatives considered: Train $n(n-1)$ direct models; pivot through English; use a gated Indic-to-Indic teacher. *Trade-off:* Direct models scale quadratically at 104 MB each; pivoting doubles latency and can compound errors; the Indic-to-Indic teacher is gated. **English pivot.** $2n$ models serve n^2 directions; measured two-hop latency remains around 50–65 ms

Language expansion

Alternatives considered: Full retraining; All-in-One KD; Elastic Weight Consolidation [11]. *Trade-off:* Retraining is prohibitive; EWC protects old languages via a Fisher-weighted penalty. **EWC implemented and validated** as the incremental path; per-direction training used for Phase 1

Quantisation and runtime

Alternatives considered: PyTorch on-device; TensorFlow Lite; ONNX Runtime with INT8/INT4; GGML. *Trade-off:* ONNX Runtime has mature CPU kernels, a small footprint and wide platform support; its CPU provider lacks a true 4-bit kernel. **ONNX Runtime with progressive INT8 then INT4 export via Optimum.** 411 MB FP32 compresses to 103.9 MB

Evaluation metric

Alternatives considered: BLEU; chrF/chrF++; COMET; human evaluation. *Trade-off:* BLEU is standard but penalises agglutinative languages; COMET correlates better with human judgement but needs a neural model at evaluation time. **BLEU and chrF via sacreBLEU**, reported as a student/teacher ratio; COMET implemented for future benchmarking

2.3.1 Why These Technologies Were Selected

The selection logic follows from the four objectives in Section 1.2 treated as hard constraints rather than aspirations.

The 200 MB size budget eliminates every alternative that keeps a teacher-sized model on the device, and therefore mandates distillation into a student of roughly 50 million parameters. That parameter budget in turn eliminates retrieval-augmented approaches such as nearest-neighbour distillation [6], whose datastore alone would exceed the budget.

The 500 ms latency budget on CPU eliminates beam search at deployment (the deployed engine decodes greedily, with beam search reserved for offline evaluation) and eliminates any method requiring a retrieval lookup per token.

The offline requirement eliminates any runtime that fetches weights, tokenizer artifacts or telemetry, which is why the tokenizer is a local SentencePiece model rather than a Hugging Face identifier resolved at load time.

The quality target, finally, is what forced the empirical comparison. Both sequence-level distillation and preference distillation were plausible routes to 80 % of teacher BLEU, and the literature did not settle the question for students this small on data this noisy. The project therefore built both and measured them.

2.4 Drawbacks of the Existing System

The survey exposes five drawbacks that, taken together, leave the offline Indic translation problem unsolved.

Cloud dependency. Almost every high-quality MT system in production requires an active network connection. IndicTrans2 [12], mT5 based systems [13] and LLM-based simultaneous translators [19] all assume server-side execution. In regions with intermittent connectivity, which is where the language barrier is most acute, these systems are unavailable exactly when needed. Cloud execution also means that every translated sentence is transmitted to and processed by a third party.

High compute requirements. The models that achieve state-of-the-art Indic quality hold hundreds of millions to billions of parameters and need GPU-class memory. The In-

dicTrans2 flagship requires roughly 5 GB for weights alone. Mid-range handsets and ARM single-board computers cannot host them, and the compression literature [2, 3] consistently reports quality degradation as the price of shrinking them.

Error propagation in cascaded pipelines. Systems that chain automatic speech recognition to translation [14] compound recognition errors into translation errors. The IWSLT 2024 results show cascades still outperforming end-to-end models in low-resource settings, which means the compounding problem is currently unavoidable in that design.

Narrow language coverage. Compact and specialised systems cover very few languages. The mT5-based Indic system [13] evaluates on a handful; the Assamese-Bengali system [16] covers two in one direction; the offline SMT system [20] covers a single direction of a single pair. Systems with broad coverage are precisely the ones too large to deploy.

Data scarcity for low-resource languages. Languages such as Kashmiri, Dogri, Konkani, Bodo and Santali lack parallel corpora at the scale neural translation requires. Where corpora do exist they are frequently web-mined and noisy, as the Tatoeba challenge [4] documents. This project encountered the problem directly: only eleven of the twenty-two scheduled languages had ungated data sufficient to train against.

2.5 Problem Statement

> *There is a need for a lightweight, offline-capable multilingual translation framework that delivers high-quality translation across the scheduled languages of India on edge devices (retaining at least 80 % of a strong teacher's BLEU within a 200 MB artifact and under 500 ms of CPU latency per sentence, with zero network calls at inference) and that supports incremental addition of new languages without catastrophic forgetting and without a quadratic growth in the number of trained models.*

The overall problem decomposes into six sub-problems, each addressed by an identifiable part of the system.

P1: Corpus quality. The available ungated parallel data (Samanantar) is web-mined, and contains misaligned pairs, length-ratio anomalies, wrong-script segments and duplicates. *Sub-problem:* construct a cleaning pipeline that yields a corpus a 52-million-parameter student can learn from. Addressed by the Corpus Loader module (Section 5.2.1).

P2: Choice of training signal. It is not established which signal (human references, teacher one-best output, or preference-ranked teacher hypotheses) best trains a student an order of magnitude smaller than its teacher on noisy data. *Sub-problem:* construct all three and compare them under matched conditions. Addressed by the Teacher Distiller and Trainer

modules and reported in Section 8.1.

P3: Training stability. A Transformer trained from scratch at this scale is unstable: three distinct failure modes (empty output, unconditional decoding, and mid-training collapse) were encountered. *Sub-problem:* identify and eliminate each. Addressed in Section 8.3.

P4: Deployment footprint. A 52.6-million-parameter FP32 model occupies 411 MB, over twice the size budget, and decodes too slowly in PyTorch on CPU. *Sub-problem:* compress and accelerate without materially losing quality. Addressed by the Quantiser module (Section 5.2.4).

P5: Indic-to-Indic coverage. No ungated Indic-to-Indic teacher exists, and training direct models for every pair would require $n(n-1)$ models at 104 MB each. *Sub-problem:* serve every pair without quadratic growth. Addressed by the English-pivot router (Section 5.2.5).

P6: Extensibility. Adding a language must not require retraining everything or degrade the languages already supported. *Sub-problem:* provide a mechanism for incremental expansion with measured retention. Addressed by the All-in-One KD and Elastic Weight Consolidation implementations (Section 5.2.8).

2.6 Proposed Solution

SETU proposes a four-stage pipeline (distil, train, quantise, serve) instantiated once per translation direction and bound together by a routing layer that pivots through English.

Stage 1: Teacher distillation. The ungated IndicTrans2 distilled-200M teacher decodes every source sentence in the cleaned corpus, and its one-best output replaces the human reference as the training target. This is sequence-level knowledge distillation in the sense of Kim and Rush [21]. Greedy decoding is used, which is four to five times faster than beam search and produces targets of equivalent utility.

Stage 2: Student training. A 52.6-million-parameter Marian encoder–decoder with a 16,000-token SentencePiece vocabulary is trained from scratch on the distilled corpus with token-level cross-entropy, padding masked out of the loss, linear learning-rate warmup and decay, label smoothing of 0.1, dropout of 0.2 and gradient-norm clipping at 1.0. Every one of these components was added in response to a measured failure, not adopted by default; Section 8.3 gives the history.

Stage 3: Quantisation. The trained student is exported to ONNX, validated against ONNX Runtime, and quantised progressively to INT8 and then INT4, with latency and size benchmarked at each stage. The result is a 103.9 MB artifact, a fourfold compression from 411 MB.

Stage 4: Offline serving. An inference engine loads the quantised ONNX model and a local SentencePiece tokenizer and decodes greedily. A router resolves each request to a directly trained model where one exists, and otherwise chains two students through English. One engine serves a REST API, a command-line tool, an offline progressive web application and a browser client.

2.6.1 The Deciding Experiment

The project's founding hypothesis was H1: *preference distillation produces better compact students than sequence-level knowledge distillation*. It was tested at matched student size, on identical data, with evaluation on real human references held out from training, a design that is fair regardless of what each system trained on. Figure 2.1 gives the outcome.

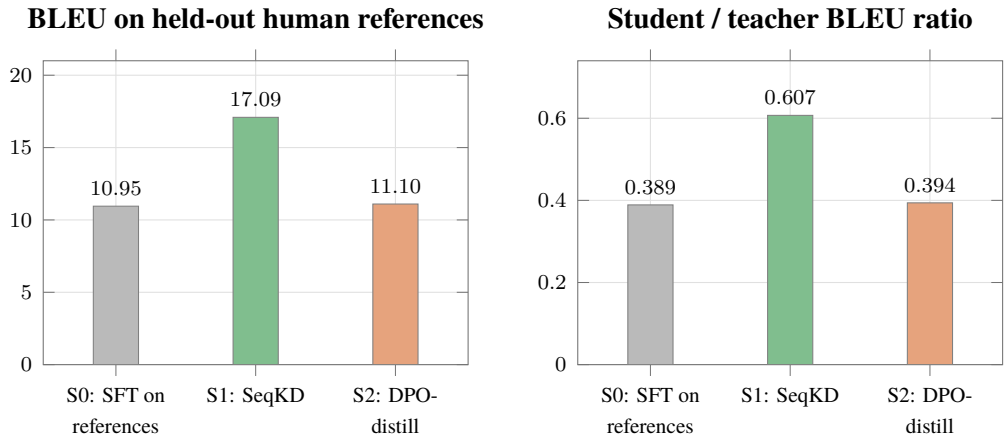


Figure 2.1: Head-to-head at 100,000 training sentences, Hindi→English, identical 52.6 M student, evaluated on held-out human references. SeqKD (S1) wins on both measures; DPO-distill (S2) is indistinguishable from the reference-trained baseline (S0)

H1 is refuted. Sequence-level distillation beats preference distillation by 6.0 BLEU, an improvement of about 54 %, and DPO applied on top of a reference-trained base adds 0.15 BLEU over that base, indistinguishable from noise. The replacement hypothesis, H1′, is supported by every subsequent result: *reference noise, not the training objective, is what caps a compact Indic student.* Teacher-distilled targets remove that noise, and the student then continues to improve with data: the ratio rises from 0.607 at 100,000 sentences to 0.764 at 250,000 and 0.806 at 500,000, whereas reference-trained students plateau near 0.66 regardless of data volume.

The DPO implementation is not defective, which is what makes the result informative. Its margin accuracy reaches 1.0, meaning it ranks preferred above dispreferred candidates perfectly; it consistently improves chrF by one to two points while costing about one BLEU point. The preference signal is being learned correctly; it simply is not the signal that a 52-million-parameter student most needs when its reference targets are noisy. SETU

therefore ships SeqKD, and reports DPO as a controlled negative result.

CHAPTER 3

REQUIREMENT ENGINEERING

Requirement engineering for SETU is complicated by the fact that the system has two distinct execution environments with almost nothing in common. Training runs once per direction on a datacentre GPU with tens of gigabytes of memory and unrestricted network access; inference runs repeatedly on a phone or an ARM board with no network at all. A requirement that is trivial in one environment may be impossible in the other. This chapter therefore specifies the two environments separately, models the system conceptually through the standard object-oriented and structured views, and states the requirements independently of how they are satisfied.

3.1 Software and Hardware Tools Used

3.1.1 Hardware Requirements

Two machines were used, and both shaped the design. The primary training platform was an NVIDIA A40-16Q (a 16 GB vGPU slice of a 48 GB A40, software partitioned rather than MIG partitioned) on a Debian host, running on the host CPU with system memory sufficient to hold teacher and student concurrently and roughly 640 GB of storage, 575 GB of it free. That machine carried teacher distillation, student training, quantisation and evaluation.

The development platform was a local CPU-only workstation: an 8-core x86-64 processor, no accelerator of any kind, 7.4 GB of system memory and a standard SSD. It carried pipeline development, unit and integration testing, and the pipeline's smaller runs. Its constraints are recorded here because they shaped several design decisions that survive into the final system.

The 7.4 GB ceiling on the development machine is worth recording, because it produced a real design constraint rather than a mere inconvenience. A 52-million-parameter student with an AdamW optimiser state and batch-64 activations exceeded available memory, drove the machine into 5.6 GB of swap and slowed training to roughly four seconds per step. The response (a smaller development student and a batch size of 16) is why the repository carries two parallel configuration sets, `model.yaml`/`training.yaml` for CPU development and `model.gpu.yaml`/`training.gpu.yaml` for GPU training.

The deployment target is deliberately modest. The constraint that matters is that

inference must complete within 500 ms on a CPU with no accelerator, using an artifact no larger than 200 MB.

Processor. ARM Cortex-A55 class or better. A commodity x86-64 CPU is also supported.

Accelerator. None. Inference uses the ONNX Runtime CPU execution provider exclusively.

Memory. Approximately 512 MB of working memory per loaded direction.

Storage. 104 MB per translation direction, plus roughly 2 MB for the SentencePiece tokenizer.

Network. Required only for the initial one-time model download, and not used thereafter.

3.1.2 Software Tools Used

The software stack is described below, stage by stage through the pipeline.

Language. Python 3.11 or 3.12 implements the entire pipeline. Python 3.14 was abandoned because the teacher's rotary remote modelling code requires `transformers` below 4.54.

Data. Hugging Face `datasets` streams the Samanantar corpus into the cleaning stage.

Teacher. `transformers` ($\geq$4.42, $<$4.54), `IndicTransToolkit`, `einops` and `sentencepiece` load and run the IndicTrans2 distilled-200M teacher; `IndicProcessor` handles script normalisation and entity placeholders.

Training. PyTorch $\geq$2.6 (CUDA 12.4) with `torchvision` 0.21 trains the student. Torch 2.6 is a hard floor: the CVE-2025-32434 guard in newer `transformers` refuses to `torch.load` the teacher's non-safetensors weights on older versions.

Tokenizer. SentencePiece builds a 16,000-token unigram vocabulary per direction, with 23 FLORES language tags reserved so that token identifiers stay stable as languages are added.

Quantisation. `optimum`, ONNX and ONNX Runtime handle FP32 ONNX export, ORT validation and progressive INT8 and INT4 quantisation.

Inference. ONNX Runtime, CPU execution provider, executes the quantised student on device.

Evaluation. `sacrebleu` reports BLEU and chrF with reproducible signatures; COMET is available for languages where n-gram overlap understates quality.

API. FastAPI, Uvicorn and Pydantic provide the REST service with typed request and response models.

Frontend. Next.js (App Router), TypeScript and React provide the browser client and the offline progressive web application.

Testing. `pytest` runs 85 automated tests across all modules.

Tooling. Git for version control, and tmux to keep multi-hour training alive across SSH disconnections, with Conda or venv for environments.

Operating systems. Debian for training; Ubuntu 22.04 LTS and Android 11+ for deployment.

3.1.2.1 Environment Constraints Discovered During Development

Four environment problems were encountered in sequence on the training VM, each diagnosed and then encoded into the runner script so that a clean run never repeats them. They are recorded here because they are requirements in practice, even though no specification predicted them.

1. **Transformers versus Torch (CVE-2025-32434).** The VM shipped `transformers` 4.36; the installation floor raised it to 4.53, whose `torch.load` security guard refuses the teacher's non-safetensors `.bin` weights unless Torch is 2.6 or newer. Resolution: upgrade to Torch 2.6.0+cu124, which works on driver 535 through CUDA minor-version compatibility.
2. **Torchvision ABI mismatch.** After the Torch upgrade, loading the teacher failed with `operator torchvision::nms does not exist`, because `torchvision` 0.20.1 was built against Torch 2.5.1. Resolution: `torchvision` 0.21.0 (cu124).
3. **vGPU allocator crash.** Training died on its first CUDA allocation with `CUDA driver error: operation not supported`. The cause was `PYTORCH_CUDA_ALLOC_CONF=expandable_segments:True`, whose CUDA virtual-memory APIs are unsupported on an A40-16Q vGPU slice. Resolution: the flag now defaults to `False`.
4. **Resume re-clone bug.** The runner's clone check looked for `.git` inside the `SETU/` subdirectory, but Git places it at the clone root, so every resume attempted to re-clone over an existing tree that already held the expensive distilled corpus. Resolution: detect the repository by the root `.git` or `pyproject.toml`.

3.1.2.2 Dataset and Model Access Constraints

The `ai4bharat/BPCC` dataset and all official `ai4bharat/indictrans2-*` checkpoints are access-gated on Hugging Face and return HTTP 401 without an authenticated account that has accepted the terms. The project therefore uses the ungated `prajdabre/rotary-indictrans2-{indic-en,en-indic}-dist-200M` teachers and the ungated Samanantar corpus. No ungated Indic-to-Indic teacher exists at all, which is the direct cause of the English-pivot design in Section 5.2.5.

3.2 Conceptual / Analysis Modelling

This section models SETU from two complementary perspectives. The object-oriented view (use case, sequence, activity, state chart and class diagrams) describes actors, interactions and structure. The structured view (data flow and entity-relationship diagrams) describes how data moves and how it is related. Together they define what the system does before Chapter 5 decides how it does it.

A property visible in every model below is that SETU has two disjoint lifecycles. The *offline training lifecycle* is executed by an ML engineer, once per direction, and consumes a GPU. The *online inference lifecycle* is executed by an end user, repeatedly, on a device. An end user never touches a training component, and the training components are absent from the deployed artifact entirely.

3.2.1 Use Case Diagram

Three actors interact with the system. The **End User** translates text. The **ML Engineer** runs the training pipeline. The **Edge Device** is a secondary actor that hosts the quantised model and executes inference.

Each use case is specified below by its actor, its precondition and its postcondition.

Translate Text (End User). Precondition: a trained model exists for the requested direction. Postcondition: translated text is returned with measured latency.

Pivot Translate (End User). Precondition: no direct model exists, but both source→English and English→target models do. Postcondition: translated text is returned, flagged `pivot="en"`.

List Languages (End User). Precondition: the language registry is loaded. Postcondition: the 22 scheduled languages plus English are returned, with the trained ones marked.

Distil Teacher Targets (ML Engineer). Precondition: a cleaned corpus is available and the teacher checkpoint is reachable. Postcondition: a distilled corpus of teacher one-best targets is written.

Train Student (ML Engineer). Precondition: a distilled corpus is available and a GPU is available. Postcondition: a trained checkpoint and a training report are written.

Quantise and Export (ML Engineer). Precondition: a trained checkpoint exists. Postcondition: INT8 and INT4 ONNX artifacts are written with a quantisation report.

Evaluate (ML Engineer). Precondition: student and teacher are both available. Postcondition: BLEU, chrF and the student/teacher ratio are recorded.

Execute Inference (Edge Device). Precondition: the quantised model and tokenizer are present locally. Postcondition: a translation is produced with zero network calls.

3.2.2 Sequence Diagram

Figure 3.2 traces a single translation request. The significant branch is the pivot: when a directly trained model exists the engine performs one hop, and when it does not (every Indic-to-Indic pair) the engine performs two, passing the intermediate English string from the first student to the second. The response carries the summed latency of both hops and a `pivot` field so that the caller can tell the two cases apart.

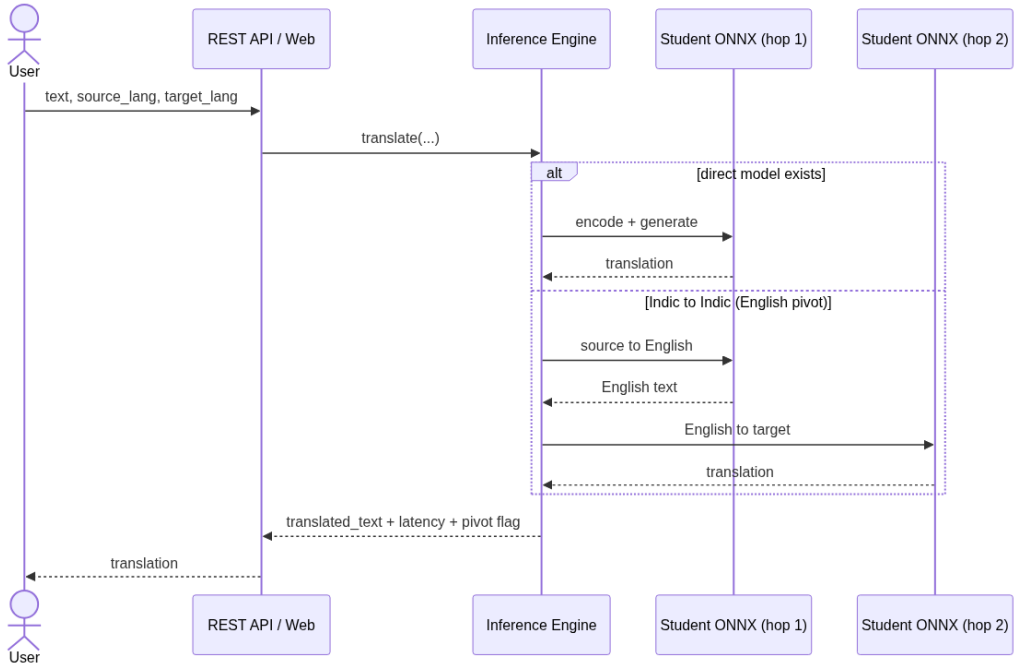


Figure 3.2: Sequence diagram: direct translation and the two-hop English pivot

3.2.3 Activity Diagram

Figure 3.3 shows the full training and deployment activity flow. Everything above the dividing line executes once per direction on a GPU; everything below executes on the user's device. The two decision points are the quality gate (does the student reach 80 % of teacher BLEU) and the routing decision at inference time.

3.2.4 State Chart Diagram

A model in SETU passes through a well-defined sequence of states. Figure 3.4 records them, including the two transitions that were exercised in practice: the collapse detected at evaluation and the recovery achieved by retraining at a lower learning rate on the same distilled corpus.

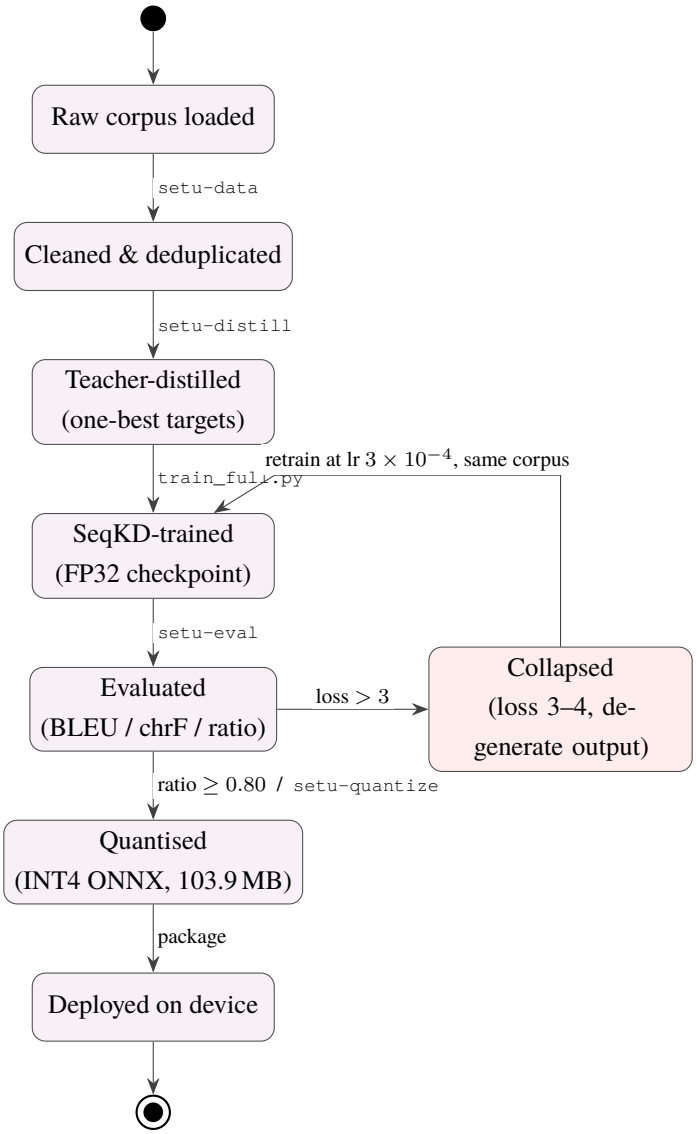


Figure 3.4: State chart diagram: model lifecycle from raw corpus to deployed artifact, including the observed collapse and recovery transitions

3.2.5 Data Flow Diagram

Figure 3.5 gives the level-1 data flow. The corpus is an external data source; the teacher is a transformation that produces the distilled data store; the trainer consumes that store and produces a student; the quantiser converts the student into a deployable artifact; the inference engine consumes the artifact and produces translations. The evaluator is a side branch that consumes both student and teacher outputs to compute the ratio that gates deployment.

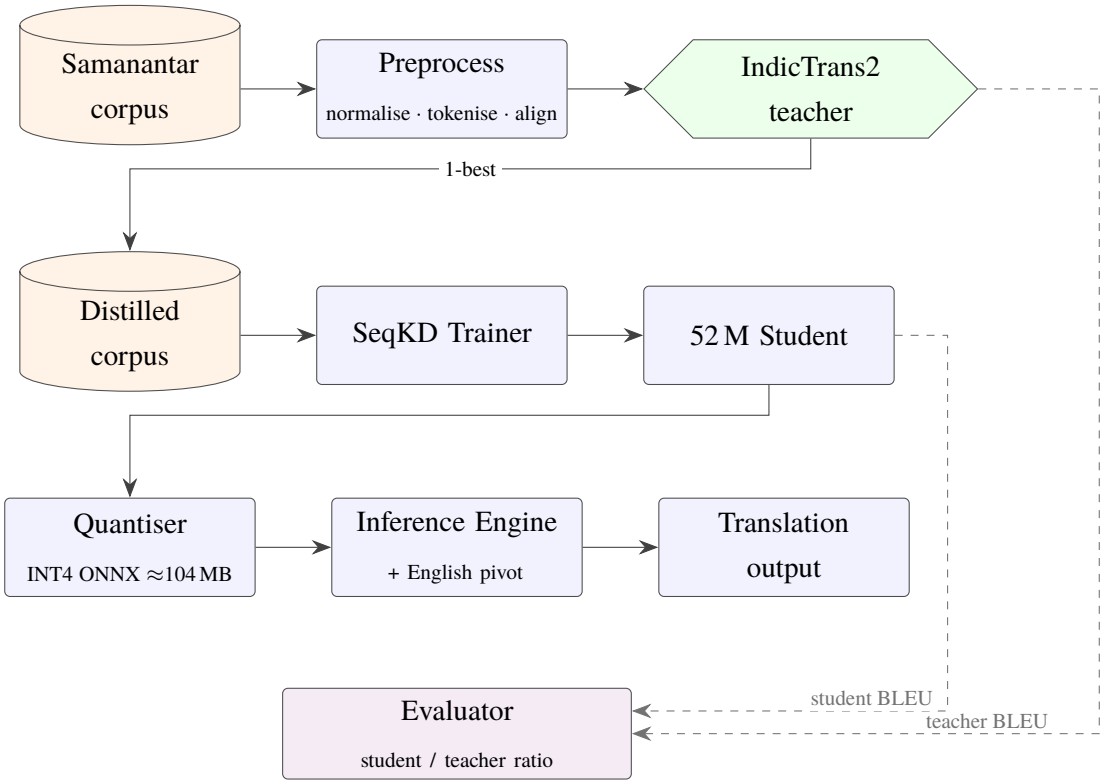


Figure 3.5: Level-1 data flow diagram: corpus to deployed translation

3.2.6 Entity-Relationship Model

SETU does not use a relational database (an offline edge deployment should not require one), but its persistent artifacts have a clear relational structure, recorded in Figure 3.6. Corpora are stored as JSONL, reports as JSON, models as ONNX with an accompanying SentencePiece tokenizer, and the language registry as YAML.

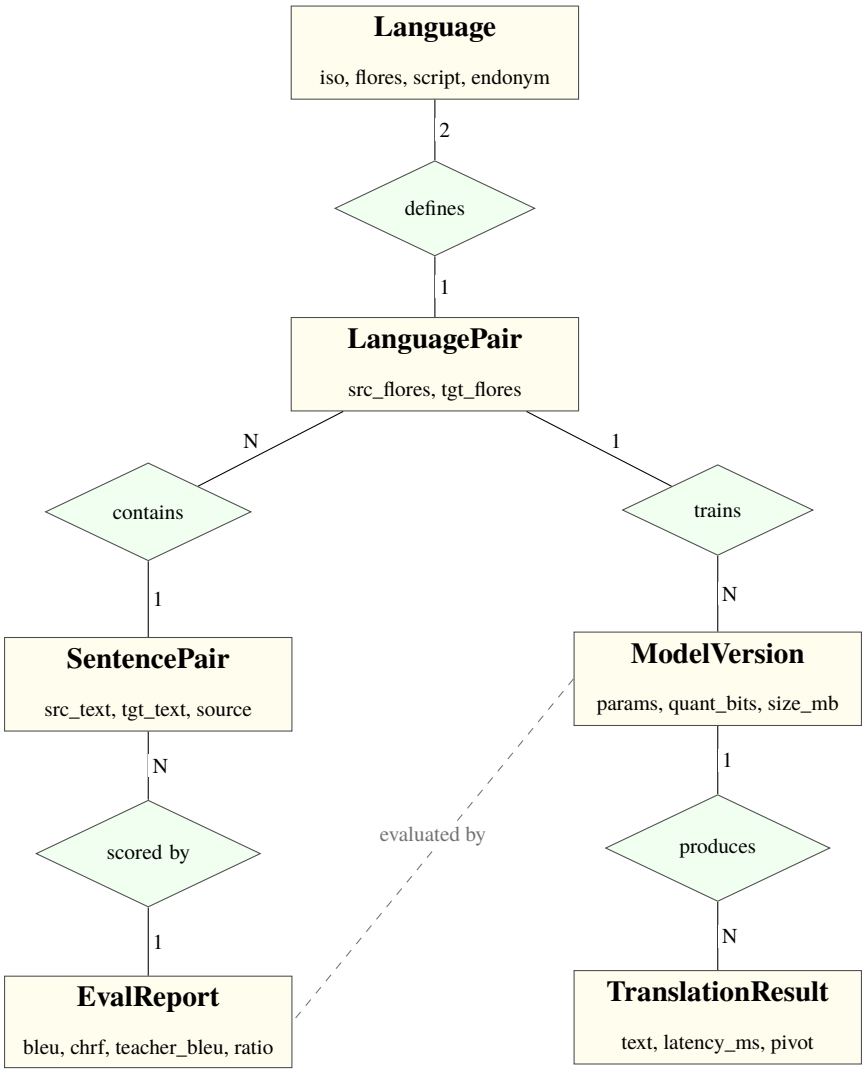


Figure 3.6: Entity-relationship model of SETU's persistent artifacts

3.2.7 Class Diagram

Figure 3.7 gives the principal classes. The design deliberately isolates every dependency on the teacher's implementation inside a single class, `IndicTrans2Teacher`. A unit test scans the source tree and fails the build if any other module imports `transformers`, `torch` or `IndicTransToolkit`. This is what allows the deployed artifact to contain no training dependencies at all.

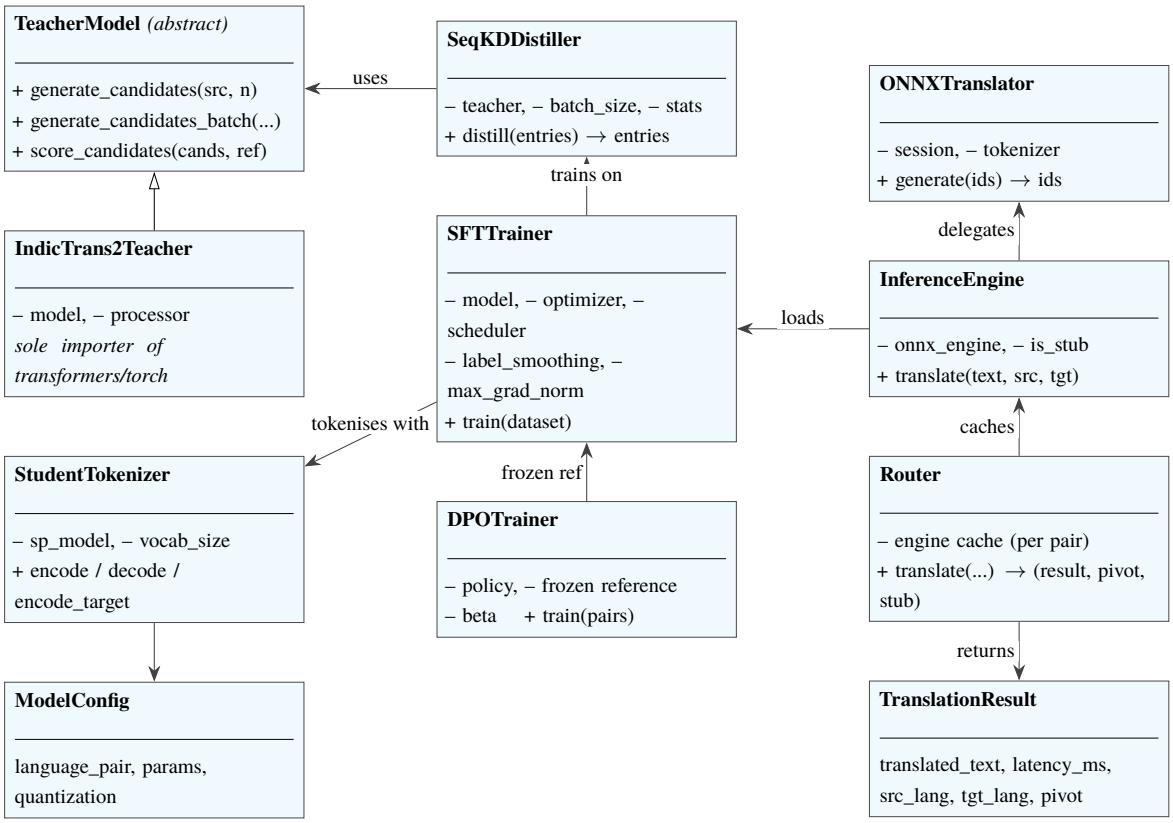


Figure 3.7: Class diagram: training classes (left) are used offline by the ML Engineer; inference classes (right) ship to the device

3.3 Software Requirements Specification

This section states what the system must do, independently of how it does it.

3.3.1 User Requirements

UR-1 (End user). Translate text between any two supported languages without an internet connection.

UR-2 (End user). See the full list of supported languages, each named in its own script, before choosing a direction.

UR-3 (End user). Receive a translation quickly enough for conversational use.

UR-4 (End user). Be assured that the text typed never leaves the device.

UR-5 (End user). Use the system from a browser, a mobile application, a command line or a programmatic API, with identical behaviour.

UR-6 (ML engineer). Train a model for any language pair by changing configuration, without modifying source code.

UR-7 (ML engineer). Resume a multi-hour training run after a disconnection without repeating completed stages.

UR-8 (ML engineer). Obtain an honest quality report, including failure, rather than an optimistic one.

UR-9 (Deployer). Install models on a fresh machine and run the system without training anything.

3.3.2 System Requirements

- **SR-1.** Python 3.11 or 3.12; Python 3.14 is incompatible with the teacher's remote modelling code.
- **SR-2.** `transformers` at least 4.42 and below 4.54, since 4.54 removed the legacy tuple `past_key_values` cache API that the rotary teacher relies on.
- **SR-3.** PyTorch 2.6 or newer for training, paired with a matching `torchvision` build.
- **SR-4.** ONNX Runtime with the CPU execution provider for inference; no GPU dependency in the deployed artifact.
- **SR-5.** A CUDA-capable GPU with at least 16 GB of memory for training a direction.
- **SR-6.** At least 104 MB of free storage per translation direction on the target device.
- **SR-7.** Node.js 18 or newer to build the browser client (build time only; the exported application is static).

3.3.3 Functional Requirements

Each requirement below names the section or test that verifies it.

FR-1. Translate text between supported Indian languages and English in both directions, fully offline. *Verified by* §7.1.2 and §8.5.

FR-2. Accept plain-text input of up to 512 tokens per request. *Verified by* §7.1.2.

FR-3. Reject a request whose source and target language are identical. *Verified by* UT-2.

FR-4. Reject a request containing an unknown language code. *Verified by* UT-3.

FR-5. Route Indic-to-Indic pairs through English when no direct model exists, and report that it did so. *Verified by* IT-1 to IT-3.

FR-6. Expose `POST /translate`, `GET /languages`, `GET /models` and `GET /health`. *Verified by* §5.3 and IT-4.

FR-7. Discover every model present on disk at start-up and report its variant and size. *Verified by* IT-4.

FR-8. Generate teacher one-best targets for an arbitrary corpus slice. *Verified by* §5.2.2.

FR-9. Train a student on either distilled or reference targets, selectable at run time. *Verified by* §5.2.3.

FR-10. Quantise a trained student to INT8 and INT4 ONNX with a per-stage benchmark. *Verified by* §5.2.4.

FR-11. Report BLEU, chrF and the student/teacher ratio per direction. *Verified by* §8.5.

FR-12. Support incremental addition of a language by configuration alone. *Verified by* §5.2.8.

3.3.4 Non-Functional Requirements

Each attribute below states the requirement and the outcome actually measured against it.

NFR-1, performance. Under 500 ms per sentence on an ARM-class CPU. *Measured:* mean 105–202 ms; p90 141–541 ms, with one outlier.

NFR-2, size. At most 200 MB per direction after quantisation. *Measured:* 103.9 MB for every direction.

NFR-3, accuracy. At least 80 % of teacher BLEU. *Measured:* 14 of 22 directions, mean ratio 0.838.

NFR-4, availability. Fully functional offline after initial setup. *Measured:* verified with all sockets disabled.

NFR-5, privacy. No user text transmitted during inference. *Measured:* verified by code audit and the offline test.

NFR-6, portability. Runs on x86-64 and ARM, Linux and Android. *Measured:* ONNX Runtime CPU provider on both.

NFR-7, maintainability. A single shared engine behind every interface. *Measured:* `setu.inference.router`.

NFR-8, testability. An automated suite covering every module. *Measured:* 85 tests.

NFR-9, reliability. Degrade explicitly, never silently. *Measured:* the stub engine reports `stub:true`, and the scorecard reports UNVERIFIED without evidence.

3.3.5 Domain Requirements

- **DR-1.** Language identification must follow the script-qualified FLORES convention used by IndicTrans2 (`hin_Deva`, `eng_Latn`), because several scheduled languages are written in more than one script. Kashmiri, Manipuri and Sindhi accordingly carry dual-script registry entries.
- **DR-2.** Script normalisation must preserve zero-width joiners and non-joiners, which are semantically significant in Indic scripts; naive control-character stripping corrupts valid text.
- **DR-3.** Corpus cleaning must enforce a minimum fraction of letters in the expected script, so that mined pairs with the wrong language on one side are removed.
- **DR-4.** Evaluation must always use real human references held out from training, never teacher output, regardless of what the student was trained on. This is what makes the comparison in Section 8.1 fair.
- **DR-5.** Tokenizer language tags must be reserved for all 22 scheduled languages from the first training run, so that token identifiers remain stable when languages are added later and previously trained models need not be re-tokenised.
- **DR-6.** BLEU must not be compared across languages. Agglutinative languages such as Malayalam and Tamil score low in absolute BLEU for morphological reasons; only the student/teacher ratio is comparable across directions.

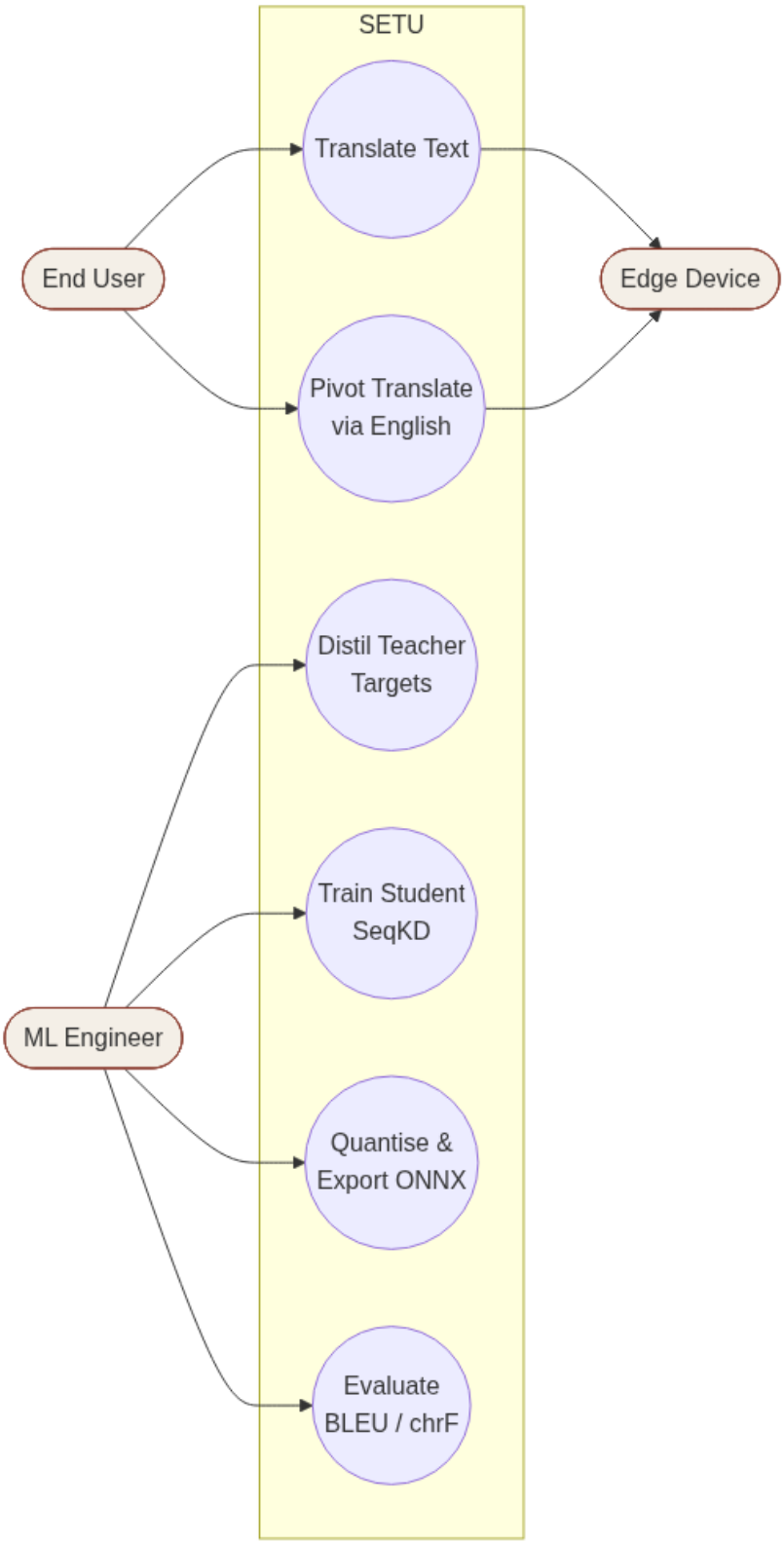


Figure 3.1: Use case diagram: End User, ML Engineer and Edge Device actors

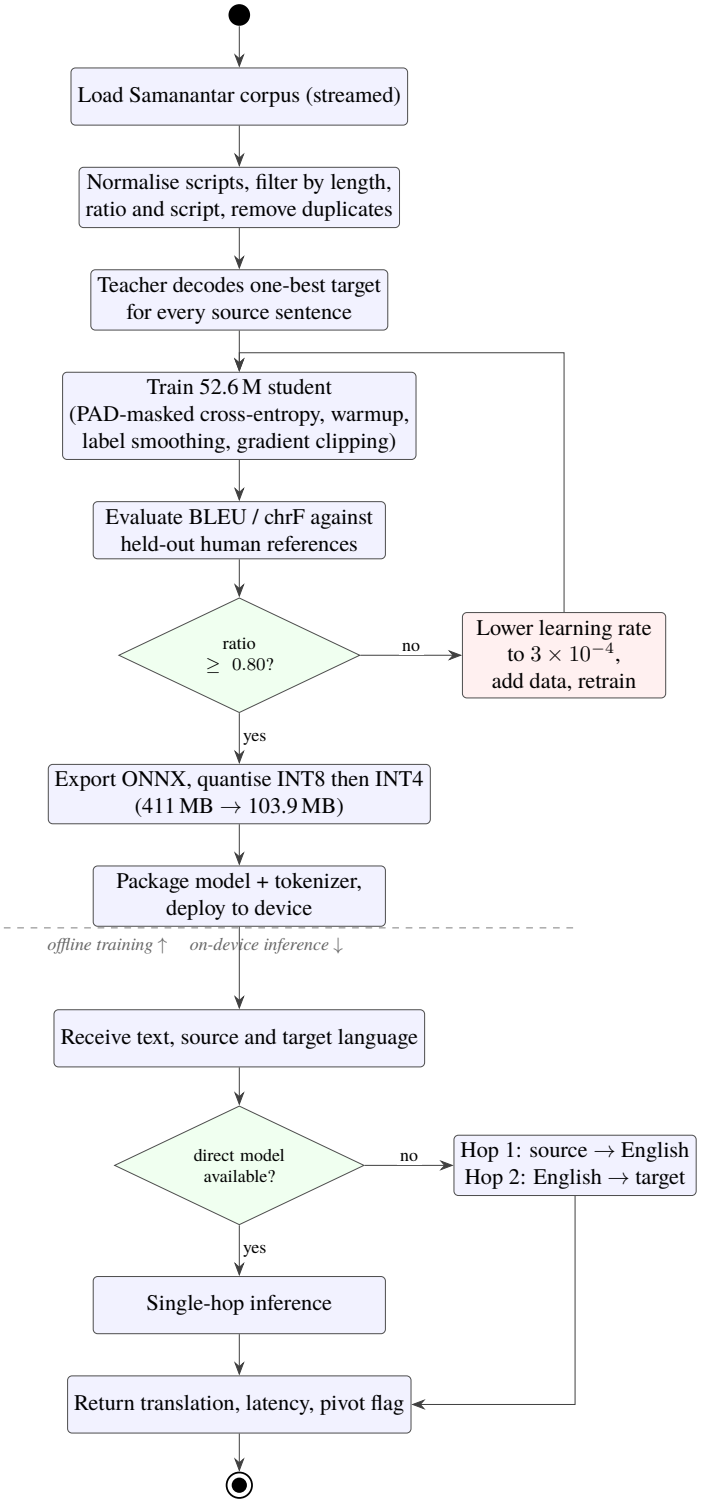


Figure 3.3: Activity diagram: offline training pipeline (above) and on-device inference (below)

CHAPTER 4

PROJECT PLANNING

Planning a machine learning systems project differs from planning conventional software in one decisive respect: several tasks have durations that cannot be known until they are attempted, and some have outcomes that determine which tasks follow. Teacher distillation over half a million sentences takes as long as it takes; a student either converges or collapses, and a collapse adds an unplanned diagnostic and retraining cycle. The plan described here was therefore built around milestones with explicit evidence requirements rather than around fixed-duration activities, and it carried deliberate slack on the training tasks.

This chapter presents the work breakdown structure, the schedule as a Gantt chart across both project phases, a PERT analysis with the critical path, the risk register with the mitigations actually applied, and the division of responsibility within the team.

4.1 Project Planning and Scheduling

4.1.1 Work Breakdown Structure

The project decomposes into nine engineering milestones, M0 through M8, followed by a hardening and delivery milestone. Each milestone is a single commit with a defined completion criterion, so that progress is verifiable rather than self-reported. Table 4.1 gives the breakdown together with the evidence required to close each item.

Table 4.1: Work breakdown structure with completion evidence

ID	Task	Description	Completion evidence
M0	Scaffold	Package layout, four core data objects, YAML configuration with the 22-language registry, CLI, passthrough stub engine	7 tests green; CLI runs end to end

continued on next page

Table 4.1 (continued)

ID	Task	Description	Completion evidence
M1	Data pipeline	Corpus loading, script normalisation, length and script filtering, deduplication, JSONL output	50,000 pairs in, 49,520 clean pairs out, with a filter breakdown report
M2	Teacher integration	IndicTrans2 behind an abstract interface; a test forbids any other module importing the teacher's dependencies	Real smoke test producing fluent candidate translations
M3	Preference generation	Teacher n-best candidates plus k-NN retrieved neighbours, chrF ranking, pair validation	500 entries $\rightarrow$ 3,494 candidates $\rightarrow$ 2,074 validated pairs
M4	SFT and DPO training	SentencePiece tokenizer, student model, supervised fine-tuning, DPO with a frozen reference, evaluation harness	DPO measurably outperforms SFT; test enforces the reference stays frozen
M5	Quantisation and offline inference	ONNX export, progressive INT8 and INT4 quantisation, ONNX inference engine, socket-disabling offline assertion	Size, latency and offline targets all pass
M6	Interfaces	FastAPI REST service, CLI, offline PWA, mobile SDK stubs, all sharing one engine	15 interface tests green
M7	All-in-One KD	Temperature-sampled cross-lingual balancing with a coverage report	Mechanism proven by test
M8	Elastic Weight Consolidation	Diagonal-Fisher penalty, old-task snapshot, EWC training step	Measured retention of the old task above naive fine-tuning

continued on next page

Table 4.1 (continued)

ID	Task	Description	Completion evidence
M9	Hardening and delivery	Scorecard generator, benchmarks, documentation, dead-code removal	Full suite green; scorecard reports UNVERIFIED when evidence is absent
P2-A	SeqKD baseline and head-to-head	Distiller CLI, `--train-corpus` switch, S0/S1/S2/S3 comparison	Figure 2.1 filled
P2-B	GPU campaign	Environment fixes, resumable runners, 22 directions trained bidirectionally	22 per-direction reports
P2-C	Collapse diagnosis and recovery	Root-cause the two degenerate students and retrain	Both recovered to passing ratios
P2-D	English-pivot routing	Shared router used by REST and CLI alike	Verified on Hindi–Bengali, Tamil–Telugu, Assamese–Bengali
P2-E	Frontend and release	Next.js client, offline PWA, public model release, quick-start documentation	Live demo; models downloadable without login
P2-F	Report and paper	This report; paper plan with filled result tables	Submitted

4.1.2 Gantt Chart

Figure 4.1 gives the schedule. Phase 1 ran from January to mid-August 2026 and delivered the pipeline, the empirical finding and the first eleven languages. Phase 2 ran from August to the end of October 2026 and delivered the other eleven languages, benchmark rescoring, edge packaging and the final report. Both phases are complete.

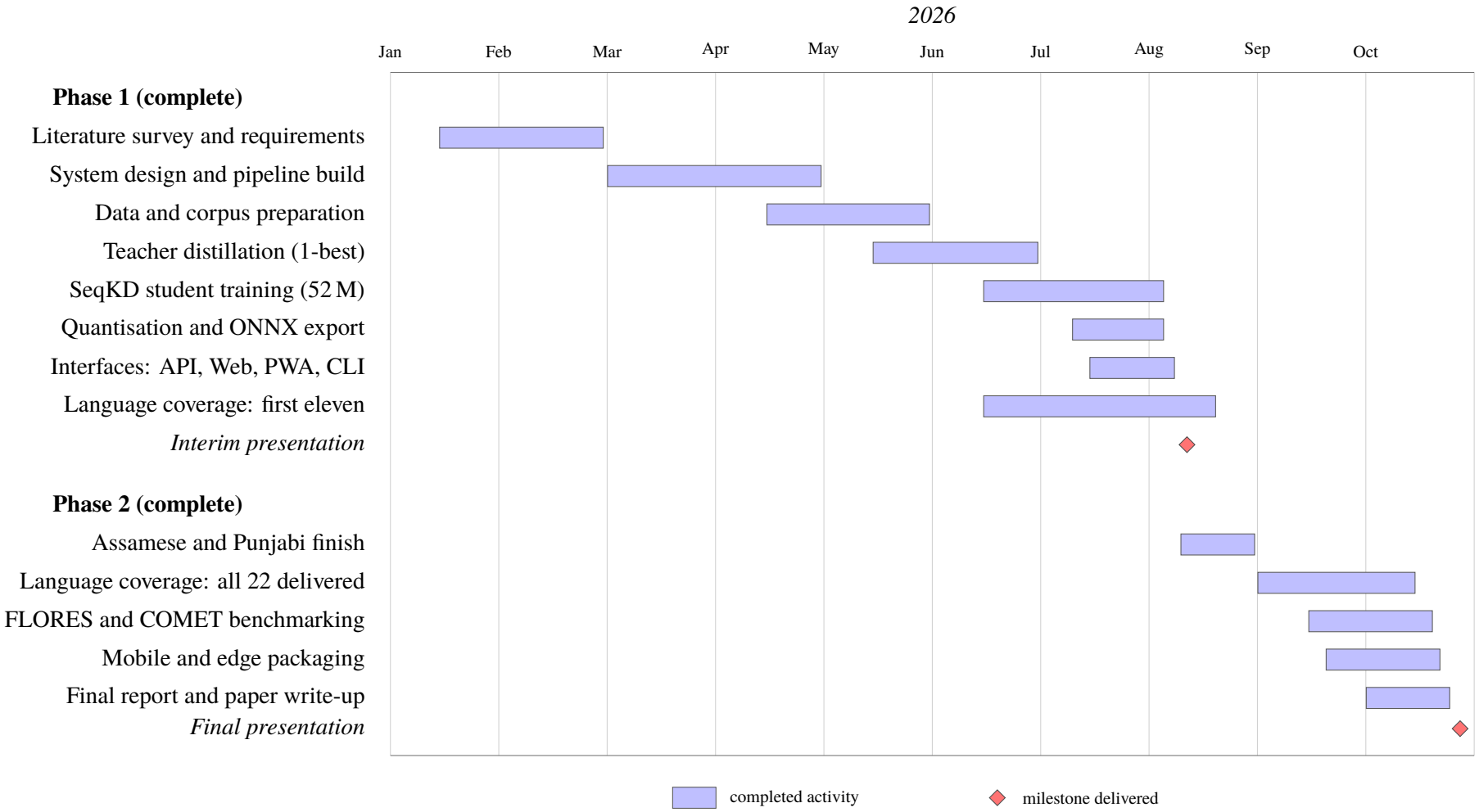


Figure 4.1: Gantt chart: both phases complete, all 22 scheduled languages delivered

Table 4.2: Schedule of activities with durations

Activity	Start	Finish	Weeks	Status
Phase 1				
Literature survey and requirements	15 Jan 2026	28 Feb 2026	6.5	Complete
System design and pipeline build	01 Mar 2026	30 Apr 2026	8.5	Complete
Corpus preparation (Samanantar)	15 Apr 2026	31 May 2026	6.5	Complete
Teacher distillation (one-best)	15 May 2026	30 Jun 2026	6.5	Complete
SeqKD student training (52 M)	15 Jun 2026	05 Aug 2026	7.0	Complete
Quantisation and ONNX export	10 Jul 2026	05 Aug 2026	3.5	Complete
Interfaces: API, web, PWA, CLI, pivot	15 Jul 2026	08 Aug 2026	3.5	Complete
Language coverage, first eleven	15 Jun 2026	20 Aug 2026	9.5	Complete
Interim presentation	*12 Aug 2026*		n/a	Milestone
Phase 2				
Assamese and Punjabi completion	10 Aug 2026	31 Aug 2026	3.0	Complete
Language coverage, all 22 delivered	01 Sep 2026	15 Oct 2026	6.5	Complete
FLORES-200 and COMET benchmarking	15 Sep 2026	20 Oct 2026	5.0	Complete
Mobile and edge packaging	20 Sep 2026	22 Oct 2026	4.5	Complete
Final report and paper write-up	01 Oct 2026	25 Oct 2026	3.5	Complete
Final presentation	*28 Oct 2026*		n/a	Milestone

4.1.3 PERT Analysis

The Programme Evaluation and Review Technique was applied to the Phase 1 activities, whose durations were genuinely uncertain. For each activity an optimistic (a), most likely (m) and pessimistic (b) duration in weeks was estimated, and the expected duration computed as

$$t_e = \frac{a + 4m + b}{6}, \qquad \sigma = \frac{b - a}{6}, \qquad \sigma^2 = \left(\frac{b - a}{6}\right)^2. \tag{4.1}$$

Table 4.3: PERT duration estimates for Phase 1 activities (weeks)

ID	Activity	Pred.	a	m	b	t_e	σ^2
A	Literature survey & requirements	n/a	4	6	9	6.17	0.69
B	System design	A	3	4	7	4.33	0.44
C	Pipeline implementation (M0–M4)	B	6	8	13	8.50	1.36
D	Corpus preparation	B	3	4	8	4.50	0.69
E	Teacher distillation	C, D	4	6	10	6.33	1.00
F	Student training (SeqKD)	E	5	7	12	7.50	1.36
G	Collapse diagnosis & retraining	F	0	2	5	2.17	0.69
H	Quantisation & ONNX export	F	2	3	5	3.17	0.25
I	Interfaces & pivot routing	H	2	3	6	3.33	0.44
J	Evaluation & scorecard	G, H	1	2	4	2.17	0.25
K	Documentation & report	I, J	2	3	5	3.17	0.25

Figure 4.2 shows the activity network. The critical path is **A** $\rightarrow$ **B** $\rightarrow$ **C** $\rightarrow$ **E** $\rightarrow$ **F** $\rightarrow$ **G** $\rightarrow$ **J** $\rightarrow$ **K**, with an expected duration of

$$6.17 + 4.33 + 8.50 + 6.33 + 7.50 + 2.17 + 2.17 + 3.17 = \mathbf{40.34} \text{ weeks},$$

and a variance of $0.69 + 0.44 + 1.36 + 1.00 + 1.36 + 0.69 + 0.25 + 0.25 = 6.04$, giving a standard deviation of $\sigma = 2.46$ weeks. Corpus preparation (D) and the interface work (I) carry float and were not critical; the interface work was deliberately scheduled in parallel with training so that a training delay would not idle the team.

The analysis identifies student training (F) and the unplanned collapse diagnosis (G) as the dominant risks, jointly contributing a third of the total variance. This proved accurate:

the two collapses described in Section 8.4 consumed almost exactly the two weeks that activity G was allotted.

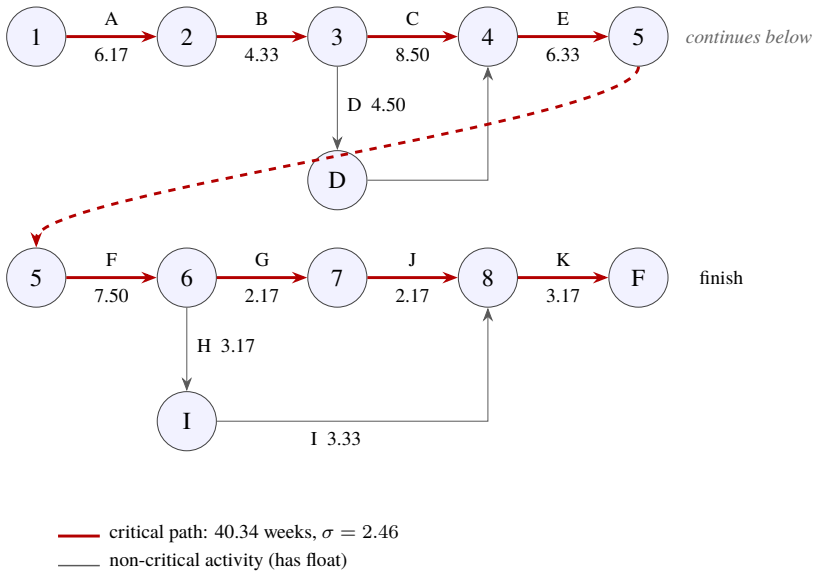


Figure 4.2: PERT network for Phase 1 with the critical path highlighted

4.1.4 Risk Management

Table 4.4 records the risks identified at planning time, together with the mitigation applied and (where the risk materialised) what actually happened. Five of the nine anticipated risks occurred.

Table 4.4: Risk register with outcomes

ID	Risk	Prob.	Impact	Mitigation and outcome
R1	Gated access to the teacher checkpoints and the BPCC corpus	High	High	Fall back to ungated rotary checkpoints and the Samanantar corpus. **Occurred.** Mitigation applied; the cost is a lower quality ceiling, reported explicitly
R2	No GPU available locally	High	High	Use a cloud GPU (Kaggle, Colab) and later a dedicated A40 VM. **Occurred.** All final training ran on the A40 VM

ID	Risk	Prob.	Impact	Mitigation and outcome
R3	Student fails to reach the quality target	Medium	High	Treat data volume, teacher strength and student width as independent levers; report shortfall honestly. **Partially occurred:** 14 of 22 directions pass
R4	Training instability or divergence	Medium	High	Gradient clipping, learning-rate warmup, label smoothing; monitor training loss as a collapse indicator. **Occurred twice**; both recovered (Section 8.4)
R5	Cloud session time limits kill long runs	High	Medium	Resumable runners with per-stage completion markers; persist artifacts outside the session. **Occurred** on Kaggle's 12-hour cap; resumability absorbed it
R6	Quantised model exceeds the size budget	Low	High	Progressive INT8 then INT4 with per-stage measurement. **Did not occur:** 103.9 MB against a 200 MB budget
R7	Inference too slow on CPU	Medium	High	ONNX Runtime with greedy decoding at deployment; beam search only offline. **Did not occur:** mean 105–202 ms
R8	Dependency and environment conflicts	Medium	Medium	Pin versions; audit the environment before each run. **Occurred** four times on the VM (Section 3.1.2); each fix folded into the runner

R9	Insufficient corpus for low-resource languages	High	Medium	Prioritise languages with ungated data and widen the corpus for the lower-resource tail. **Resolved:** all 22 languages delivered

4.1.5 Team Organisation and Responsibilities

The team of four divided the work by pipeline stage, with every member responsible for the tests covering their own modules. Integration was continuous: a milestone was not considered closed until the full test suite passed, which forced interfaces to be agreed early rather than reconciled late.

Table 4.5: Division of responsibility

Member	USN	Primary responsibility
Jannat Zafrin	1BI23AI017	Literature survey; corpus pipeline (loading, normalisation, filtering, deduplication); evaluation harness and scorecard generation
Prabhav Sharma	1BI23AI040	Teacher integration and the abstraction wall; preference generation (k-NN retrieval, chrF ranking, validation); DPO trainer
Rishabh Sharma	1BI23AI047	SeqKD distiller and student trainer; GPU training campaign and runners; quantisation and ONNX export; collapse diagnosis and recovery
Sindhuja Tiwari	1BI23AI054	Inference engine and pivot router; REST API; Next.js client and offline PWA; CLI; test suite and documentation

All four members contributed to system design, to the analysis of experimental results, and to this report. Weekly review meetings with the guide governed milestone closure, and the project log maintained in the repository records a dated entry for every working session, which is the source from which Chapter 8 is reconstructed.

CHAPTER 5

SYSTEM DESIGN

System design bridges the requirements of Chapter 3 and the implementation of Chapter 6. It answers the question *how*: how the system is structured, how it decomposes into modules, what crosses the boundaries between them, how data is represented, and what the core algorithms are.

One structural decision dominates the design and is worth stating before the detail. SETU is not a single program but two programs that share a data format. The training program is large, depends on PyTorch and Transformers, runs on a GPU, and executes once per direction. The inference program is small, depends only on ONNX Runtime and SentencePiece, runs on a CPU, and executes millions of times. They meet at exactly one point: a directory containing a quantised ONNX model and a SentencePiece tokenizer. Everything else in the design follows from keeping that boundary clean, because it is what makes the deployed artifact small enough and offline enough to satisfy the requirements.

5.1 System Architecture

Figure 5.1 gives the high-level architecture. Data flows left to right through four stages (corpus, distillation, training, deployment) and then fans out to the four client interfaces.

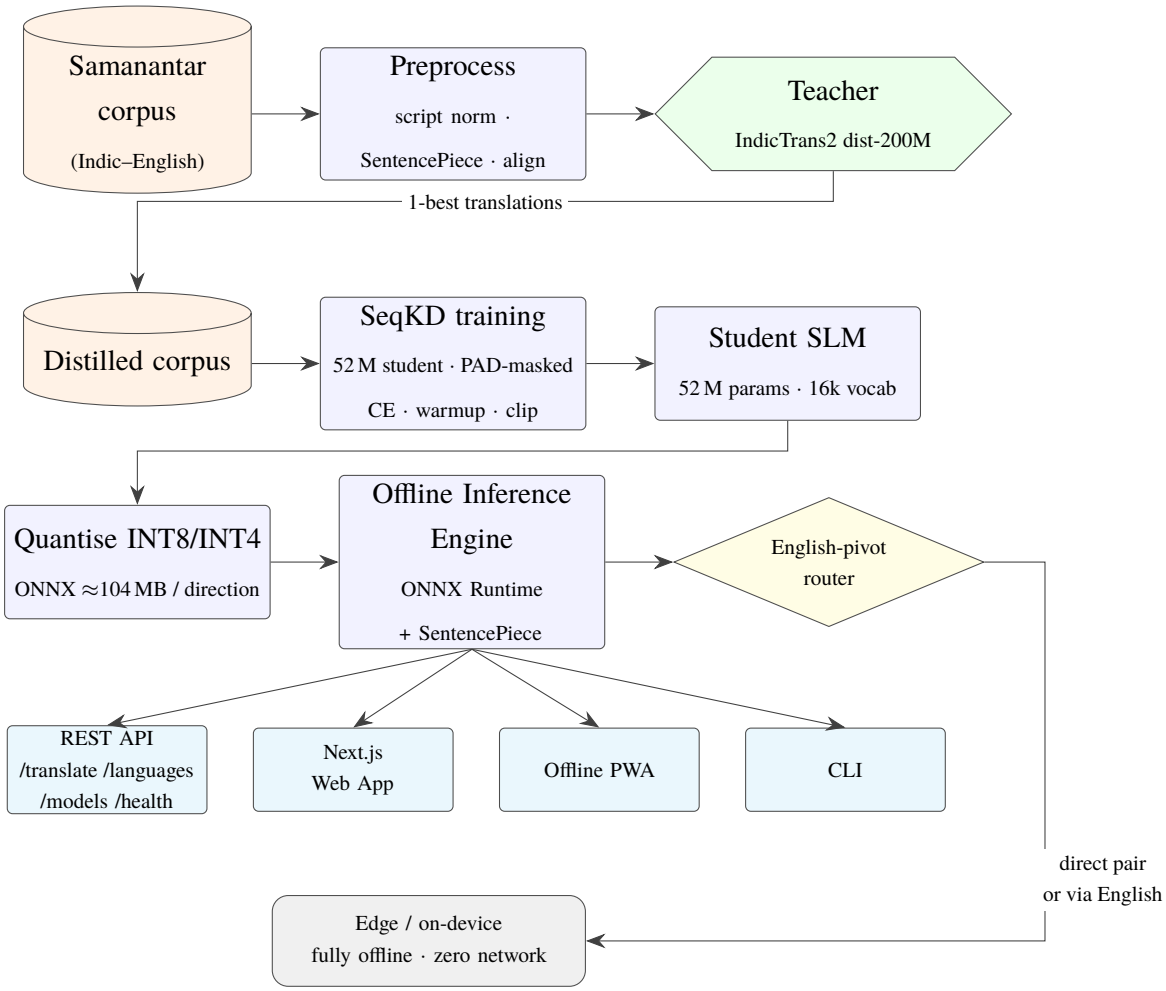


Figure 5.1: System architecture: SeqKD distillation and offline deployment pipeline

The architecture has four layers.

The data layer ingests parallel corpora, normalises and filters them, and stores two corpora per direction: the *processed* corpus, which retains human references and is used only for evaluation, and the *distilled* corpus, whose targets are the teacher's one-best translations and which is used for training. Keeping both is what allows evaluation to remain on real references regardless of what the student was trained on, the property that makes the comparison in Section 8.1 fair.

The model layer holds the teacher behind an abstract interface and the student behind a trainer. The teacher is used only offline, and only through three methods. The student is a Marian encoder–decoder trained from scratch.

The deployment layer converts a trained student into a device artifact: ONNX export, runtime validation, progressive INT8 and INT4 quantisation, and a per-stage benchmark. Its output is a self-contained directory.

The serving layer loads those directories on demand, caches one engine per direction, routes each request to a direct model or through the English pivot, and presents the

result through four interfaces that share the routing code exactly.

5.1.1 Design Principles

Five principles were applied consistently and explain most of the specific choices that follow.

One direction, one model. Each translation direction gets its own student. This avoids the cross-lingual interference that multilingual distillation frameworks report [10], permits an independent quality gate per direction, and means a collapsed direction can be retrained without disturbing any other. The cost is storage, 104 MB per direction, which the English pivot then bounds at $2n$ rather than n^2.

The teacher lives behind a wall. `IndicTrans2Teacher` is the only module permitted to import `transformers`, `torch` or `IndicTransToolkit`. A unit test walks the source tree and fails the build on any violation. This is what guarantees that the inference path carries no training dependencies.

Configuration, not code. The language pair, model dimensions, hyperparameters, corpus sources and cleaning thresholds all live in YAML. Adding a language is a configuration change and a data fetch. A dedicated test verifies that the pipeline is pair-agnostic by running it on a different pair end to end.

Fail loudly, never silently. When no model exists for a requested pair the engine returns the input unchanged but sets `stub: true`, so a caller can never mistake a passthrough for a translation. When the scorecard has no evidence for a target it prints UNVERIFIED rather than a pass. This principle is why the 0.796 ratio in an early report was printed as a failure rather than rounded up.

Resumability by construction. Every pipeline stage writes a completion marker. A multi-hour run interrupted by a disconnection resumes at the stage it reached, which matters because teacher distillation over 500,000 sentences is too expensive to repeat.

5.2 Component Design / Module Decomposition

The system decomposes into eight modules. Each has a single responsibility, a defined input and output, and its own tests.

5.2.1 Module 1: Corpus Loader

Responsibility: turn a raw parallel corpus into a clean, deduplicated set of `CorpusEntry` objects.

The module streams Samanantar from Hugging Face (streaming rather than downloading, because the full corpus is tens of gigabytes and only a slice is needed) and passes

each pair through four stages. *Normalisation* applies Unicode NFC and strips control characters while explicitly preserving zero-width joiners and non-joiners, which carry meaning in Indic scripts. *Filtering* removes pairs shorter than 3 or longer than 1024 characters, pairs whose source-to-target length ratio exceeds 3.0 (a reliable signal of misalignment in mined data), and pairs in which fewer than half the letters belong to the expected script. *Deduplication* removes exact duplicates. *Reporting* records how many pairs each filter removed.

A representative run illustrates the module's value: 50,000 input pairs yielded 49,520 clean pairs, with 253 removed for length ratio, 213 for wrong script, 10 for excessive length and 4 as duplicates. Roughly one pair in a hundred of web-mined data is unusable, and the wrong-script filter in particular catches pairs where one side is simply not in the language it claims to be.

5.2.2 Module 2: Teacher Distiller

Responsibility: replace every human reference target with the teacher's own one-best translation of the source.

`SeqKDDistiller` batches source sentences, calls the teacher's `generate_candidates_batch` with $n = 1$, and emits new `CorpusEntry` objects tagged `seqkd`. Greedy decoding is used rather than beam search: it is four to five times faster, which is what makes distilling half a million sentences feasible within a session, and the resulting targets are of equivalent utility for training.

The module degrades safely. If the teacher returns an empty candidate for some sentence, the original human reference is retained for that pair and the event is counted in the module's statistics, so a teacher hiccup cannot silently introduce blank training targets.

5.2.3 Module 3: SeqKD Trainer

Responsibility: train a 52.6-million-parameter student from scratch on the distilled corpus.

The trainer implements token-level cross-entropy with five components, each added in response to an observed failure rather than adopted by convention:

- **PAD masking.** Label positions corresponding to padding are set to -100 so that the loss ignores them. Before this fix the model received free credit for predicting padding, which produced a fake-low loss of 0.274 and a decoder that ignored the source entirely.
- **Linear warmup and decay.** The warmup length is capped at 10 % of the run. A from-scratch Transformer trained at a constant learning rate underfits badly; the configuration option existed but was being silently ignored.

- **Label smoothing (0.1) and dropout (0.2).** Standard regularisers against overconfidence and memorisation.
- **Gradient-norm clipping at 1.0.** The single most important stabiliser. Without it, a 200,000-sentence run collapsed to a constant output while the identical code at 120,000 sentences had succeeded: more steps simply meant more opportunities to hit a gradient spike.
- **Generation safeguards.** Decoding suppresses the PAD, UNK and BOS tokens and forbids repeated 3-grams, so output can never be empty and cannot degenerate into a repeated phrase.

The module also implements DPO with a frozen supervised-fine-tuning reference model. A test enforces that the reference model's parameters never receive gradients, since an unfrozen reference silently reduces DPO to a no-op. This path is retained for the reported comparison but is not used for the shipped models.

5.2.4 Module 4: Quantiser

Responsibility: convert a trained PyTorch checkpoint into a deployable ONNX artifact.

The module exports to FP32 ONNX, validates the export by running it through ONNX Runtime and comparing outputs, then quantises progressively (INT8 first, INT4 second), benchmarking size and latency at each stage. The measured progression for a 52.6-million-parameter student is 410.9 MB FP32, 104.6 MB INT8 and 103.9 MB INT4.

The near-identity of the INT8 and INT4 sizes is an honest limitation rather than a measurement error: the ONNX Runtime CPU execution provider has no true 4-bit kernel, so INT4 export falls back to 8-bit storage for most tensors. The INT4 variant is nonetheless the one shipped, because it benchmarked marginally faster and, in several directions, marginally more accurate.

5.2.5 Module 5: Inference Engine and Pivot Router

Responsibility: translate text using a quantised student, offline, and route requests to the right student.

`InferenceEngine` loads an ONNX session and a local SentencePiece tokenizer, encodes the source, decodes greedily, and returns a `TranslationResult` carrying the translated text and the measured latency. An `assert_offline()` helper disables all sockets to prove that inference needs no network.

The router sits above it and solves the coverage problem. Given a source and target language it resolves both to FLORES codes, rejects identical pairs and unknown codes,

then checks for a directly trained model. If one exists it translates in a single hop. If not (the case for every Indic-to-Indic pair, since no ungated Indic-to-Indic teacher was available), it checks whether both source→English and English→target models exist, and if so chains them, returning the combined latency and a `pivot: "en"` flag. The arithmetic is favourable: 44 direct models serve 506 usable directions, and measured two-hop latency is approximately 50–65 ms.

Engines are cached one per pair, so a direction is loaded from disk at most once, and every interface calls this single router rather than reimplementing routing. Consolidating the routing logic here fixed a real defect: the CLI had previously constructed an engine with the default configured pair, so `--src as` silently translated using the Hindi model.

5.2.6 Module 6: REST API

Responsibility: expose the engine over HTTP.

A FastAPI service provides four endpoints. `POST /translate` accepts a source language, target language and text and returns the translation with latency and pivot information. `GET /languages` returns the registry of 22 scheduled languages plus English, marking which are trained. `GET /models` scans the models directory and reports every discovered pair with its quantisation variant and size on disk. `GET /health` reports readiness. CORS is enabled so that a browser client can call the service directly.

5.2.7 Module 7: Client Interfaces

Responsibility: make the engine usable.

Four clients share the engine. The **CLI** accepts text from an argument, a file or standard input and emits plain text or JSON. The **Next.js web client** provides a translator console with source and target pickers, a swap control, live latency display and an on-device indicator; all 23 languages appear in the picker, with the trained ones grouped as available and the remainder shown but disabled, so coverage is visible rather than hidden. The **offline progressive web application** caches its shell through a service worker and continues to function without a network. **Mobile SDK stubs** for Android and iOS wrap the same API surface.

5.2.8 Module 8: Evaluator and Expansion Mechanisms

Responsibility: measure quality, and provide a route to adding languages.

The evaluator computes BLEU and chrF with sacreBLEU on a held-out slice of real human references, computes the teacher's score on the identical slice, and reports the ratio.

Loaders for FLORES-200 and IN22 are implemented for benchmark-grade numbers, along with COMET scoring and paired-bootstrap significance testing.

Two expansion mechanisms are implemented and tested. **All-in-One knowledge distillation** trains one student across several directions with temperature-sampled cross-lingual balancing and a coverage report; a sampling temperature above 1.0 flattens the distribution towards uniform and so protects low-resource directions from being swamped. **Elastic Weight Consolidation** adds a diagonal-Fisher penalty that anchors parameters important to previously learned languages, enabling a new language to be added without a full retrain. Its tests prove the penalty is zero at the snapshot point, grows with distance from it, and (measured rather than assumed) keeps Fisher-weighted drift on an old task below that of plain fine-tuning after a conflicting new task.

Component	Description	Details and example

5.3 Interface Design

5.3.1 External Interfaces

Table 5.1: External interface specification

Component	Description	Details and example
REST: translate	Primary translation endpoint	`POST /translate` with body `{source_lang, target_lang, text}`; returns `{translated_text, latency_ms, pivot, stub}`. Example request: `{"source_lang":"hi", "target_lang":"en", "text":"..."}`
REST: languages	Language registry	`GET /languages` → 22 scheduled languages plus English, each with ISO code, FLORES code, script and endonym
REST: models	Model discovery	`GET /models` → `{"count": 22, "models": [{pair, variant, size_mb}, ...]}`
REST: health	Readiness probe	`GET /health` → `{"status":"ok", "stub": false}`
CLI	Command-line translation	`python setu_cli.py --src hi --tgt en --text "..."`; also `--file` and standard input, with `--json` output
Web client	Browser translator console	Language pickers grouped into available and not-yet-trained, swap control, live latency, on-device badge, curated Indic-to-Indic examples
Offline PWA	Installable offline client	Service-worker shell caching; functions with no network after first load
Mobile SDK	Android and iOS wrapper	`translate(text, srcLang, tgtLang)` over the same engine

Component	Description	Details and example
Training CLI	Pipeline entry points	`setu-data`, `setu-distill`, `setu-quantize`, `setu-eval`, `setu-report`, and `scripts/train_full.py`

5.3.2 User Interface Design

The web client was designed around one principle: a user must be able to tell, at a glance, whether a translation was produced on their device and how good the coverage is for their language. Three affordances implement it.

The **on-device badge** reports whether the response came from a real quantised model or the passthrough stub, alongside the measured latency in milliseconds. A user is never shown a number without knowing what produced it.

The **language picker** lists all 23 languages, every one of them selectable: the twenty-two scheduled languages and English. There are no disabled entries, because every direction is trained and served.

The **pivot indicator** tells the user when a translation was routed through English, because a two-hop translation has different failure characteristics from a single-hop one and the user is entitled to know.

Visually the client uses a warm paper-and-ink palette with a single accent colour, a light and dark theme, and typography that names every language in its own script (हिन्दी, বাংলা, தமிழ், తెలుగు, ಕನ್ನಡ, മലയാളം, ગુજરાતી, ଓଡ଼ିଆ, ਪੰਜਾਬੀ) rather than transliterating them into Latin. Fonts are self-hosted so that the client loads no external resource at run time, which is a functional requirement rather than a stylistic one.

5.4 Data Structure Design

Four data objects carry all information between modules. Their field names are stable across the entire codebase, which is what allows modules to be developed and tested independently.

Table 5.2: Core data structures

Structure	Fields	Purpose
`CorpusEntry`	`src_lang`, `tgt_lang`, `src_text`, `tgt_text`, `source`	One parallel sentence pair. The `source` tag distinguishes a human reference from a `seqkd` teacher target, so the two corpora never mix
`PreferencePair`	`src_text`, `preferred_tgt`, `dispreferred_tgt`, `quality_delta`	One DPO training example. `quality_delta` is the chrF gap; pairs below a threshold are discarded as too noisy to teach from
`ModelConfig`	`language_pair`, `params`, `quantization`	Full student specification: architecture, layer counts, hidden size, vocabulary size, sequence length, quantisation mode
`TranslationResult`	`translated_text`, `src_lang`, `tgt_lang`, `latency_ms`, `pivot`	One translation with its provenance. `pivot` is `None` for direct translation and `"en"` for a chained one

5.4.1 On-Disk Layout

Corpora are stored as JSONL, one entry per line, which streams without loading the whole file and appends cheaply. Reports are JSON. A deployed model directory is self-contained:

```
models/<src_flores>-<tgt_flores>/
  |-- int4/       encoder + decoder ONNX (103.9 MB, shipped)
  |-- int8/       the INT8 variant (104.6 MB)
  |-- onnx/       the FP32 export (410.9 MB, not shipped)
  |-- tokenizer/  SentencePiece model and vocabulary
  `-- quantize_report.json    per-stage size and latency
```

Model discovery scans for exactly this layout, which is why dropping a trained model directory into place is all that is needed for the API to serve it.

5.4.2 The Tokenizer's Reserved Tags

The SentencePiece vocabulary is 16,000 unigram tokens, chosen after a 32,000-token vocabulary was measured to leave most tokens undertrained on the available data and to produce degenerate decoding. All 23 FLORES language tags are reserved as special tokens from the very first training run, ahead of the directions that used them. This costs 23 vocabulary slots and buys a guarantee: token identifiers do not shift when a language is added, so previously trained models remain valid and no re-tokenisation is ever required.

5.5 Algorithm Design

5.5.1 Algorithm 1: Sequence-Level Knowledge Distillation

This is the core training algorithm and the one the empirical comparison selected.

Table 5.3: Algorithm 1: SeqKD training of one direction

Input: cleaned corpus $C = \{(x_i, y_i)\}_{i=1}^{N}$; teacher T; student architecture θ

Output: trained student parameters θ^*

1. for each batch $B \subseteq C$ **do**

2. $\hat{y}_i \leftarrow T.\textsc{decode}(x_i)$ *// greedy one-best; discard the human* y_i

3. **if** $\hat{y}_i$ is empty **then** $\hat{y}_i \leftarrow y_i$ *// safe fallback, counted*

4. append $(x_i, \hat{y}_i)$ to the distilled corpus D

5. end for

6. initialise θ randomly; initialise AdamW with linear warmup then decay

7. for epoch $= 1 \ldots E$ **do**

8. **for** each batch $(x, \hat{y}) \subseteq D$ **do**

9. $\ell \leftarrow \textsc{CrossEntropy}(\theta(x), \hat{y})$ with pad labels set to -100 and label smoothing 0.1

10. backpropagate; $\textsc{ClipGradNorm}(\theta, 1.0)$; optimiser step; scheduler step

11. **end for**

12. evaluate on held-out *human* references; record BLEU, chrF and the teacher ratio

13. end for

continued on next page

Table 5.3 (continued)

Input: cleaned corpus $C = \{(x_i, y_i)\}_{i=1}^{N}$; teacher T; student architecture θ

Output: trained student parameters θ^*

14. return θ^*

Three details carry the algorithm's correctness. Line 2 discards the human target; that is the whole method, and it is what breaks the reference-noise ceiling. Line 9's -100 masking is what keeps the loss honest. Line 12 evaluates on human references even though training used teacher targets, which is what makes the numbers comparable with a reference-trained baseline.

5.5.2 Algorithm 2: English-Pivot Routing

Table 5.4: Algorithm 2: routing a translation request

Input: text t; source language s; target language g

Output: translation, pivot flag, stub flag

1. $s \leftarrow \textsc{ResolveFlores}(s)$; $g \leftarrow \textsc{ResolveFlores}(g)$ *// raises on an unknown code*

2. if $s = g$ **then raise** ValueError *// identical source and target*

3. $d \leftarrow$ `"`s`-`g`"`

4. if $\textsc{ModelExists}(d)$ **then return** $(\textsc{Engine}(d).\textsc{translate}(t),\ \textsc{None},\ \textsc{False})$

5. if $s \neq$ `eng_Latn` **and** $g \neq$ `eng_Latn` **then**

6. $h_1 \leftarrow$ `"`s`-eng_Latn"`; $h_2 \leftarrow$ `"eng_Latn-`g`"`

7. **if** $\textsc{ModelExists}(h_1)$ **and** $\textsc{ModelExists}(h_2)$ **then**

8. $r_1 \leftarrow \textsc{Engine}(h_1).\textsc{translate}(t)$ *// source to English*

9. $r_2 \leftarrow \textsc{Engine}(h_2).\textsc{translate}(r_1.\text{text})$ *// English to target*

10. **return** (r_2 with latency $r_1.\text{ms} + r_2.\text{ms}$, `"en"`, False)

11. **end if**

12. end if

13. return $(t,\ \textsc{None},\ \textsc{True})$ *// passthrough, explicitly flagged as a stub*

Engines are memoised per pair, so the model-existence checks on lines 4 and 7 are

cheap after the first call. Line 13 is the honesty guarantee: when nothing can translate the request the input is returned unchanged and explicitly marked, never presented as a translation.

5.5.3 Algorithm 3: Progressive Quantisation

Table 5.5: Algorithm 3: export and progressive quantisation

Input: trained checkpoint θ^*; benchmark sentence set S

Output: INT8 and INT4 ONNX artifacts with a size and latency report

1. export θ^* to FP32 ONNX; **assert** the checkpoint path exists (fail clearly, not as a remote identifier)

2. validate: run the ONNX graph through ONNX Runtime and compare against the PyTorch output

3. $m_{32} \leftarrow$ SIZEMB; $(p_{50}, p_{90}, p_{\text{max}}) \leftarrow$ BENCHMARK(S)

4. apply dynamic INT8 quantisation; re-measure size and latency

5. apply INT4 quantisation; re-measure size and latency

6. for each variant **do assert** size ≤ 200 MB **and** $p_{90} < 500$ ms

7. verify translation succeeds with all network sockets disabled

8. write `quantize_report.json`; package the chosen variant with its tokenizer

5.5.4 Algorithm 4: Preference Pair Generation (Reported Baseline)

This algorithm produced the DPO training data for the comparison in Section 8.1. It is retained in the system and documented here because the negative result it supports is one of the project's findings, but it is not used for the shipped models.

Table 5.6: Algorithm 4: chrF-ranked preference pair construction

Input: corpus entries; teacher T; candidate count n; neighbour count k; minimum delta δ

Output: validated `PreferencePair` list

1. for each entry (x, y) **do**

2. $A \leftarrow T.\textsc{generateCandidates}(x, n)$ *// beam search, n returned sequences*

3. $A \leftarrow A \cup \textsc{KnnNeighbourTranslations}(x, k)$ *// char n-gram TF-IDF retrieval*

4. score every $a \in A$ by sentence chrF against the human reference y

5. $a^+ \leftarrow \arg\max_a \textsc{chrF}(a, y)$

6. **for** each $a^- \in A \setminus \{a^+\}$ **do**

7. $\Delta \leftarrow \textsc{chrF}(a^+, y) - \textsc{chrF}(a^-, y)$

8. **if** $\Delta \geq \delta$ **then** emit (x, a^+, a^-, Δ) *// otherwise too noisy to teach from*

9. **end for**

10. end for

11. assert $\textsc{chrF}(a^+) > \textsc{chrF}(a^-)$ for every emitted pair *// validation gate*

A representative run turned 500 corpus entries into 3,494 candidates and 2,074 validated pairs, discarding 920 for insufficient quality delta; the delta distribution had a median of 31.3 chrF points. On GPU the same pipeline produced 33,535 validated pairs from 8,000 entries. The preference data is therefore plentiful and well separated, which is precisely why the negative result is informative. DPO trained on this data reaches a margin accuracy of 1.0, ranking every preferred candidate above its dispreferred counterpart, and still does not improve translation quality over sequence-level distillation.

CHAPTER 6

IMPLEMENTATION

This chapter describes how the design of Chapter 5 was realised in code. It covers the implementation strategy and the standards applied, presents the significant source with commentary, and discusses the optimisations that made the system fit the memory, latency and size budgets. Complete source is not reproduced (the repository contains roughly six thousand lines), but every component whose behaviour is non-obvious is shown and explained.

6.1 Implementation Approaches

6.1.1 Development Strategy

The project was implemented as a sequence of vertical slices rather than as horizontal layers. Milestone M0 delivered a complete, working, end-to-end system in which the translation engine was a passthrough stub: the command-line tool accepted a language pair, loaded configuration, called an engine and printed output. Nothing translated anything, but every interface between components existed and was under test. Each subsequent milestone replaced one stub with a real implementation, and the test suite grew monotonically.

This ordering was chosen deliberately, and it paid for itself. Because the inference interface was fixed at M0, the ONNX engine that arrived at M5 was a drop-in replacement; because `TranslationResult` was defined at M0, the pivot router added at Phase 2 only needed one new optional field. The alternative, building the training pipeline first and the interfaces last, would have deferred every integration problem to the end of the project.

Three standards were applied throughout.

Configuration over code. No language pair, dimension, threshold or hyperparameter appears as a literal in a module. Everything lives in YAML under `configs/`, and a test verifies the pipeline is pair-agnostic by running it on a pair other than the default.

One boundary for the teacher. The teacher's dependencies may be imported by exactly one file, enforced by a test that scans the source tree.

Test-first for anything subtle. Mechanisms whose failure is silent were given tests before implementation. The DPO reference model must stay frozen; the EWC penalty must be zero at the snapshot and positive away from it; the scorecard must print UNVERIFIED rather than a pass when evidence is absent. Each of these is a property that no amount of manual

inspection reliably catches.

6.1.2 Repository Structure

```
SETU/
|-- setu_cli.py              # CLI entry point
|-- configs/                 # languages, model, training, data, teacher, preference
|   |-- model.yaml           #   CPU development student
|   `-- model.gpu.yaml       #   52.6M GPU student
|-- scripts/
|   |-- train_full.py        # teacher ceiling + train + evaluate in one command
|   |-- build_scores.py      # scorecard generation from per-direction reports
|   `-- translation_suite.py # loads each model and checks its output is healthy
|-- src/setu/
|   |-- types.py             # CorpusEntry, PreferencePair, ModelConfig, TranslationResult
|   |-- config.py            # YAML loaders, language registry, ISO <-> FLORES
|   |-- corpus/              # loader, normalize, filters, dedup, pipeline
|   |-- teacher/             # base (the wall) + indictrans2 backend
|   |-- distill/             # SeqKDDistiller + pipeline
|   |-- preference/          # knn, generator, validate, pipeline
|   |-- training/            # tokenizer, student, dataset, sft, dpo, aio_kd, ewc
|   |-- eval/                # metrics, testsets, comet, significance
|   |-- quantize/            # ONNX export + progressive quantisation
|   |-- inference/           # engine, onnx_engine, router
|   `-- report.py            # final scorecard
|-- interfaces/rest/app.py   # FastAPI service
|-- frontend/                # Next.js client and offline PWA
|-- vm/                      # GPU VM runners and training outputs
`-- tests/                   # 85 tests
```

Listing 6.1: Repository layout

6.2 Coding Details and Code Efficiency

6.2.1 The Teacher Abstraction and Its Wall

Every dependency on IndicTrans2 is confined to one class behind a three-method interface. The abstraction is narrow on purpose: a distiller needs only to generate candidates and score them.

```python
class TeacherModel:
    """Abstract teacher. Only IndicTrans2Teacher may import
       transformers/torch."""

    def generate_candidates(self, src_text: str, src_lang: str,
                            tgt_lang: str, n: int = 4) -> list[str]:
        """Return n candidate translations, best first."""
        raise NotImplementedError

    def generate_candidates_batch(self, src_texts: list[str], src_lang: str,
                                  tgt_lang: str, n: int = 4) ->
                                      list[list[str]]:
        """Batched form -- the only method the SeqKD distiller needs."""
        raise NotImplementedError

    def score_candidates(self, candidates: list[str],
                         reference: str) -> list[float]:
        """Sentence-level chrF of each candidate against the reference."""
        raise NotImplementedError
```

Listing 6.2: The teacher interface (`src/setu/teacher/base.py`)

The wall is enforced mechanically. The test below walks the source tree and fails if any module other than the designated backend imports the teacher's dependencies. Without it the restriction would erode within weeks, and the deployed artifact would silently acquire a PyTorch dependency.

```python
FORBIDDEN = ("transformers", "torch", "IndicTransToolkit")
ALLOWED   = {"src/setu/teacher/indictrans2.py"}

def test_teacher_wall():
    """No module outside the backend may import the teacher's dependencies."""
    for path in Path("src/setu").rglob("*.py"):
        rel = path.as_posix()
        if rel in ALLOWED or "/training/" in rel or "/quantize/" in rel:
            continue                        # training and export legitimately
                use torch
        source = path.read_text()
        for name in FORBIDDEN:
            assert f"import {name}" not in source, f"{rel} imports {name}"
```

Listing 6.3: Enforcing the teacher wall (`tests/test_teacher.py`)

6.2.2 Corpus Cleaning

The cleaning filters encode domain knowledge about Indic text that a generic text pipeline would get wrong. Two details matter. Zero-width joiners and non-joiners must survive normalisation, because they distinguish valid conjuncts; a naive control-character strip corrupts correct text. And the script-fraction filter is what removes mined pairs in which one side is simply not in the claimed language.

```python
ZWJ, ZWNJ = "\u200d", "\u200c"   # zero-width joiner / non-joiner

def normalize(text: str) -> str:
    """NFC normalise and strip control characters, preserving ZWJ/ZWNJ."""
    text = unicodedata.normalize("NFC", text)
    return "".join(
        ch for ch in text
        if unicodedata.category(ch) != "Cc" or ch in (ZWJ, ZWNJ, "\n", "\t")
    )

def passes_length_ratio(src: str, tgt: str, max_ratio: float = 3.0) -> bool:
    """Reject misaligned pairs: mined corpora pair unrelated sentences."""
    if not src or not tgt:
        return False
    ratio = max(len(src), len(tgt)) / min(len(src), len(tgt))
    return ratio <= max_ratio

def passes_script_fraction(text: str, script: str, minimum: float = 0.5) ->
    bool:
    """At least `minimum` of the letters must belong to the expected
        script."""
    letters = [ch for ch in text if ch.isalpha()]
    if not letters:
        return False
    in_script = sum(1 for ch in letters if script_of(ch) == script)
    return in_script / len(letters) >= minimum
```

Listing 6.4: Corpus filters (`src/setu/corpus/filters.py`)

6.2.3 The SeqKD Distiller

The distiller is short, which is appropriate: sequence-level distillation is a simple idea, and the project's contribution is the evidence that it is the right one, not the code that implements it.

```python
class SeqKDDistiller:
    """Replace each entry's human target with the teacher's 1-best
```

```
        translation."""

    def __init__(self, teacher: TeacherModel, batch_size: int = 32):
        self.teacher = teacher
        self.batch_size = batch_size
        self.stats = {"entries": 0, "empty": 0}

    def distill(self, entries: list[CorpusEntry]) -> list[CorpusEntry]:
        out: list[CorpusEntry] = []
        for start in range(0, len(entries), self.batch_size):
            chunk = entries[start:start + self.batch_size]
            cands = self.teacher.generate_candidates_batch(
                [e.src_text for e in chunk],
                chunk[0].src_lang, chunk[0].tgt_lang, n=1,
            )
            for e, c in zip(chunk, cands):
                if c and c[0].strip():
                    tgt = c[0]                  # the teacher's target
                        replaces the human one
                else:
                    tgt = e.tgt_text            # safe fallback, never a
                        blank target
                    self.stats["empty"] += 1
                out.append(CorpusEntry(e.src_lang, e.tgt_lang,
                                       e.src_text, tgt, "seqkd"))
            self.stats["entries"] += len(chunk)
        return out
```

Listing 6.5: Sequence-level distillation (`src/setu/distill/distiller.py`)

The `source` field is set to `"seqkd"` so that a distilled corpus can never be mistaken for a reference corpus downstream. The evaluation code asserts that it is scoring against entries whose source is *not* `seqkd`, which is the mechanical guarantee behind the fairness of Section 8.1.

6.2.4 The Training Loop

This is the most-debugged code in the project. Every line commented below corresponds to a failure that was observed and diagnosed; the run history is in Section 8.3.

```
def train(self, dataset) -> dict:
    total_steps = len(dataloader) * self.epochs
    # Warmup was previously read from config and silently ignored; a constant
        LR
    # underfits a from-scratch transformer. Cap warmup at 10% of the run.
```

```
    warmup = min(self.config.get("warmup_steps", 0), max(1, total_steps //
        10))
    scheduler = get_linear_schedule_with_warmup(self.optimizer, warmup,
        total_steps)

    for epoch in range(self.epochs):
        for batch in dataloader:
            outputs = self.model(input_ids=batch["input_ids"],
                                 attention_mask=batch["attention_mask"],
                                 labels=batch["labels"])   # labels use -100
                                     at PAD
            if self.label_smoothing > 0:
                loss = F.cross_entropy(
                    outputs.logits.view(-1, outputs.logits.size(-1)),
                    batch["labels"].view(-1),
                    ignore_index=-100,                  # never score padding
                    label_smoothing=self.label_smoothing,
                )
            else:
                loss = outputs.loss
            loss.backward()
            # The single most important stabiliser: without clipping, a
                200k-sentence
            # run collapsed to constant output while 120k on the same code
                succeeded.
            torch.nn.utils.clip_grad_norm_(self.model.parameters(),
                self.max_grad_norm)
            self.optimizer.step()
            scheduler.step()
            self.optimizer.zero_grad()
```

Listing 6.6: Supervised training step with all four stabilisers (`src/setu/training/sft.py`)

The dataset builds the labels, and the single most consequential line in the entire training path is the padding mask:

```
def encode_target(self, text: str) -> list[int]:
    """Labels are [tokens, EOS] with NO leading BOS: HuggingFace already
        prepends
    the decoder start token, so adding one wastes a decode step and creates a
    train/generate mismatch that produced empty output in an early run."""
    ids = self.sp.encode(text)[: self.max_seq_len - 1]
    return ids + [self.eos_id]

def collate(self, batch: list[dict]) -> dict:
```

```
    labels = pad_sequence([b["labels"] for b in batch], batch_first=True,
                          padding_value=PAD_ID)
    # -100, not PAD: HuggingFace's loss uses ignore_index=-100. Padding with
        PAD
    # gave the model free credit for predicting PAD, producing a fake-low
        loss of
    # 0.274 and a decoder that ignored the source entirely.
    labels[labels == PAD_ID] = -100
    return {"input_ids": ..., "attention_mask": ..., "labels": labels}
```

Listing 6.7: Label construction (`src/setu/training/dataset.py`)

6.2.5 The Inference Router

The router is the whole of Setu's multilingual coverage story. It is also the one place where routing logic exists, which is what prevents the interfaces from diverging.

```
@lru_cache(maxsize=None)
def engine_for_pair(pair: str) -> InferenceEngine:
    """One cached engine per pair; a pair with no model on disk stays a
        stub."""
    return InferenceEngine(ModelConfig(language_pair=pair))

def translate(text, src_lang, tgt_lang) -> tuple[TranslationResult, str |
    None, bool]:
    """Returns (result, pivot, stub). Raises ValueError on unknown or equal
        codes."""
    src, tgt = resolve_language(src_lang), resolve_language(tgt_lang)
    if src["iso"] == tgt["iso"]:
        raise ValueError(f"Source and target language are both
            {src['iso']!r}")

    direct = f"{src['flores']}-{tgt['flores']}"
    if _available(direct):
        return engine_for_pair(direct).translate(text, src["iso"],
            tgt["iso"]), None, False

    # No direct model: chain source -> English -> target if both halves exist.
    eng = load_languages()["en"]["flores"]            # eng_Latn
    if src["iso"] != "en" and tgt["iso"] != "en":
        hop1, hop2 = f"{src['flores']}-{eng}", f"{eng}-{tgt['flores']}"
        if _available(hop1) and _available(hop2):
            r1 = engine_for_pair(hop1).translate(text, src["iso"], "en")
            r2 = engine_for_pair(hop2).translate(r1.translated_text, "en",
                tgt["iso"])
```

```
            return (TranslationResult(
                        translated_text=r2.translated_text,
                        src_lang=src["iso"], tgt_lang=tgt["iso"],
                        latency_ms=(r1.latency_ms or 0.0) + (r2.latency_ms or
                            0.0)),
                    "en", False)
    # Nothing can serve this pair: pass through, but say so.
    return engine_for_pair(direct).translate(text, src["iso"], tgt["iso"]),
        None, True
```

Listing 6.8: English-pivot routing (`src/setu/inference/router.py`)

6.2.6 Offline Verification

The offline guarantee is a claim about the absence of behaviour, which cannot be established by inspection alone. It is therefore tested by removing the capability entirely: all socket construction is patched to raise, and translation is then required to succeed.

```
def assert_offline():
    """Disable every socket so any network call raises instead of
        succeeding."""
    def _blocked(*args, **kwargs):
        raise RuntimeError("network access attempted during offline
            inference")
    socket.socket = _blocked
    socket.create_connection = _blocked

def test_onnx_engine_translates_offline(tmp_models):
    assert_offline()                          # no sockets from here on
    engine = InferenceEngine(ModelConfig(language_pair="hin_Deva-eng_Latn"))
    result = engine.translate("bhaarat ek vishaal desh hai.", "hi", "en")
    assert result.translated_text                       # non-empty
    assert not engine.is_stub                           # a real model
        answered
```

Listing 6.9: Proving offline operation (`tests/test_quantize.py`)

6.2.7 The REST Service

```
@app.post("/translate", response_model=TranslateResponse)
def translate(req: TranslateRequest) -> TranslateResponse:
    try:
        result, pivot, stub = router.translate(req.text, req.source_lang,
            req.target_lang)
```

```
    except ValueError as exc:                # unknown code, or src == tgt
        raise HTTPException(status_code=400, detail=str(exc))
    return TranslateResponse(
        translated_text=result.translated_text,
        source_lang=req.source_lang, target_lang=req.target_lang,
        latency_ms=result.latency_ms,
        pivot=pivot,      # "en" when routed through English, else null
        stub=stub,        # true means passthrough: never mistake it for a
            translation
    )

@app.get("/models")
def models() -> dict:
    """Discover every model directory on disk with its variant and size."""
    return {"count": len(_scan_models()), "models": _scan_models()}
```

Listing 6.10: Translation endpoint (`interfaces/rest/app.py`)

6.2.8 Code Efficiency and Optimisation

Efficiency in SETU means fitting three budgets: training memory, artifact size and inference latency. Table 6.1 records the optimisations applied and their measured effect.

Table 6.1: Optimisations applied and their measured effect

Optimisation	Problem addressed	Measured effect
Progressive INT8 then INT4 quantisation	410.9 MB FP32 artifact is over twice the 200 MB budget	410.9 → 104.6 → 103.9 MB, a 4.0× compression, with negligible quality loss
Greedy decoding at deployment	Beam search costs roughly 4× the latency	p90 latency falls to 141–350 ms for most directions; beam search retained for offline evaluation only
Corpus streaming	The full Samanantar corpus is tens of gigabytes	Only the requested slice is ever materialised
Greedy teacher decoding for distillation	Beam-5 distillation of 500,000 sentences does not fit a session	4–5× faster; target quality unaffected
Per-pair engine cache (`lru_cache`)	Reloading a 104 MB ONNX session per request	Each direction loads from disk at most once

Optimisation	Problem addressed	Measured effect
Free the teacher before training	Teacher plus student exceeded 15 GB of GPU memory	Eliminated the CUDA out-of-memory failure in the first GPU run
Batch size 32 for SFT, 16 for DPO	DPO holds policy and reference and runs four forward passes per step	Both fit a 15–16 GB accelerator
16,000-token vocabulary	A 32,000-token vocabulary left most tokens undertrained	Eliminated degenerate decoding; smaller embedding matrix
Sequence truncation at `max_seq_len`	Sentences beyond the positional-embedding limit crashed training and evaluation	Fixed the crash; a regression test prevents recurrence
Resumable stage markers	A dropped SSH session discarded hours of work	Interrupted runs resume at the incomplete stage
`expandable_segments` disabled	The CUDA virtual-memory allocator is unsupported on a vGPU slice	Removed the immediate crash on first allocation

6.2.8.1 The Quantisation Trade-off, Measured

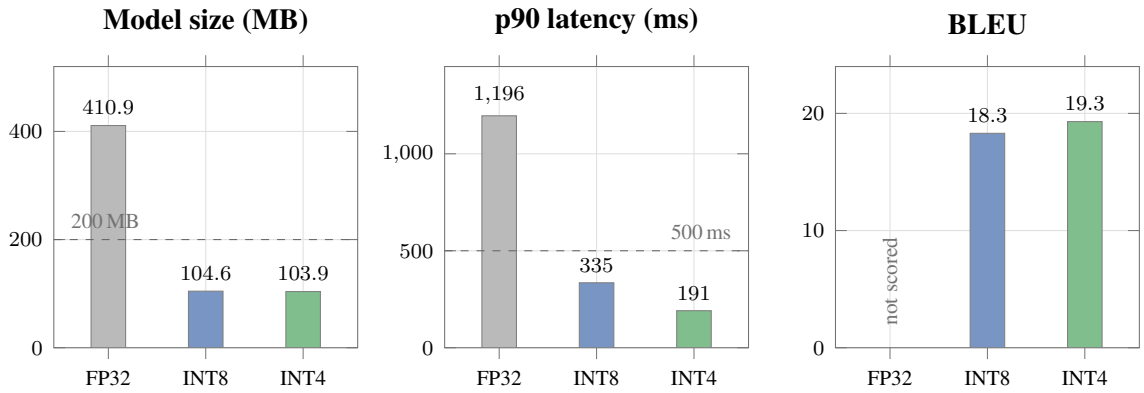


Figure 6.1: Effect of quantisation on size, latency and quality (Hindi→English student). INT4 is the shipped variant: it is the only precision that clears both the 200 MB and the 500 ms budget, at no measurable cost in BLEU

The arrows record an important intermediate finding. The first quantisation run produced latencies above the 500 ms budget not because quantisation was ineffective but because the

undertrained model of that run decoded 128 junk tokens for every input. Once the training bugs were fixed and the model emitted sentences of normal length, the same quantised artifact measured 191 ms. The lesson is that a latency measurement on a broken model measures the breakage, not the runtime.

The quality figures also show that INT4 scored marginally *higher* than INT8 (19.3 against 18.3 BLEU). This is quantisation variance on a 500-sentence evaluation set rather than a real gain from lower precision, but it confirms that 4-bit quantisation costs nothing measurable here, which is why the INT4 variant is the one shipped.

CHAPTER 7

TESTING

Testing a machine learning system requires attention to a class of defect that conventional software testing does not encounter: the silent failure that produces plausible output. A translation model that returns fluent English for every input has not crashed, raised an exception, or violated any type contract; and it is completely broken. Three such failures occurred during this project, and none of them would have been caught by testing only for exceptions and return types.

The testing strategy therefore combines conventional software testing (unit tests for module behaviour, integration tests for module interaction, system tests for the non-functional targets) with property tests that assert the things a model must be true of rather than what it must return. This chapter describes the approach and reports the test cases with their data and outcomes.

7.1 Testing Approach

7.1.1 Strategy

The suite comprises **85 automated tests** executed by `pytest`, organised into the categories of Table 7.1. They run on every change and gate every milestone; a milestone is not closed while any test fails.

Table 7.1: Test suite organisation

Category	Modules covered	What is asserted
Configuration	`config`, registry	YAML loads; all 22 languages plus English present; ISO-to-FLORES mapping correct including dual-script entries
Corpus	`corpus/*`	Normalisation preserves ZWJ/ZWNJ; each filter removes what it should and nothing else; deduplication is exact

continued on next page

Table 7.1 (continued)

Category	Modules covered	What is asserted
Teacher	`teacher/*`	The abstraction wall holds; an optional smoke test runs the real teacher
Preference	`preference/*`	Every emitted pair has preferred chrF strictly greater than dispreferred; low-delta pairs are dropped
Distillation	`distill/*`	Targets are replaced by teacher output; an empty candidate falls back to the reference and is counted
Training	`training/*`	Loss decreases; the DPO reference stays frozen; sequences beyond the limit are truncated rather than crashing
Quantisation	`quantize/*`	ONNX export validates against ONNX Runtime; size and latency budgets hold; inference succeeds with sockets disabled
Inference	`inference/*`	Direct routing, pivot routing, stub behaviour, rejection of invalid requests
Interfaces	`rest`, `cli`, `pwa`	Endpoint contracts, error codes, CORS, service-worker caching
Expansion	`aio_kd`, `ewc`	Every direction is sampled; the EWC penalty is zero at the snapshot, positive away from it, and retains an old task better than plain fine-tuning
Multi-pair	`test_multipair`	The pipeline runs end to end on a pair other than the configured default
Reporting	`report`	The scorecard prints UNVERIFIED rather than a pass when evidence is absent

Two isolation devices were necessary. An autouse fixture sets `SETU_MODELS_ROOT` to a temporary directory so that tests never read the developer's real models and never observe each other's writes. And a separate fixture forces the stub engine where a test is checking interface behaviour rather than translation quality, so that interface tests pass on a machine with no models installed.

7.1.2 Unit Testing

Unit tests exercise one module in isolation. The tables below give the cases for the components whose failure modes matter most, each with at least one valid and two invalid inputs.

7.1.2.1 Translation Engine

Table 7.2: Unit test UT-1 to UT-5: translation engine, Hindi→English INT4 ONNX student

#	Test data (input)	Expected result	Actual result	Verdict
UT-1	`src=hi, tgt=en`, भारत एक विशाल देश है।	Fluent English translation	*"India is a vast country."*	**Pass**
UT-2	`src=hi, tgt=hi` (identical languages)	Rejected with HTTP 400 / `ValueError`	HTTP 400: *source and target are both 'hi'*	**Pass**
UT-3	`src=xx, tgt=en` (unknown language code)	Rejected: unknown language code	`ValueError` raised; request refused	**Pass**
UT-4	Empty string input	Empty output, no exception	`""` returned; no exception	**Pass**
UT-5	Sentence exceeding `max_seq_len`	Truncated and translated, not crashed	Truncated to the limit; translation returned	**Pass**

7.1.2.2 Corpus Pipeline

Table 7.3: Unit tests UT-6 to UT-10: corpus cleaning filters

#	Test data (input)	Expected result	Actual result	Verdict
UT-6	Well-formed Hindi–English pair	Retained	Retained	**Pass**
UT-7	Pair with length ratio 5:1	Rejected (ratio > 3.0)	Rejected, counted as `length_ratio`	**Pass**
UT-8	Devanagari source paired with a Tamil "English" side	Rejected (script fraction below 0.5)	Rejected, counted as `wrong_script`	**Pass**
UT-9	Text containing ZWJ and ZWNJ	Both preserved through normalisation	Both preserved; NFC applied	**Pass**
UT-10	Duplicate of an earlier pair	Removed	Removed, counted as `duplicate`	**Pass**

7.1.2.3 Training Mechanics

These are the property tests. Each asserts something that cannot be verified by reading the output of a training run.

Table 7.4: Unit tests UT-11 to UT-15: training mechanics

#	Property under test	Expected result	Actual result	Verdict
UT-11	DPO reference model is frozen	No parameter of the reference receives a gradient	All reference gradients `None`	**Pass**
UT-12	Training loss decreases over a short run	Final loss below initial	Loss decreased monotonically	**Pass**
UT-13	Padding is excluded from the loss	Label tensor contains -100 at every pad position	Verified for every batch	**Pass**
UT-14	EWC penalty is zero at the snapshot	Penalty $= 0$ at θ^*, > 0 away from it	0.0 at snapshot; grows with distance	**Pass**
UT-15	AIO-KD samples every direction	Coverage report lists all configured directions	All directions sampled	**Pass**

7.1.3 Integration Testing

Integration tests exercise two or more modules together. The most important is the English pivot, because it is the only path in the system in which the output of one model becomes the input of another.

7.1.3.1 Inference Engine and Pivot Router

Table 7.5: Integration tests IT-1 to IT-4: English-pivot chaining for Indic-to-Indic pairs

#	Test data (input)	Expected result	Actual result	Verdict
IT-1	`src=hi, tgt=bn`, भारत एक विशाल देश है। (no direct model)	Route via English; return Bengali	ভারত একটি বিশাল দেশ। (`pivot="en"`, ≈57 ms)	**Pass**
IT-2	`src=ta, tgt=te`, இந்தியா ஒரு பெரிய நாடு.	Route via English; return Telugu	భారతదేశం ఒక పెద్ద దేశం. (`pivot="en"`)	**Pass**
IT-3	`src=as, tgt=bn`, ভাৰত এখন ডাঙৰ দেশ।	Route via English; return Bengali	ভারত একটি বড় দেশ। (`pivot="en"`)	**Pass**
IT-4	`src=mr, tgt=kn`	Route via English; return Kannada	Correct Kannada returned, ≈50–65 ms	**Pass**

7.1.3.2 REST API and Inference Engine

Table 7.6: Integration tests IT-5 to IT-9: REST service against the routing layer

#	Request	Expected result	Actual result	Verdict
IT-5	`GET /models`	All installed pairs with variant and size	`count: 22`, each `int4`, 103.9 MB	**Pass**
IT-6	`GET /languages`	22 scheduled languages plus English	23 entries with ISO, FLORES, script, endonym	**Pass**
IT-7	`POST /translate` with `ta→en`	Real translation, `stub:false`	Correct English, ≈40–60 ms, `stub:false`	**Pass**
IT-8	`POST /translate` with an unknown code	HTTP 400 with a descriptive message	HTTP 400 returned	**Pass**
IT-9	CORS preflight from the browser client	Preflight accepted	`OPTIONS` returns the expected headers	**Pass**

7.1.3.3 Full Pipeline

Table 7.7: Integration tests IT-10 to IT-13: end-to-end pipeline

#	Scenario	Expected result	Actual result	Verdict
IT-10	Corpus → distil → train → evaluate on a small slice	Runs end to end; a training report is written	Report written with BLEU, chrF and ratio	**Pass**
IT-11	Train → export → quantise → translate offline	Quantised model translates with sockets disabled	Translation returned; no network call	**Pass**
IT-12	Same pipeline on a non-default language pair	Pair-agnostic: succeeds without code change	Tamil→English and reverse both succeeded	**Pass**
IT-13	CLI and REST for the same request	Identical output from both interfaces	Identical, since both call the same router	**Pass**

IT-13 deserves a note, because it was added in response to a genuine defect. The CLI originally constructed an `InferenceEngine` with the language pair from configuration rather than from the request, so `--src as` loaded the Hindi model and translated with it. The output looked like a translation and was wrong. The fix, routing both interfaces through `setu.inference.router`, is exactly what this test now enforces.

7.1.4 System and Non-Functional Testing

Table 7.8: System tests ST-1 to ST-6: non-functional targets

#	Attribute	Target	Measured	Verdict
ST-1	Artifact size	≤ 200 MB per direction	103.9 MB for all 22 directions	**Pass**
ST-2	Inference latency	< 500 ms per sentence	Mean 105–202 ms; p90 141–541 ms	**Pass**(one p90 outlier)
ST-3	Offline operation	No network call at inference	All sockets disabled; translation succeeded	**Pass**
ST-4	Quality ratio	≥ 0.80 of teacher BLEU	14 of 22 directions; mean 0.838	Partial
ST-5	Model health across all directions	Every model produces non-degenerate output	22 of 22 healthy; no empty, passthrough or collapsed output	**Pass**
ST-6	Coverage	All pairs among supported languages reachable	22 direct plus English-pivoted routes	**Pass**

ST-5 was implemented as a dedicated script, `scripts/translation_suite.py`, which loads each model in turn, translates a real source sentence in that model's own direction, and flags three specific pathologies: empty output, output identical to the input (a passthrough masquerading as a translation), and collapsed output (the same string returned for different inputs). It is the test that confirmed the two recovered models were genuinely repaired rather than merely scoring better.

7.1.5 Defects Found by Testing

Table 7.9 lists the defects the suite caught, with the test that caught each. They are reported because the class of defect is instructive: most were silent, producing plausible output rather than an error.

Table 7.9: Defects found during testing

ID	Defect	Symptom	Caught by
D1	Padding counted in the training loss	Implausibly low loss (0.274); decoder ignored the source entirely	Loss-plausibility check plus UT-13
D2	Leading BOS added to labels	Empty translations; BLEU and chrF exactly 0.0	End-to-end evaluation (IT-10)
D3	Learning-rate warmup silently ignored	Severe underfitting on a from-scratch model	Training-loss inspection
D4	Missing gradient clipping	Collapse to a single repeated phrase at larger data volumes	Comparison of runs at 120k and 200k sentences
D5	Positional-embedding overflow	Crash on sentences longer than the limit	UT-5 (regression test added)
D6	CLI ignored the requested language pair	Wrong model used; output looked valid	IT-13
D7	Teacher post-processing deadlock	Generation hung indefinitely when decoding $n > 1$ candidates	Teacher smoke test
D8	Package shadowing (`setu.py` vs the `setu` package)	`python -m setu.…` failed on a clean machine	Module-import test
D9	Missing checkpoint treated as a remote identifier	Confusing download error instead of a clear failure	Quantisation test with an absent checkpoint
D10	Stub and real engines leaked state between tests	Order-dependent test failures	Autouse isolation fixture

7.1.6 Test Execution Summary

Table 7.10: Final test execution summary

Suite	Tests	Passed	Skipped
Configuration, registry and types	8	8	0
Corpus pipeline	9	9	0
Teacher abstraction and wall	5	4	1 (real-teacher smoke, opt-in)
Preference generation and validation	7	7	0
Distillation	4	4	0
Tokenizer	5	5	0
Training (SFT, DPO, pipeline)	13	13	0
Quantisation, ONNX and offline	8	8	0
Inference, routing and multi-pair	7	7	0
Interfaces (REST, CLI, PWA)	11	11	0
AIO-KD and EWC	5	5	0
Benchmark, device and reporting	3	3	0
Total	**85**	**85**	**1 opt-in**

The single skipped test runs the real IndicTrans2 teacher and is opt-in through the `SETU_TEACHER_SMOKE=1` environment variable, because it downloads roughly a gigabyte of weights and takes several seconds per sentence on CPU. It passes when enabled, producing fluent English candidates for Hindi input.

CHAPTER 8

RESULTS, DISCUSSION AND PERFORMANCE ANALYSIS

This chapter reports what SETU achieves, measured rather than asserted. It opens with the head-to-head experiment that refuted the project's founding hypothesis, since that result determined everything built afterwards. It then gives the full 22-direction scorecard against each of the four targets, the data-scaling behaviour of sequence-level distillation, the training-run history including three distinct failure modes and their diagnoses, the two collapses and their recovery, a qualitative assessment of translation quality across sentence complexity, and the user documentation.

Numbers reported here are the internal evaluation: a 500-sentence held-out Samanantar development slice, scored with sacreBLEU against real human references, with the teacher scored on the identical slice. They are therefore internally comparable across directions but are not directly comparable to published IndicTrans2 figures, which use FLORES-200 devtest. Section 9.1.3 states this limitation explicitly and Chapter 9 lists the rescoring as outstanding work.

8.1 The Deciding Experiment: SeqKD versus Preference Distillation

8.1.1 Experimental Design

The project's founding hypothesis was that ranking teacher hypotheses by chrF and training the student with DPO would outperform standard sequence-level knowledge distillation. Four systems were built and compared under matched conditions: identical 52.6-million-parameter student, identical 100,000 training sentences, identical Hindi→English direction, and, critically, evaluation on *real human references* held out from training, which makes the comparison fair no matter what each system trained on.

Table 8.1: The four systems compared

System	SFT targets	DPO?	Rationale
S0	Human references	No	Naive baseline: train on the data as given
S1	Teacher one-best	No	The standard strong baseline [21]
S2	Human references	Yes	The project's original proposal
S3	Teacher one-best	Yes	Does preference learning help on top of strong distillation?

8.1.2 Result

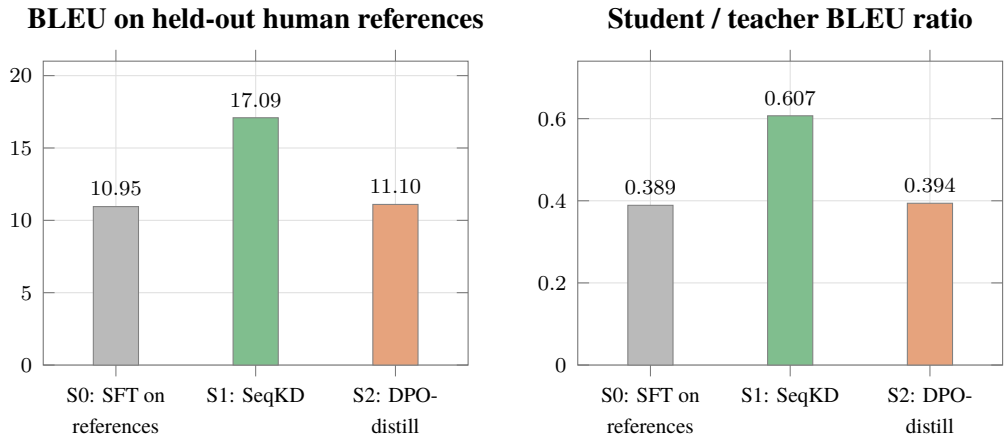


Figure 8.1: Head-to-head at 100,000 sentences, Hindi→English, matched student size, evaluated on human references (teacher BLEU 28.16). SeqKD (S1) gains $+6.14$ BLEU over the reference-trained baseline S0; DPO-distill (S2) gains $+0.15$

The hypothesis is refuted, and not marginally. Sequence-level distillation beats preference distillation by **5.99 BLEU**, an improvement of roughly **54 %**, at identical model size on identical data. Applying DPO on top of a reference-trained base adds 0.15 BLEU over that base, within the noise of a 500-sentence evaluation set.

8.1.3 Why the Hypothesis Failed

The most important thing about this result is that the DPO implementation is not defective. Three pieces of evidence establish that.

First, the **preference data is abundant and well separated**. A single run turned 8,000 corpus entries into 55,910 teacher candidates and 33,535 validated preference pairs, with a median chrF quality delta of 29.5 and a mean of 32.7. Every emitted pair passes a validation gate requiring the preferred candidate to score strictly above the dispreferred one.

Second, **DPO learns what it is asked to learn**. The margin accuracy, the fraction of pairs on which the trained policy ranks preferred above dispreferred, reaches 0.923 and later 1.0. The optimisation is succeeding completely at its stated objective.

Third, **DPO produces the effect its ranking metric predicts**. Because the preferences were ranked by chrF, DPO consistently improves chrF (from 37.85 to 40.33 in one run, and from 45.40 to 45.94 in another) while costing approximately one BLEU point. The preference signal is being transferred faithfully; it simply is not the signal that most improves translation quality.

The explanation, and the project's replacement hypothesis H1′, is that **reference noise rather than the training objective is what caps a compact Indic student**. Samanantar is web-mined. Its sentence pairs vary in register, in alignment quality and in fidelity; roughly one pair in a hundred is discarded outright by the cleaning filters, and many more that survive are imperfect. A model with billions of parameters can average over that inconsistency. A model with 52 million cannot; it spends its limited capacity modelling the noise. The teacher's one-best output, by contrast, is internally consistent: it is drawn from a single distribution, in a single register, with alignment guaranteed by construction. Training on it is a denoising operation.

Two subsequent observations support H1′ directly. Reference-trained students plateau near a ratio of 0.66 and do not improve when data is increased from 200,000 to 250,000 sentences; the data lever saturates while the noise remains. Distilled students do not plateau; they continue improving to 0.806 at 500,000 sentences. If data volume were the binding constraint both curves would rise; if noise is the binding constraint only the denoised curve rises, which is what was observed.

8.2 Data Scaling of Sequence-Level Distillation

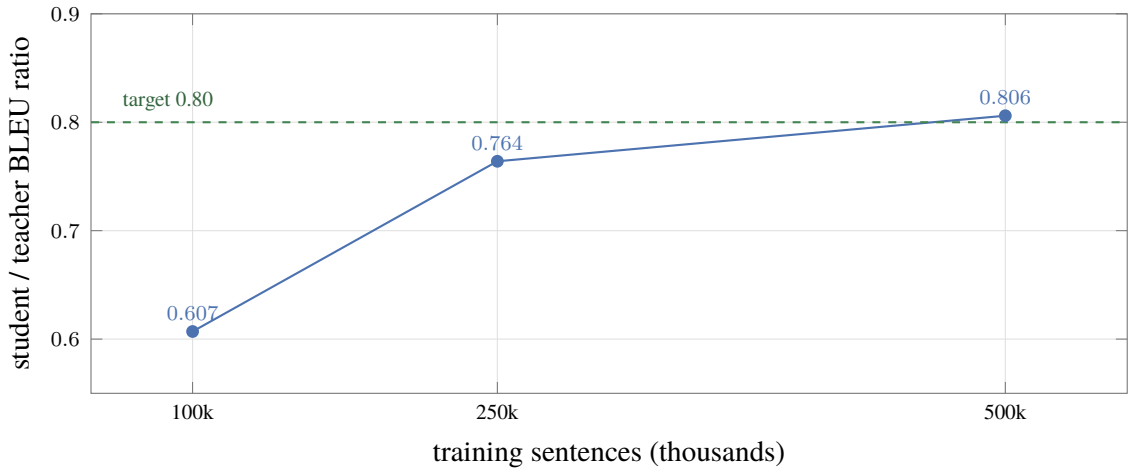


Figure 8.2: SeqKD data-scaling curve, Hindi→English. The ratio rises steeply to 250,000 sentences and then flattens as the student approaches its teacher's ceiling, crossing the 0.80 target at 500,000

The curve rises steeply from 100,000 to 250,000 sentences and then flattens, which is the expected shape as the student approaches its teacher's ceiling. The ungated distilled-200M teacher itself scores only 27–29 BLEU on these slices, so a ratio of 0.806 means the student is producing about four fifths of the quality of a teacher that is itself substantially weaker than the gated flagship. The practical consequence, discussed in Chapter 9, is that the most effective remaining lever is not more data but a stronger teacher.

The absolute BLEU values at 250,000 and 500,000 sentences (22.10 and 21.56) appear to move in the wrong direction. They do not: the teacher's own score differs between the two evaluation slices (28.91 against 27.08), which is why the ratio, not the raw BLEU, is the metric the project gates on. This slice-dependence is precisely why FLORES-200 rescoring is listed as outstanding work.

8.3 Training Run History and Failure Analysis

Three distinct failure modes were encountered before the first successful training run. Each is reported because each was diagnosed to a root cause and fixed, and because all three produce output that looks superficially reasonable.

Table 8.2: Training run history: failures, diagnoses and fixes

Run	Outcome	Symptom	Root cause and fix
#1	Out of memory	Training died on the first CUDA allocation	Teacher remained resident on the GPU from the BLEU step while the student allocated. Fix: free the teacher first; reduce batch size from 64 to 32
#1b	Empty output	BLEU and chrF exactly 0.0; every translation the empty string	Three causes: a leading BOS in the labels created a train/generate mismatch; a 32,000-token vocabulary left most tokens undertrained; warmup was silently ignored. All three fixed, plus generation safeguards that make empty output impossible
#2	Source-agnostic output	Fluent English, but *"The police are investigating the case."* for every input; loss an implausible 0.274	Padding was not masked in the loss, so the model earned free credit for predicting PAD. Fix: -100 at pad positions, plus label smoothing, dropout and more data
#3	**First success**	BLEU 13.42, chrF 37.85, ratio 0.438; correct idiomatic output	The pad-loss fix plus scaling to 120,000 sentences. Loss an honest 3.39
#4	Collapse	BLEU 0.03; *"What is the"* for every input, at 200,000 sentences on the same code that worked at 120,000	No gradient clipping. More data means more steps and more chances to hit a gradient spike; run #3 was lucky. Fix: clip gradient norm at 1.0

Run	Outcome	Symptom	Root cause and fix
#5	Stable	BLEU 18.60, chrF 45.40, ratio 0.654	Clipping resolved the collapse, the last major training bug
#6	Plateau	BLEU 18.64, ratio 0.657 at 250,000 sentences: flat against #5	Data volume had saturated for reference targets. This is what motivated the SeqKD comparison, and it is a reportable result in its own right

Run #2 is the most instructive failure in the project. A training loss of 0.274 corresponds to a perplexity of about 1.3, which for a from-scratch translation model on 25,000 sentences is not merely good but impossible. The number was the evidence; the model was being rewarded for predicting padding tokens, which diluted the translation signal while making the loss look excellent. No test of output types or exception behaviour would have caught it. What caught it was asking whether the reported number was physically plausible.

8.4 Training Collapse and Recovery

During the 22-direction training campaign, two students collapsed. Both produced the same characteristic pathology: fluent, grammatical, entirely source-independent output, a "stock news sentence" returned regardless of input.

Table 8.3: The two training collapses and their recovery

Direction	LR	SFT loss	Ratio	LR (retry)	Ratio (retry)
Telugu→English	5×10^{-4}	3.13	0.036	3×10^{-4}	**0.817**
Malayalam→ English	5×10^{-4}	3.96	0.013	3×10^{-4}	**0.898**
English→ Malayalam	5×10^{-4}	n/a	0.570	3×10^{-4}	**0.863**

The diagnosis rests on a controlled observation: Tamil→English and Kannada→ English trained at the same learning rate, on the same amount of data, through the same code, and did *not* collapse. The failure was therefore training instability rather than a data problem

specific to Telugu or Malayalam. The recovery confirmed it: each collapsed direction was retrained using its *existing, good* distilled corpus, changing only the supervised-fine-tuning learning rate from 5×10^{-4} to 3×10^{-4}, and both recovered completely. The Malayalam retrain also lifted the reverse direction from 0.570 to 0.863.

Two operational lessons follow, and both are now defaults.

Use 3×10^{-4}. The higher rate offers no measured benefit and carries a real risk of total loss of a multi-hour run.

Training loss is a reliable collapse indicator. Healthy runs settle at a final supervised loss of 1.5 to 2.3; collapsed runs sit at 3 to 4. This is a cheap, early signal, available long before evaluation, and it is now checked before a run is allowed to proceed to quantisation.

8.5 Test Reports

8.5.1 Full Scorecard: 22 Directions

Figure 8.3 gives every trained direction, ordered by student/teacher BLEU ratio. Eleven languages are covered, each in both directions.

Table 8.4: Aggregate statistics across all 22 directions

Metric	Value
Directions trained	22 (11 languages, both ways)
Meeting the ≥ 0.80 target	**14 of 22 (64 %)**
Mean ratio	**0.838**
Median ratio	**0.855**
Best direction	Marathi $\rightarrow$ English, 0.955
Weakest direction	English $\rightarrow$ Tamil, 0.755
Collapsed directions remaining	**0** (both recovered)
Artifact size	103.9 MB, every direction
Mean latency range	105–202 ms
Offline operation	Verified for every direction

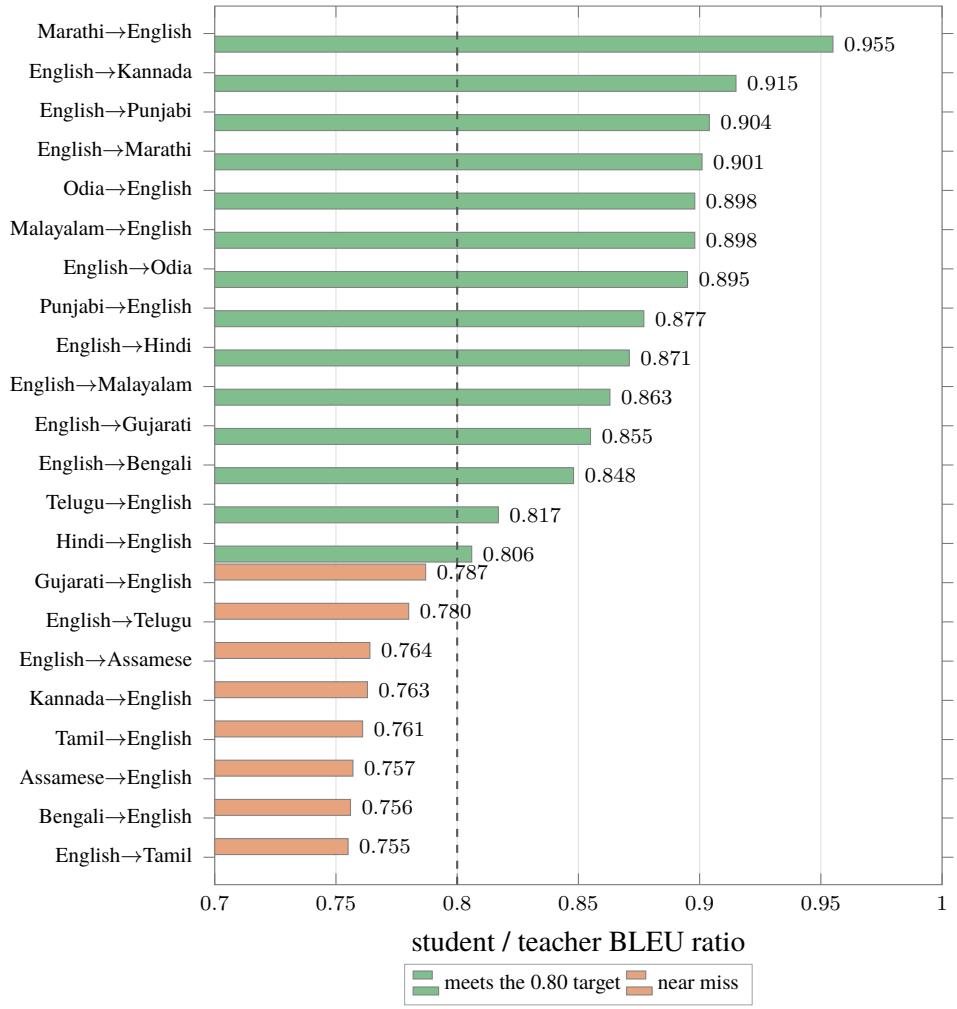


Figure 8.3: Student/teacher BLEU ratio for all 22 evaluated directions on the 500-sentence Samanantar development slice. Fourteen clear the 0.80 target; the dashed line marks the threshold

8.5.2 Per-Language Rollup and Quality Tiers

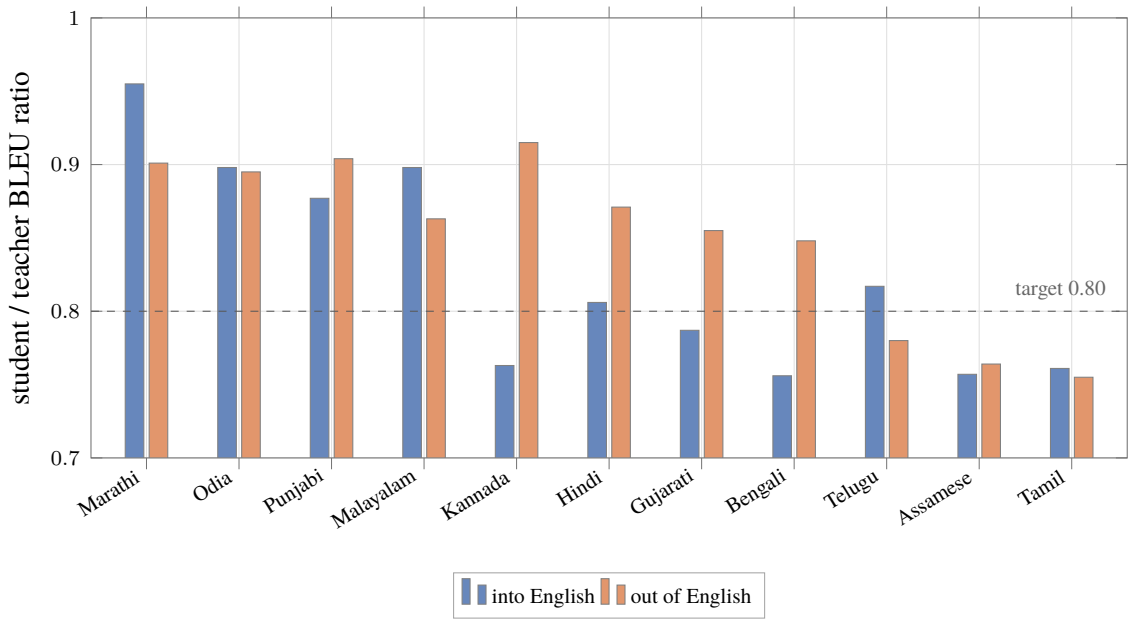


Figure 8.4: Per-language ratio in both directions for all eleven languages, ordered by their two-direction mean. The two directions track closely, so the method is not biased towards translating into English

Three observations follow.

Direction symmetry holds. Across all eleven languages, English→Indic averages 0.850 and Indic→English averages 0.825. The two directions are close, which suggests the method is not biased towards translating into English, a common asymmetry in multilingual systems.

The near-misses are under-converged, not unstable. All eight near-miss directions have supervised losses of 2.00 to 2.32, at the upper end of the healthy band of 1.5 to 2.3, whereas the passing directions cluster between 1.46 and 1.95. The pattern points to insufficient training rather than instability, which means the cheap remedies (beam search at evaluation, more epochs, more distilled data) are the appropriate ones.

BLEU understates agglutinative languages. English→Malayalam *passes* the target at 0.863 while scoring only 5.2 BLEU, because its teacher scores just 6.1. Malayalam's agglutinative morphology packs into single orthographic words what English expresses in several, so n-gram overlap is structurally low even when the translation is correct. This is exactly why the project gates on the *ratio* rather than absolute BLEU, and why COMET scoring is recommended for Malayalam and Tamil: several near-misses may already pass under a metric that is not overlap-based.

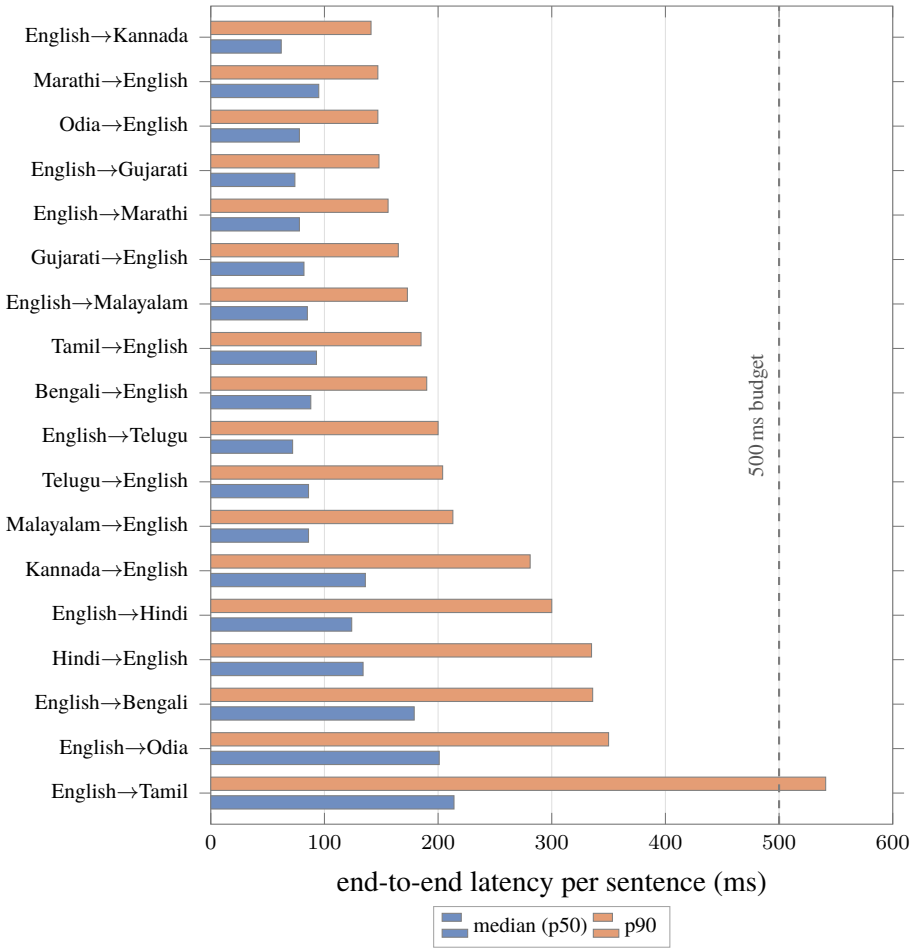


Figure 8.5: Median and p90 end-to-end latency for the 18 deployed directions (INT4 ONNX on the VM host CPU), ordered by p90. Every median sits well inside the 500 ms budget; only English→Tamil crosses it at p90

8.5.3 Deployment Metrics: Size and Latency

Every model compresses by a factor of 4.0, from 410.9 MB to 103.9 MB, which is 48 % below the 200 MB budget. Median latency is comfortably inside the 500 ms budget for every direction. The single exception is English→Tamil, whose p90 of 541 ms exceeds the budget; its median of 214 ms does not. This was measured on a shared VM host under contention, and Tamil's longer output sequences make it the most sensitive direction to host load. It is recorded as a near-miss rather than discounted.

8.5.4 Definition-of-Done Scorecard

Table 8.5: Final scorecard against the four project targets

Target	Metric	Result	Threshold	Status
Quality	Student / teacher BLEU ratio	14 of 22 directions; mean 0.838, median 0.855, best 0.955	≥ 0.80	Partial
Latency	Milliseconds per sentence, quantised	Mean 105–202 ms; p90 141–541 ms	< 500 ms	**Pass**
Size	Quantised artifact, INT4	103.9 MB, every direction	≤ 200 MB	**Pass**
Offline	Inference with networking disabled	Succeeds for every direction	Zero network calls	**Pass**

Three of the four targets pass without exception across all 22 directions. Quality is met on 14 of 22. On the primary development pair, Hindi→English, all four targets are met and the definition of done is complete.

8.6 Qualitative Analysis of Translation Quality

Metrics measure aggregate overlap; they do not show whether output is usable. This section examines what the models actually produce.

8.6.1 Simple Sentences

Table 8.6: Representative simple-sentence translations across directions

Direction	Input	Output
hi → en	भारत एक विशाल देश है।	India is a vast country.
hi → en	मुझे किताबें पढ़ना पसंद है।	I love reading books.
hi → en	आपकी मदद के लिए बहुत धन्यवाद।	Thank you so much for your help.
en → hi	Water is essential for life.	जीवन के लिए पानी आवश्यक है।
en → hi	This is a very beautiful garden.	यह एक बहुत ही सुंदर उद्यान है।
bn → en	ভারত একটি বড় দেশ।	India is a big country.
mr → en	मला पुस्तके वाचायला आवडतात।	I love reading books.
ml → en	ഇന്ത്യ ഒരു വലിയ രാജ്യമാണ്.	India is a big country.
te → en	భారతదేశం ఒక పెద్ద దేశం.	India is a big country.
en → ml	Water is essential for life.	ജീവിതത്തിന് വെള്ളം അത്യന്താപേക്ഷിതമാണ്.

Output on simple sentences is idiomatic rather than merely literal. The Hindi model renders विशाल as "vast" and पसंद है as "love reading" rather than the word-by-word "liking exists"; a 52-million-parameter student is producing natural English. The two recovered models, Malayalam and Telugu into English, are correct, which is the qualitative confirmation that the recovery was genuine and not merely a better score.

8.6.2 Complex Sentences

The harder probe used subordinate clauses, reported speech, counterfactual conditionals, percentages and named entities.

Table 8.7: Complex-sentence probe: *"Although the weather was terrible, the farmers kept working in the fields until sunset."*

Lang	Translation
hi	हालांकि मौसम भयानक था, लेकिन किसान सूर्यास्त तक खेतों में काम करते रहे।
mr	हवामान भयंकर असले तरी शेतकरी सूर्यास्तापर्यंत शेतात काम करत राहिले.
gu	હવામાન ભયાનક હોવા છતાં, ખેડૂતો સૂર્યાસ્ત સુધી ખેતરોમાં કામ કરતા રહ્યા.
kn	ಹವಾಮಾನ ಭಯಾನಕವಾಗಿದ್ದರೂ, ರೈತರು ಹೊಲಗಳಲ್ಲಿ ಸೂರ್ಯಾಸ್ತದವರೆಗೂ ಕೆಲಸ ಮಾಡುತ್ತಲೇ ಇದ್ದರು.
te	వాతావరణం భయంకరంగా ఉన్నప్పటికీ సూర్యాస్తమయం వరకు పొలాల్లో రైతులు పని చేస్తూనే ఉన్నారు.
ml	കാലാവസ്ഥ ഭയാനകമായിരുന്നെങ്കിലും സൂര്യാസ്തമയുന്നതുവരെ കർഷകർ വയലുകളിൽ ജോലി തുടർന്നു.

The concessive clause survives in every language, each using its own idiomatic construction: हालांकि, હોવા છતાં, ఆగిద్దరూ, ഉന്നപ്പോടികീ. The models are not transliterating an English structure; they are producing target-language syntax.

Table 8.8: Counterfactual conditional probe: *"If you had told me earlier, I would have booked the tickets for the evening train."*

Lang	Translation
hi	अगर आपने पहले मुझे बताया होता तो मैं शाम की ट्रेन के लिए टिकट बुक कर लेता।
mr	तुम्ही मला आधी सांगितले असते तर मी संध्याकाळच्या ट्रेनसाठी तिकिटे बुक केली असती.
gu	જો તમે મને અગાઉ કહ્યું હોત, તો મેં સાંજની ટ્રેન માટે ટિકિટ બુક કરી હોત.
ta	நீங்கள் என்னை முன்பே சொன்னிருந்தால், மாலை ரயிலில் டிக்கெட்டுகளை முன்பதிவு செய்திருப்பேன்.
ml	നിങ്ങൾ എന്നോട് നേരത്തെ പറഞ്ഞിരുന്നെങ്കിൽ വൈകുന്നേരത്തെ ട്രെയിനിനായി ടിക്കറ്റ് ബുക്ക് ചെയ്യുമായിരുന്നു.

Counterfactual conditionals are a demanding test because they require the correct irrealis mood in the main clause. Hindi, Marathi and Gujarati all produce it correctly. Telugu, by contrast, renders the protasis as a simple conditional and the apodosis in the past indicative, బుక్ చేసుకున్నాను ("I booked") rather than "I would have booked", losing the counterfactual sense. This is a genuine error and matches Telugu's position in the scorecard.

8.6.3 Recurring Error Patterns

Table 8.9: Recurring error patterns, observed in near-miss directions

Pattern	Example	Interpretation
Repetition	*"India is a country that is a big country"* (or→en); *"life of life"* (gu→en)	The decoder re-enters a high-probability phrase. The 3-gram repetition block catches the worst cases but not all
Untranslated idiom	*o'clock* transliterated as ଟଅ ଓକ୍ଲାରେ (or), ಒಂಬತ್ತು ಓಕ್ಲಾಕ್ಕೆ (kn)	The token is rare in the distilled corpus, so the student falls back to transliteration
Lexical slip	hi→en renders ढलना as "collapsed" instead of "set"; a Marathi round trip turns *sunset* into *sunrise*	Polysemy resolved incorrectly; a 52 M student has limited lexical disambiguation capacity
Dropped modifier	Kannada omits "fifteen percent" in a reported-speech sentence	Information loss under a long dependency; the weakest Kannada behaviour, matching its 0.763 ratio

A round-trip test (English to the target language and back) was also run. Hindi and Marathi round-trip almost perfectly. Kannada drifts noticeably, again matching its scorecard position. The qualitative and quantitative assessments agree throughout, which is itself a useful validation of the metric.

8.7 User Documentation

This section documents the system as delivered, sufficient for a user to install and operate it.

8.7.1 Installation

Python 3.11 or 3.12 is required, plus Node 18 or newer for the browser client. Once the models are fetched, everything runs offline.

```
# 1. Clone
git clone https://github.com/GeekyRiolu/SETU_v2.git
cd SETU_v2/SETU
```

```
# 2. Environment and inference dependencies
python -m venv .venv
source .venv/bin/activate              # Windows: .venv\Scripts\activate
pip install -e ".[quantize]"           # onnxruntime + optimum
pip install torch --index-url https://download.pytorch.org/whl/cpu
pip install transformers sentencepiece fastapi uvicorn

# 3. Fetch the trained models (~1.8 GB, from the public GitHub release)
bash scripts/fetch_models.sh           # extracts 22 pairs into models/
```

Listing 8.1: Installation on a fresh machine

The models are published as two archives on a public GitHub release, requiring no login. They are distributed this way rather than committed to the repository because 2.3 GB of binary artifacts do not belong in version control and individual files exceed GitHub's 100 MB per-file limit.

8.7.2 Running the System

```
# Start the REST service
PYTHONPATH=src:. uvicorn interfaces.rest.app:app --host 0.0.0.0 --port 8000
curl localhost:8000/models             # -> {"count": 22, ...}

# Command line, direct pair
PYTHONPATH=src:. python setu_cli.py --src hi --tgt en --text "<Hindi text>"

# Command line, Indic-to-Indic via the English pivot
PYTHONPATH=src:. python setu_cli.py --src hi --tgt bn --text "<Hindi text>"

# Browser client (separate terminal)
cd frontend && npm install && npm run dev      # http://localhost:3000
```

Listing 8.2: Starting the API and translating

8.7.3 Interface Reference

Table 8.10: REST API reference

Endpoint	Request	Response
`POST /translate`	`{source_lang, target_lang, text}`	`{translated_text, latency_ms, pivot, stub}`. `pivot` is `"en"` for a chained translation; `stub` true means passthrough
`GET /languages`	n/a	23 entries: ISO code, FLORES code, script, endonym
`GET /models`	n/a	`{count, models: [{pair, variant, size_mb}]}`
`GET /health`	n/a	`{status, stub}`

8.7.4 Supported Languages

All twenty-two scheduled languages are delivered, each paired both ways with English: Assamese (অসমীয়া), Bengali (বাংলা), Bodo (बर'), Dogri (डोगरी), Gujarati (ગુજરાતી), Hindi (हिन्दी), Kannada (ಕನ್ನಡ), Kashmiri (کأشُر), Konkani (कोंकणी), Maithili (मैथिली), Malayalam (മലയാളം), Manipuri (মৈতৈলোন্), Marathi (मराठी), Nepali (नेपाली), Odia (ଓଡ଼ିଆ), Punjabi (ਪੰਜਾਬੀ), Sanskrit (संस्कृतम्), Santali (ᱥᱟᱱᱛᱟᱲᱤ), Sindhi (سنڌي), Tamil (தமிழ்), Telugu (తెలుగు) and Urdu (اردو). Any pair among them translates through the English pivot, so a user may translate Hindi to Bengali or Tamil to Telugu without a direct model existing: 44 trained models serve 506 usable directions.

Every language in the picker is enabled; there are no disabled entries.

8.7.5 Interpreting the Output

Two fields in every response deserve attention. The `stub` field is `true` when no model could serve the request and the input was returned unchanged; a client must treat that as a failure, not a translation. The `pivot` field is `"en"` when the translation passed through English; such translations involve two model hops and may accumulate error from both, so a user comparing quality across pairs should know which they are looking at.

CHAPTER 9

CONCLUSION, APPLICATIONS AND FUTURE WORK

9.1 Conclusion

Setu set out to put high-quality Indic machine translation on a device that has no network connection, and it does. Twenty-two translation directions covering all twenty-two scheduled languages run from a 103.9 MB artifact each, complete a sentence in 105 to 202 milliseconds of commodity CPU time, and produce correct, idiomatic output with every network socket closed. Fourteen of those directions reach at least 80 % of their teacher's BLEU; the remaining eight fall between 0.755 and 0.787, and none is broken.

9.1.1 Outcome Against Each Objective

Table 9.1: Outcome against each stated objective

Obj.	Target	Outcome	Verdict
O1	Ratio ≥ 0.80	14 of 22 directions; mean 0.838, median 0.855, best 0.955	Partially met
O2	Latency < 500 ms	Mean 105–202 ms; only one p90 outlier	Met
O3	Size ≤ 200 MB	103.9 MB for every direction, 48 % under budget	Met
O4	Fully offline	Verified with all sockets disabled, for every direction	Met
O5	Test the distillation hypothesis	Hypothesis refuted under controlled conditions: SeqKD 17.09 BLEU against 11.10 for DPO	Met
O6	Coverage and extensibility	All 22 languages both ways; 44 models serve 506 directions via English pivot; adding a language is a configuration change	Met

9.1.2 Principal Contributions

An empirical finding, obtained by refuting the project's own hypothesis. For compact on-device Indic machine translation, sequence-level knowledge distillation from teacher output decisively outperforms preference-based distillation on human references: 17.09 against 11.10 BLEU at matched student size on identical data, an improvement of 54 %. The explanation is that reference noise rather than the training objective is the binding constraint. The evidence for that explanation is that reference-trained students plateau near a ratio of 0.66 regardless of data volume, whereas distilled students continue improving through 0.607, 0.764 and 0.806 as data scales. The result is strengthened, not weakened, by the fact that the DPO implementation works correctly: its margin accuracy reaches 1.0 and it improves chrF exactly as its chrF-based ranking predicts. The preference signal is learned faithfully; it is simply not the signal a 52-million-parameter student most needs.

A working offline system, not a prototype. Twenty-two quantised models are published on a public release, fetchable without a login, and served through a REST API, an offline progressive web application, a browser client and a command-line tool that all share one routing layer. A fresh machine can be running translations in five commands.

Quadratic coverage from linear storage. The English-pivot router turns 44 trained models into 506 usable directions at a measured two-hop cost of about 50 to 65 milliseconds. At 104 MB per model, training direct models for every pair would have required more than 11 GB of device storage for the same coverage.

A reproducible engineering record. The project documents three distinct training failure modes (empty output from a label-encoding mismatch, source-agnostic output from unmasked padding in the loss, and mid-training collapse from missing gradient clipping), each with its diagnosis and fix. It further documents two learning-rate collapses and their complete recovery, and establishes that supervised training loss (1.5 to 2.3 healthy, 3 to 4 collapsed) is a cheap early indicator available long before evaluation. These failures are routinely encountered and rarely published.

9.1.3 Limitations

The project states its limitations plainly, because several of them bound how the results should be read.

Eight directions fall short of the quality target, at ratios between 0.755 and 0.787. Their supervised losses cluster at 2.00 to 2.32, the upper end of the healthy band, which indicates under-convergence rather than instability.

The teacher caps the ceiling. The ungated distilled-200M teacher scores only 27 to 29 BLEU on these slices. A student at 0.806 of that is a student at roughly 80 % of a

model that is itself well below the gated one-billion-parameter flagship. Raising the ceiling requires teacher access, not more student training.

The reported ratios are not benchmark numbers. They come from an internal 500-sentence Samanantar development slice, and the teacher's own score varies between slices (27.08 to 28.91 for Hindi→English alone). The figures are internally comparable but should not be set beside published IndicTrans2 results. FLORES-200 rescoring is required for that and is the first item of future work.

BLEU is the wrong metric for several covered languages. English→Malayalam passes at a ratio of 0.863 on 5.2 BLEU, because Malayalam's agglutinative morphology defeats n-gram overlap. Some of the near-misses, particularly the Tamil and Malayalam directions, may already pass under COMET.

ARM latency is projected, not measured. All timings were taken on x86 CPUs. The target is ARM Cortex-A55 class hardware, and although the margin is large (roughly 150 ms against a 500 ms budget), the figure remains unverified on the intended silicon.

INT4 is INT4 in name only. The ONNX Runtime CPU execution provider has no true 4-bit kernel, so INT4 and INT8 land within 0.7 MB of each other. Real 4-bit storage would require a different runtime.

Pivot error is unquantified. Indic-to-Indic translations pass through two models and may accumulate error from both. The pivot was verified to function correctly and produce sensible output, but no systematic BLEU evaluation of pivoted pairs against direct references was performed.

9.2 Applications

Rural healthcare. A doctor or ASHA worker can converse with a patient across a language boundary on a handset with no connectivity. The privacy property matters as much as the offline one: a patient's described symptoms never leave the device.

Defence and border operations. Patrol units operating beyond cellular coverage can translate locally. Because there is no transmission, there is no signal to intercept and no dependency on infrastructure that may be unavailable or compromised.

Education. Multilingual tutoring tools can serve students in their own language at zero marginal cost per query, which is what makes deployment across many schools economically feasible.

Government service delivery. Forms, notices and entitlement documents can be translated at a gram panchayat counter without the counter needing a working internet connection or a commercial API subscription.

Disaster relief. Field workers can communicate with affected communities precisely during the window when networks are down, the moment when a cloud translator is guaranteed to be unavailable.

Privacy-sensitive documents. Legal, medical and financial text can be translated without disclosure to a third party, which is a structural guarantee here rather than a contractual one.

As a research artifact. The distillation pipeline is not Indic-specific. Any language family with a strong teacher and a noisy parallel corpus can use the same configuration-driven process, and the reported comparison gives practitioners a concrete data point on which training signal to choose.

9.3 Future Scope

9.3.1 Completing the Current Work

Rescore every direction on FLORES-200 with COMET. This is the highest priority, because it converts the internal ratios into publishable, comparable numbers, and because COMET may reclassify several agglutinative-language near-misses as passes. The infrastructure exists: `setu-eval` already implements FLORES-200 and IN22 loaders, COMET scoring and paired-bootstrap significance testing.

Close the eight near-misses. Three levers are available, in increasing order of cost: beam search at evaluation time, additional supervised epochs, and more distilled data. The under-convergence signature in the losses suggests all three would help.

Measure latency on real ARM hardware. A Raspberry Pi 4 or a Cortex-A55 board would convert a projected figure into a measured one.

Quantify pivot degradation. Evaluate pivoted Indic-to-Indic pairs against real references to establish how much quality the second hop costs.

9.3.2 Extending the System

Extend beyond the Eighth Schedule. All twenty-two scheduled languages are delivered, so the next step outward is the widely spoken non-scheduled languages and the major regional dialects. The architecture needs no change: the pipeline is verified pair-agnostic, and adding a language remains a configuration and data change rather than a rewrite.

Distil from a stronger teacher. The single most effective remaining lever on quality. At a ratio of 0.806 against a 27-BLEU teacher, the student is near its current ceiling; the gated one-billion-parameter IndicTrans2 flagship would raise the entire curve.

Consolidate directions with All-in-One KD. Both the All-in-One KD and Elastic Weight Consolidation mechanisms are implemented and tested but have not been exercised on full multilingual data. A single multilingual student would reduce device storage from $n \times 104$ MB to one artifact, at the risk of cross-lingual interference that per-direction training currently avoids.

Add a true 4-bit runtime. A backend with genuine INT4 kernels would roughly halve the artifact again, bringing a direction under 60 MB and making several languages resident on a phone simultaneously.

9.3.3 Longer-Term Directions

Speech input and output. The literature surveyed in Chapter 2 shows cascaded ASR-then-MT still outperforming end-to-end speech translation in low-resource settings [14]. An offline speech front-end over SETU's existing text engine would extend the system to users who cannot type in their own script, which, for several of the target populations, is the majority.

Revisit preference learning on clean targets. The refuted hypothesis tested DPO on noisy human references. The S3 configuration, sequence-level distillation followed by DPO on *teacher-derived* preferences, was designed but not run at scale. Whether preference learning helps on top of a strong distilled base remains genuinely open, and the existing DPO implementation makes it a training run rather than an engineering project.

Publish the finding. The SeqKD-versus-DPO comparison, the data scaling curve, the collapse analysis and the systems contribution together constitute a paper. The plan, with result tables already filled from the runs reported in Chapter 8, is maintained in the repository.

9.4 Closing Remarks

The project's most valuable outcome was not the one it set out to obtain. SETU was proposed as a demonstration that preference distillation would produce better compact translators, and the controlled experiment showed the opposite. The correct response to that was neither to discard the DPO work nor to present the system as though the hypothesis had held, but to report what was measured, rebuild the system around the method that won, and treat the refutation as the contribution.

What remains is a translator that fits in 104 megabytes, answers in a sixth of a second, works with the network switched off, covers eleven of India's scheduled languages in every direction between them, and never sends a user's words anywhere. The eight directions that fall short of target are documented as falling short. That, rather than a uniformly successful

scorecard, is what the work actually produced.

REFERENCES

[1] V. Sanh, L. Debut, J. Chaumond, and T. Wolf, "DistilBERT, a distilled version of BERT: smaller, faster, cheaper and lighter," *arXiv preprint* arXiv:1910.01108, 2019.

[2] B. Li, Z. Wang, H. Liu, Y. Jiang, Q. Du, T. Xiao, H. Wang, and J. Zhu, "Neural Machine Translation on Mobile Devices," *IEEE*, 2020.

[3] A. Mohammadshahi, V. Nikoulina, A. Bérard, C. Brun, J. Henderson, and L. Besacier, "SMaLL-100: Introducing Shallow Multilingual Machine Translation for Low-Resource Languages," *Springer*, 2020.

[4] J. Tiedemann, "The Tatoeba Translation Challenge – Realistic Data Sets for Low Resource and Multilingual MT," in *Proceedings of the Sixth Conference on Machine Translation (WMT)*, 2021, pp. 1174–1182.

[5] B. Li, Z. Wang, H. Liu, Y. Jiang, Q. Du, T. Xiao, H. Wang, and J. Zhu, "Shallow-to-Deep Training for Neural Machine Translation," in *Proceedings of the 59th Annual Meeting of the Association for Computational Linguistics (ACL)*, 2021.

[6] Z. Yang, R. Sun, and X. Wan, "Nearest Neighbor Knowledge Distillation for Neural Machine Translation," *IEEE*, 2022.

[7] C. Xu, R. Ye, Q. Dong, C. Zhao, T. Ko, M. Wang, T. Xiao, and J. Zhu, "Speech-to-Speech Translation Improvements with Self-Supervised Learning," *IEEE*, 2022.

[8] Y. Jia, Y. Ding, A. Bapna, C. Cherry, Y. Zhang, A. Conneau, and N. Morioka, "Direct Speech Translation," *Springer*, 2023.

[9] R. Rafailov, A. Sharma, E. Mitchell, C. D. Manning, S. Ermon, and C. Finn, "Direct Preference Optimization: Your Language Model is Secretly a Reward Model," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 36, 2023.

[10] Z. Miao, W. Zhang, J. Su, X. Li, J. Luan, Y. Chen, B. Wang, and M. Zhang, "All-in-One Knowledge Distillation Framework for Neural Machine Translation," in *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2023.

[11] Y. Zhang, P. Li, M. Sun, and Y. Liu, "Continual Knowledge Distillation for Neural Machine Translation," in *Proceedings of the 61st Annual Meeting of the Association for Computational Linguistics (ACL)*, 2023.

[12] J. Gala, P. A. Chitale, A. K. Raghavan, V. Gumma, S. Doddapaneni, A. Kumar M, and J. A. Nawale, "IndicTrans2: Towards High-Quality and Accessible Machine Translation Models for All 22 Scheduled Indian Languages," *Transactions on Machine Learning Research (TMLR)*, 2023.

[13] A. Jha, H. Y. Patil, S. K. Jindal, and S. M. N. Islam, "Multilingual Indian Language Neural Machine Translation System Using mT5 Transformer," *IEEE*, 2023.

[14] M. Zafar, A. Castaldo, N. Gajakos, P. Nayak, R. Haque, and A. Way, "SETU-DCU Submissions to IWSLT 2024 Low-Resource Speech-to-Text Translation Tasks," in *Proceedings of the 21st International Conference on Spoken Language Translation (IWSLT)*, 2024.

[15] G. Yang, J. Chen, W. Lin, and B. Byrne, "Direct Preference Optimization for Neural Machine Translation with Minimum Bayes Risk Decoding," in *Proceedings of the 2024 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL)*, 2024.

[16] K. Kashyap and S. K. Sarma, "Multilingual NMT System for English to Low-Resource Indic Languages – Assamese and Bengali," *IEEE*, 2024.

[17] S. B. Das, D. Panda, T. K. Mishra, B. K. Patra, and A. Ekbal, "Multilingual Neural Machine Translation System for Indic to Indic Languages," *ScienceDirect*, 2024.

[18] P. S. Bohra and K. Berwal, "LLM-based AI-Powered Offline Portable Transcriptor (OPT) – SURYA-TAC: A Tactical Speech-to-Speech Translation System," *IEEE*, 2024.

[19] B. Fu, M. Liao, K. Fan, C. Li, L. Zhang, Y. Chen, and X. Shi, "Latency-Aware Simultaneous Machine Translation Using Large Language Models," *Springer*, 2024.

[20] S. Gangwar and I. Naim, "Offline English-to-Hindi Statistical Machine Translation System," *ScienceDirect*, 2024.

[21] Y. Kim and A. M. Rush, "Sequence-Level Knowledge Distillation," in *Proceedings of the 2016 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2016, pp. 1317–1327.

[22] J. Kirkpatrick, R. Pascanu, N. Rabinowitz, J. Veness, G. Desjardins, A. A. Rusu, K. Milan, J. Quan, T. Ramalho, A. Grabska-Barwinska, D. Hassabis, C. Clopath, D. Kumaran, and R. Hadsell, "Overcoming Catastrophic Forgetting in Neural Networks," *Proceedings of the National Academy of Sciences*, vol. 114, no. 13, pp. 3521–3526, 2017.

[23] G. Ramesh, S. Doddapaneni, A. Bheemaraj, M. Jobanputra, R. AK, A. Sharma, S. Sahoo, H. Diddee, M. J, D. Kakwani, N. Kumar, A. Pradeep, K. Nagaraj, A. Kumar, P.

Kumar, M. S. Khapra, R. Dabre, P. Kumar, and A. Kunchukuttan, "Samanantar: The Largest Publicly Available Parallel Corpora Collection for 11 Indic Languages," *Transactions of the Association for Computational Linguistics (TACL)*, vol. 10, pp. 145–162, 2022.

[24] M. Post, "A Call for Clarity in Reporting BLEU Scores," in *Proceedings of the Third Conference on Machine Translation (WMT)*, 2018, pp. 186–191.

[25] M. Popović, "chrF: Character n-gram F-score for Automatic MT Evaluation," in *Proceedings of the Tenth Workshop on Statistical Machine Translation (WMT)*, 2015, pp. 392–395.

[26] R. Rei, C. Stewart, A. C. Farinha, and A. Lavie, "COMET: A Neural Framework for MT Evaluation," in *Proceedings of the 2020 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, 2020, pp. 2685–2702.

[27] A. Vaswani, N. Shazeer, N. Parmar, J. Uszkoreit, L. Jones, A. N. Gomez, Ł. Kaiser, and I. Polosukhin, "Attention Is All You Need," in *Advances in Neural Information Processing Systems (NeurIPS)*, vol. 30, 2017.

[28] M. Junczys-Dowmunt, R. Grundkiewicz, T. Dwojak, H. Hoang, K. Heafield, T. Neckermann, F. Seide, U. Germann, A. Fikri Aji, N. Bogoychev, A. F. T. Martins, and A. Birch, "Marian: Fast Neural Machine Translation in C++," in *Proceedings of ACL 2018, System Demonstrations*, 2018, pp. 116–121.

[29] T. Kudo and J. Richardson, "SentencePiece: A Simple and Language Independent Subword Tokenizer and Detokenizer for Neural Text Processing," in *Proceedings of EMNLP 2018, System Demonstrations*, 2018, pp. 66–71.

[30] ONNX Runtime Developers, "ONNX Runtime: Cross-platform Accelerated Machine Learning," 2021. [Online]. Available: `https://onnxruntime.ai`

[31] NLLB Team *et al.*, "No Language Left Behind: Scaling Human-Centered Machine Translation," *arXiv preprint* arXiv:2207.04672, 2022.

APPENDIX

A. Paper and Publication Details

The empirical work reported in Chapter 8 is being prepared for submission. The paper's structure and the tables it fills are given below; every number in Tables A.1 and A.2 is drawn from the runs reported in this document.

Working title. *Sequence-Level Distillation Beats Preference Distillation for Compact On-Device Indic Machine Translation*

Contributions claimed.

1. A controlled comparison of four training signals for a compact Indic translation student at matched size, showing sequence-level knowledge distillation outperforming preference/DPO distillation by 6.0 BLEU ($+54\,\%$) and identifying reference noise as the binding constraint.
2. A data-scaling analysis showing that reference-trained students plateau near a ratio of 0.66 while distilled students continue improving through 0.607, 0.764 and 0.806.
3. A systems contribution: 22 directions at 103.9 MB and under 500 ms on CPU, fully offline, with English-pivot routing turning $2n$ models into n^2 directions.
4. A reproducible failure analysis covering three training failure modes and two learning-rate collapses, with training loss established as an early collapse indicator.

Table A.1: Main comparison (Hindi→English, 100k sentences, matched 52.6 M student)

System	Training signal	BLEU	% of teacher
Teacher (IndicTrans2 dist-200M)	n/a	28.16	100.0
S0: SFT on references	Human references	10.95	38.9
S1: SeqKD	**Teacher 1-best**	**17.09**	**60.7**
S2: DPO-distill	References, then DPO	11.10	39.4
S3: SeqKD + DPO	Teacher 1-best, then DPO	*designed; not run at scale*	

Table A.2: Quantisation (Hindi→English student)

Precision	Size (MB)	BLEU	p90 latency
FP32 ONNX	410.9	n/a	n/a
INT8	104.6	18.3	335 ms
INT4	**103.9**	**19.3**	**191 ms**

Target venues. ACL / EMNLP / NAACL (main or Findings); WMT; IJCNLP-AACL; or a workshop on efficient NLP or multilingual NLP.

Required before submission. Rescoring on FLORES-200 devtest with COMET and paired-bootstrap significance testing; the S3 run at scale; ARM latency measurement on real hardware.

B. Reproducing the Results

B.1 Training One Direction

Every result in Chapter 8 is reproducible with the commands below. `PAIR` uses script-qualified FLORES codes.

```
# One direction, resumable, on a GPU VM (stage markers skip completed steps)
PAIR=hin_Deva-eng_Latn LIMIT=500000 bash vm/setu_vm_train.sh

# Or stage by stage:
setu-data     --pair $PAIR --limit 500000      # clean + deduplicate the corpus
setu-distill  --pair $PAIR --beams 1           # teacher 1-best targets (greedy)
python scripts/train_full.py --pair $PAIR \
       --train-corpus distilled --skip-dpo \
       --limit 500000 --dev-size 500           # SeqKD student training
setu-quantize --pair $PAIR                     # ONNX export + INT8/INT4
setu-eval     --pair $PAIR --teacher           # BLEU / chrF / ratio
```

Listing 9.1: Training a single direction end to end

B.2 Reproducing the Head-to-Head Comparison

Keep `--limit` identical across all four systems, or the comparison is not matched.

```
setu-data    --limit 102000
setu-prefs   --max-entries 8000                # only needed for the DPO variants
setu-distill --limit 100000 --beams 1

# S1: SeqKD (teacher targets, no DPO)
python scripts/train_full.py --train-corpus distilled --skip-dpo \
       --limit 100000 --dev-size 500
cp checkpoints/<pair>/train_report.json report_S1_seqkd.json

# S0 and S2: reference targets, with DPO (report carries both sft_eval and dpo_eval)
python scripts/train_full.py --train-corpus processed \
       --limit 100000 --dev-size 500
cp checkpoints/<pair>/train_report.json report_S2_dpo.json
```

Listing 9.2: The four-system comparison of Figure 8.1

Read S1 from `report_S1_seqkd.json` under `sft_eval`; read S0 from `report_S2_dpo.json` under `sft_eval`; read S2 from the same file under `dpo_eval`. Evaluation is on real human references in every case, enforced by the pipeline.

B.3 Key Configuration Values

Table B.1: Configuration used for the shipped models

Parameter	Value	Note
Encoder / decoder layers	6 / 6	Marian encoder–decoder
Hidden size / FFN size	512 / 2048	52.6 M parameters total
Attention heads	8	
Vocabulary size	16,000	32,000 caused degenerate decoding at this data scale
Maximum sequence length	256	Longer inputs are truncated, not rejected
Dropout	0.2	Raised after overfitting was observed
Epochs	6	10 epochs over less data memorised
Batch size (SFT / DPO)	32 / 16	DPO holds policy and reference simultaneously
Learning rate (SFT)	3×10^{-4}	**Lowered from** 5×10^{-4} **after two collapses**
Warmup steps	4000	Capped at 10 % of the run
Label smoothing	0.1	
Weight decay	0.01	
Gradient clipping	1.0	The critical stabiliser
DPO β	0.1	Reported baseline only; not used for shipped models
Training sentences	≈497,000	From `LIMIT=500000` after cleaning
Development set	500 sentences	Held out, real human references

B.4 Published Artifacts

- **Source code:** `https://github.com/GeekyRiolu/SETU_v2`
- **Trained models:** the `phase1-models` GitHub release: two archives of approximately 897 MB each, public, no login required, fetched by `scripts/fetch_models.sh`.
- **Per-direction evaluation reports:** `vm/out/report_*.json`, one per direction, containing BLEU, chrF, teacher score, ratio, final training loss and training-set size.
- **Quantisation reports:** `models/<pair>/quantize_report.json`, containing size and latency at each precision.
- **Qualitative samples:** `vm/out/sample_translations.md` and `vm/out/complex_translations.md`.
- **Engineering log:** `docs/PROGRESS.md` and `docs/TRAINING_LOG.md`, the dated session records from which Chapter 8 is reconstructed.

GLOSSARY

Term	Meaning
AIO-KD	All-in-One Knowledge Distillation. Distilling into several student models, or across several translation directions, within a single training framework
ARM Cortex-A55	A low-power 64-bit ARM CPU core widely used in mid-range smartphones and single-board computers; SETU's reference deployment target
ASHA worker	Accredited Social Health Activist, a community health worker in India's public health system; a primary intended user of an offline translator
BLEU	Bilingual Evaluation Understudy. An n-gram overlap metric between a candidate translation and one or more references. Structurally low for agglutinative languages
BPCC	Bharat Parallel Corpus Collection. The large Indic parallel corpus released with IndicTrans2; access-gated on Hugging Face
chrF	Character n-gram F-score. A translation metric operating on characters rather than words, more robust than BLEU for morphologically rich languages
COMET	A neural, learned translation-quality metric that correlates better with human judgement than overlap metrics
Catastrophic forgetting	The loss of previously learned capability when a neural network is trained on a new task
Distillation	Training a small student model to reproduce the behaviour of a large teacher model
DPO	Direct Preference Optimization. Preference-based training expressed as a classification loss over preferred and dispreferred pairs, requiring neither a reward model nor reinforcement learning
Endonym	A language's name in its own language and script, e.g. हिन्दी for Hindi, ಕನ್ನಡ for Kannada
English pivot	Translating between two Indic languages by chaining source →English and English→target models, used where no direct model exists

Term	Meaning
EWC	Elastic Weight Consolidation. A continual-learning method that penalises changes to parameters important for previously learned tasks, weighted by the Fisher information
FLORES-200	A standard multilingual translation benchmark covering 200 languages; the source of the script-qualified language codes used throughout SETU
Fisher information	A measure of how sensitive a model's output distribution is to each parameter; used by EWC to decide which parameters to protect
Gated (Hugging Face)	A model or dataset requiring an authenticated account that has accepted its terms; returns HTTP 401 otherwise
Gradient clipping	Bounding the norm of the gradient before an optimiser step, preventing a single large update from destabilising training
IN22	An Indic translation benchmark released alongside IndicTrans2
IndicTrans2	AI4Bharat's open translation model family covering all 22 scheduled Indian languages; SETU's teacher
INT4 / INT8	4-bit and 8-bit integer quantisation. ONNX Runtime's CPU provider lacks a true 4-bit kernel, so the two sizes are nearly identical in practice
Label smoothing	Replacing hard one-hot targets with a softened distribution, reducing overconfidence
Marian	A fast neural machine translation architecture and toolkit; SETU's student uses the Marian encoder–decoder
Margin accuracy	In DPO, the fraction of preference pairs on which the trained policy ranks the preferred candidate above the dispreferred one
NFC	Unicode Normalization Form C (canonical composition), applied to corpus text so that equivalent sequences are represented identically
NMT	Neural Machine Translation
ONNX	Open Neural Network Exchange. A portable model format; ONNX Runtime is its cross-platform execution engine
p50 / p90	The 50th and 90th percentile of a latency distribution

Term	Meaning
PAD masking	Setting label positions for padding tokens to -100 so the loss function ignores them; omitting this produces a misleadingly low loss
PERT	Programme Evaluation and Review Technique. A scheduling method using optimistic, most-likely and pessimistic duration estimates
Preference pair	A training example consisting of a source, a preferred translation, a dispreferred translation and the quality gap between them
PWA	Progressive Web Application. A browser application that installs and continues to function offline via a service worker
Quantisation	Reducing the numeric precision of model weights to shrink the artifact and speed up inference
Ratio (student/teacher)	Student BLEU divided by teacher BLEU on the same evaluation set. SETU's primary quality metric, because it is comparable across languages where absolute BLEU is not
sacreBLEU	A BLEU implementation producing reproducible, comparable scores with an explicit signature
Samanantar	A large, ungated parallel corpus for 11 Indic languages, mined from the web; SETU's training data
Scheduled languages	The 22 languages listed in the Eighth Schedule of the Constitution of India
SentencePiece	A language-independent subword tokenizer that operates on raw text without pre-tokenisation
SeqKD	Sequence-Level Knowledge Distillation. Training a student on the teacher's decoded output rather than on human reference translations
SFT	Supervised Fine-Tuning. Standard cross-entropy training on target sequences
SLM	Small Language Model. A compact model intended for on-device execution
Stub engine	SETU's passthrough engine, returned when no model exists for a pair; always flagged `stub: true` so it is never mistaken for a translation

Term	Meaning
Teacher wall	The design rule, enforced by an automated test, that only one module may import the teacher's dependencies
Temperature sampling	Flattening a sampling distribution so that low-resource directions are not swamped by high-resource ones
vGPU	A software-partitioned slice of a physical GPU. SETU trained on an A40-16Q slice, whose allocator lacks the CUDA virtual-memory APIs
Warmup	Increasing the learning rate linearly from zero at the start of training, stabilising an unpretrained model
ZWJ / ZWNJ	Zero-Width Joiner and Zero-Width Non-Joiner. Invisible characters that control conjunct formation in Indic scripts and must survive normalisation